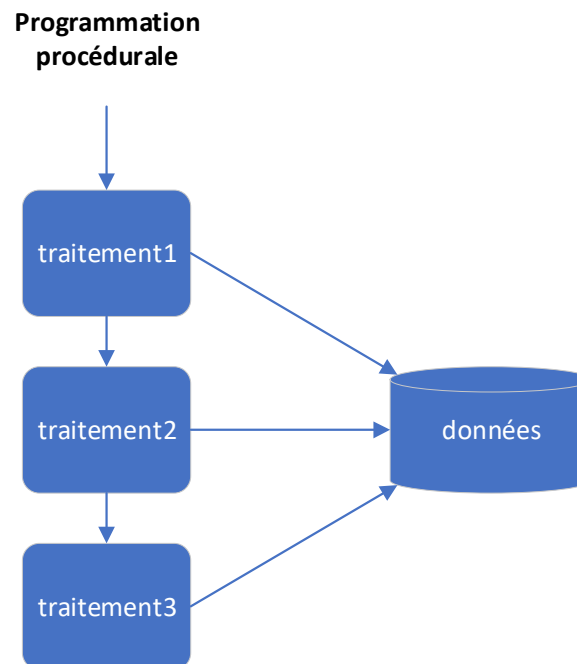


Introduction à la programmation orientée objet

Approche classique de développement d'un programme

- La méthode **classique**, également appelée **procédurale**, est axée sur la **conception descendante** (TopDown): on décompose le traitement principal du problème en sous-traitements élémentaires indépendants correspondant à des sous-programmes de réalisation simple et facile. Chaque sous-programme est soit une procédure, soit une fonction.
- **L'unité de décomposition est donc le traitement** (l'action).
- **Les données et les traitements** apparaissent comme des entités **séparées** dans un programme.
- Les données sont accessibles et modifiables par le code contenu dans les procédures et fonctions par le biais des paramètres.



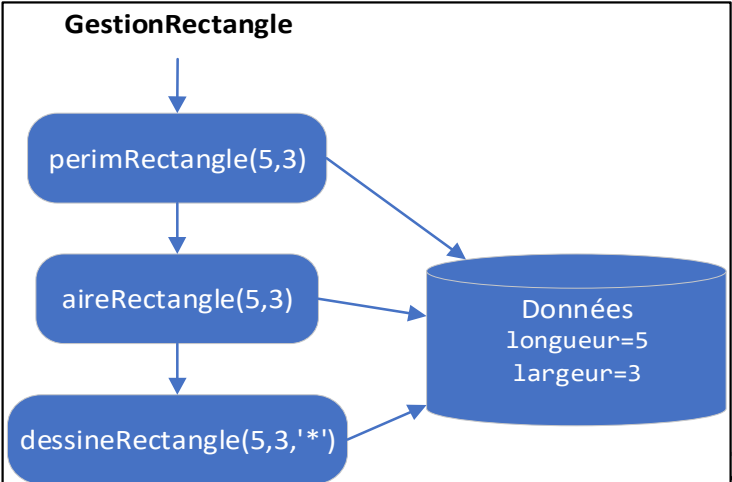
Approche classique de développement d'un programme: exemple

On veut calculer l'aire et le périmètre d'un rectangle à partir de sa longueur et sa largeur. On veut afficher une représentation d'un rectangle en utilisant un symbole, par exemple '*'. Les sous-traitements sont:

- Le calcul du périmètre: `perimRectangle()`
- Le calcul de l'aire : `aireRectangle()`
- L'affichage du rectangle: `dessineRectangle()`

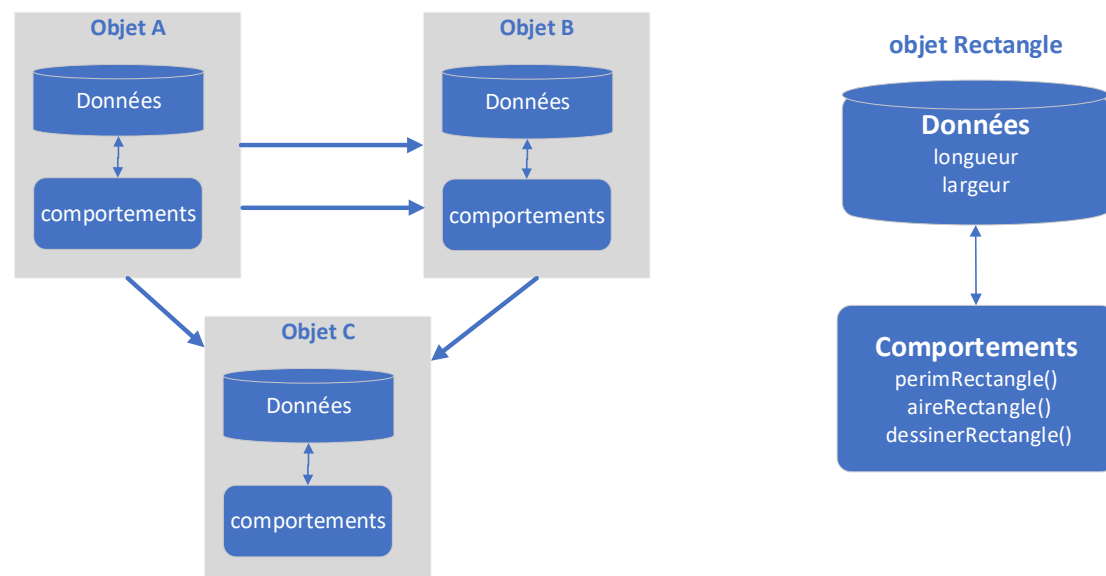
```
public class GestionRectangle {
    public static int perimRectangle(int pLong, int pLarg) {
        int perim;
        perim = (pLong + pLarg) * 2;
        return perim;
    }
    public static int aireRectangle(int pLong, int pLarg) {
        return pLong * pLarg;
    }
    public static void dessineRectangle(int pLong, int pLarg, char pSymbole) {
        System.out.println("Représentation graphique du rectangle:");
        for (int i = 0; i < pLong; i++) {
            for (int j = 0; j < pLarg; j++) {
                System.out.print(pSymbole);
            }
            System.out.println();
        }
    }
    public static void main(String[] args) {
        int longueur, largeur;
        char symbole;
        Scanner clavier = new Scanner ((System.in));
        System.out.print("Entrez la longueur du rectangle: ");
        longueur = clavier.nextInt();
        System.out.print("Entrez la largeur du rectangle: ");
        largeur = clavier.nextInt();
        System.out.println("Le périmètre est: " + perimRectangle(longueur, largeur));
        System.out.println("L'aire est: " + aireRectangle(longueur, largeur));
        System.out.print("Entrez le symbole à utiliser pour dessiner le rectangle: ");
        symbole = clavier.next().charAt(0);
        dessineRectangle(longueur, largeur, symbole);
    }
}
```

Entrez la longueur du rectangle: 5
Entrez la largeur du rectangle: 3
Le périmètre est: 16
L'aire est: 15
Entrez le symbole à utiliser pour dessiner le rectangle: *
Représentation graphique du rectangle:



Approche de développement d'un programme orienté objet

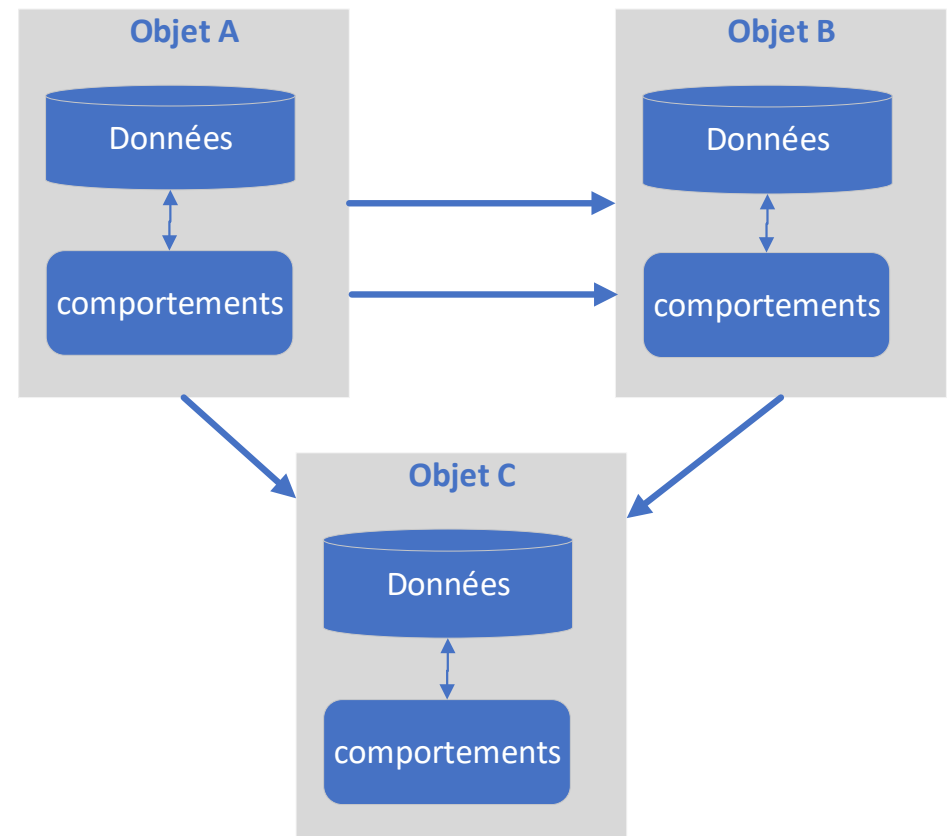
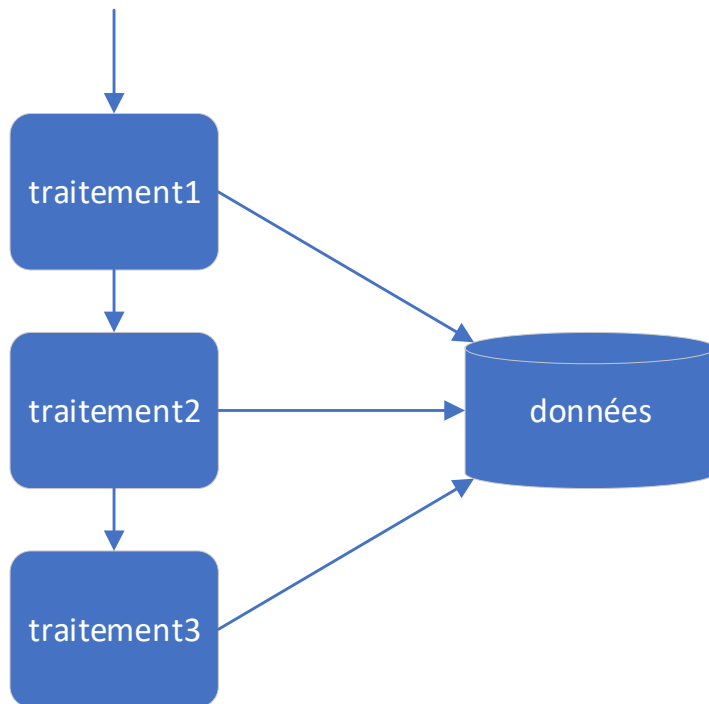
- La technique d'analyse de programmation orientée objet consiste d'abord à déterminer **les entités (objets)** appartenant au problème à modéliser ainsi que les opérations qui leur sont applicables et ensuite à se préoccuper de l'enchaînement de ces opérations. On parle d'analyse de **conception ascendante**.
- **L'unité de décomposition est l'objet.**
- En d'autres termes, l'approche orientée objet consiste en:
 1. Identifier les objets du système. Chaque objet possède des informations qui le caractérisent (données) et un ensemble de compétences ou comportements (fonctions). Les données et les comportements sont réunis dans la même entité.
 2. Faire collaborer ces objets pour qu'ils accomplissent la tâche voulue : les objets interagissent en s'envoyant des messages (appels de fonctions)
- En programmation orientée objet, les données et les traitements sont regroupés en une seule et même entité qui est l'objet.



Approche de développement procédurale VS objet

procédurale

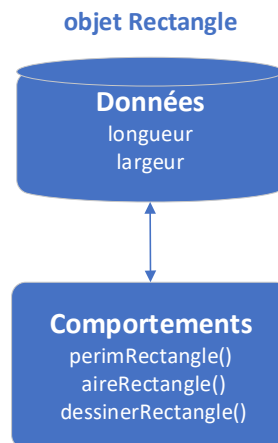
objet



Objet?

Les objets sont à la base de la programmation orientée objet.

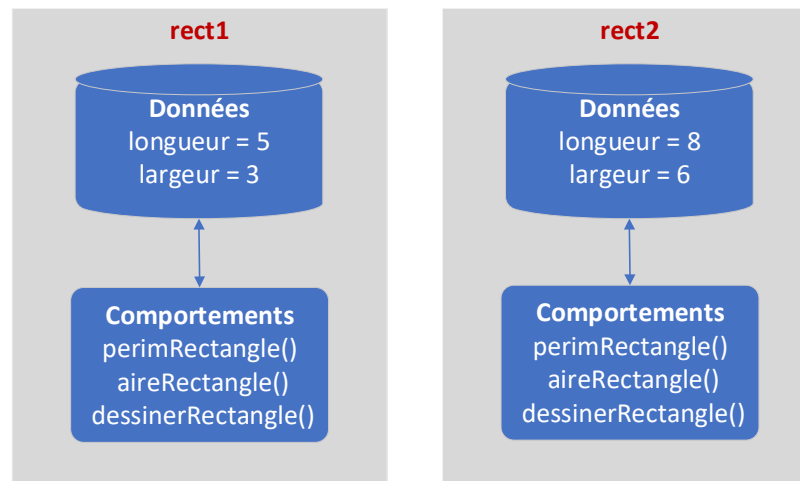
- Un **objet** possède **des attributs** (ou propriétés) qui définissent son état et des fonctionnalités permettant d'effectuer des **opérations** sur l'objet .
- Les attributs sont représentés par des variables d'instances et les fonctionnalités par des méthodes.
Un objet = attributs + méthodes
- Exemple:
 - Un objet rectangle possède une longueur et une largeur. Ces caractéristiques sont des attributs de l'objet. Elles sont représentées par des variables d'instance: longueur et largeur.
 - Les opérations qu'on est capable d'effectuer sur un objet rectangle sont: calculer son aire, calculer son périmètre et dessiner le rectangle en utilisant un symbole. Ces opérations seront représentées par des méthodes: `perimRectangle()`, `aireRectangle()`, `dessinerRectangle()`.



Occurrence d'un objet

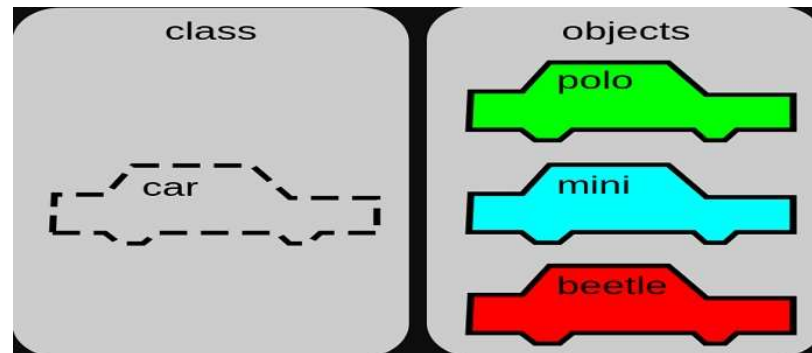
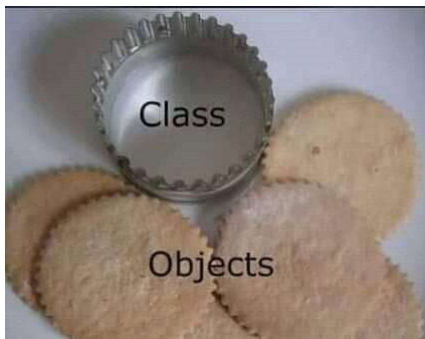
- On peut avoir **plusieurs occurrences** d'un objet. Chaque objet possède une valeur pour ces différents attributs qui font que cet objet est unique (état de l'objet).
 - L'objet rect1 aura une longueur de 5 et une largeur de 3.
 - L'objet rect2 aura une longueur de 8 et une largeur de 6.
- Chaque occurrence d'un objet possède le même comportement:
 - l'objet rect1, tout comme l'objet rect2, pourra calculer son aire, calculer son périmètre et se dessiner en utilisant un symbole.

2 occurrences de Rectangle

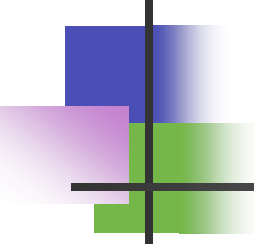


Classe?

- Chaque objet est décrit par **une classe qui contient toutes les caractéristiques qu'un objet peut avoir**. Il s'agit du **plan** à partir duquel on va pouvoir construire des objets.
- Autrement dit, les classes sont des **modèles** (comme les emporte-pièces) servant à faire des objets (comme les biscuits).



- **Tout objet appartient à une classe** qui représente **un type d'objet**. On dit aussi qu'un objet est une **instance** d'une classe.
- Exemples:
 - Rectangle est une classe (type d'objet). Les objets **rect1 et rect2 sont de type Rectangle**. On peut dire également que **rect1 et rect2 appartiennent à la classe Rectangle** ou sont **deux instances de la classe Rectangle**.
 - Car est une classe (type d'objet). Les objets polo, mini et beetle sont des objets de type Car. On peut dire également que ces 3 objets appartiennent à la classe Car ou sont 3 instances de la classe Car.

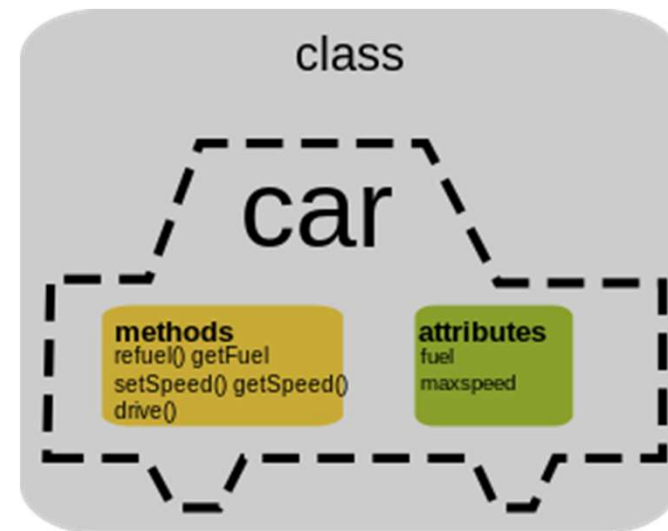


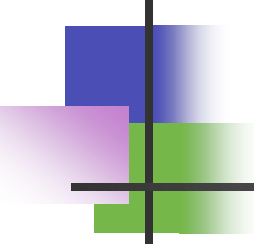
Exercice1

1. Quels sont les attributs (propriétés) et les méthodes (opérations) que peut avoir un objet de type Car (une voiture) ?
2. Décrivez l'état (ensemble des attributs) de deux occurrences de type Car.

Exercice1: corrigé

1. Quels sont les attributs (propriétés) et les méthodes (opérations) que peut avoir un objet de type Car (une voiture) ?
 - Attribut :
 - marque, modèle, immatriculation, couleur, consommation (nombre de litres / 100 km), vitesse maximale
 - Méthodes:
 - avancer, reculer, accélérer, tomber en panne, faire le plein d'essence
2. Décrivez l'état (ensemble des attributs) de deux occurrences de la classe Car.
 - Première occurrence: polo:
 - Marque: Toyota
 - Modèle: Corolla
 - Immatriculation: Y1B2C3
 - Couleur: verte
 - Consommation: 6
 - vitesse maximale: 180
 - Deuxième occurrence: beetle
 - Marque: Hundai
 - Modèle: Elantra
 - Immatriculation: Z9S2C3
 - Couleur: rouge
 - Consommation: 7
 - vitesse maximale: 170





Déclaration d'une classe

- Une classe, qui définit un type d'objet, est composée de la structure suivante :
 - Un nom.
 - **Des attributs et des méthodes** contenus **dans une même capsule**.
- Pour déclarer une nouvelle classe, il suffit d'utiliser le mot-clé `class`, suivi du nom de la classe. En java, par convention, **le nom de la classe commence par une lettre majuscule**.
- Si on veut que notre classe soit accessible par les autres classes, on précède le mot `class` par `public`.

```
public class NomDeLaClasse {
```

```
    Déclaration des variables d'instance (attributs)
```

```
    Déclaration des méthodes d'instance (fonctionnalités)
```

```
}
```

- Notes:
 - La déclaration des attributs peut être placée avant ou après la déclaration des méthodes.
 - Le terme **instance** signifie **spécifique à chaque occurrence (instance) de la classe**. Nous verrons dans le cours suivant qu'il existe des attributs et des méthodes de classe.

Les attributs d'instance

- Les **attributs** d'instance décrivent les informations caractéristiques des objets de la classe. Ils sont aussi appelés communément **champs d'instance**, **variables d'instance** ou **données membres de la classe**.
- Ils sont définis à l'aide d'instructions de déclaration de variables. Ces variables peuvent être de type simple (int, float, char...) ou de type composé (String, tableau...).

```
public class Rectangle {  
  
    private int longueur;  
    private int largeur;  
  
    public int perimRectangle() {  
        int perim;  
        perim = (longueur + largeur) * 2;  
        return perim;  
    }  
    public int aireRectangle() {  
        return longueur * largeur;  
    }  
    public void dessineRectangle(char caractere) {  
        for (int i = 0; i < longueur; i++) {  
            for (int j = 0; j < largeur; j++) {  
                System.out.print(caractere);  
            }  
            System.out.println();  
        }  
    }  
}
```

2 attributs d'instance
accessibles par toutes
les méthodes de la
classe .

Accès direct aux
attributs de la classe

Les méthodes d'instance

- Les méthodes d'instance représentent les traitements et comportements des objets de la classe. Ce sont les méthodes qui agissent sur les données.
- Elles sont aussi appelées **méthodes membres de la classe** ou **fonctions**.
- Elles se construisent comme de simples méthodes: composées d'un en-tête et d'instructions.
- Attention! Une variable locale à une méthode n'est pas un attribut d'une classe.

```
public class Rectangle {  
    private int longueur;  
    private int largeur;  
  
    public int perimRectangle() {  
        int perim;  
        perim = (longueur + largeur) * 2;  
        return perim;  
    }  
    public int aireRectangle() {  
        return longueur * largeur;  
    }  
    public void dessineRectangle(char caractere) {  
        System.out.println("Représentation graphique du rectangle:");  
        for (int i = 0; i < longueur; i++) {  
            for (int j = 0; j < largeur; j++) {  
                System.out.print(caractere);  
            }  
            System.out.println();  
        }  
    }  
}
```

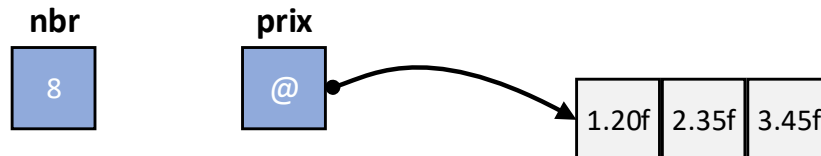
perim est une variable locale à la méthode .
Ce n'est pas un attribut.

Méthode avec paramètre pour recevoir une donnée de l'extérieur de l'instance

La variable en mémoire (rappel)

- Il y a deux types de variables: les variables de type primitif (byte, short, int, long, float, double, boolean, char) et les variables de type objet (String, les tableaux, Scanner...).
- **Une variable de type primitif (simple) stocke sa valeur en mémoire.**
- **Une variable de type objet ne stocke pas directement l'objet, mais une référence** (adresse mémoire) vers l'objet, qui lui, est stocké ailleurs dans la mémoire. Une variable de ce type est appelée variable de référence.

```
int nbr = 8; //variable de type primitif(simple)
float[] prix = {1.20f, 2.35f, 3.45f} ; //variable de type objet
```

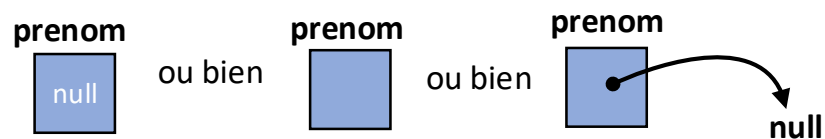


La valeur null

- En java, la valeur **null** est utilisée pour indiquer **qu'aucune valeur n'est affectée à une variable de référence**.
- Lorsqu'une variable de référence (un objet) est déclarée, mais n'est pas encore initialisée, elle contient la valeur de référence **null** (elle ne pointe vers nul part)

```
String prenom;
```

```
// prenom ne contient aucune référence: l'objet prenom n'existe pas, il doit  
// être créé à l'aide de l'opérateur new.
```



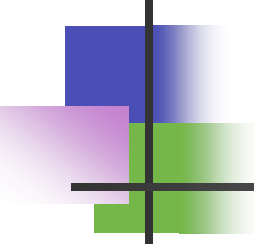
Construction des objets (1/6)

La classe est un modèle qui s'apparente à la déclaration d'un nouveau type de données.

- On peut **déclarer une variable du type de cette classe**. Cette variable est appelée **objet** ou **instance** de la classe.
- L'opération permettant de créer un objet à partir d'une classe s'appelle **instanciation**.
- Exemple:
 - `Rectangle rect = new Rectangle (8,7) // rect est une instance de Rectangle`
- Pour pouvoir instancier des rectangles comme ci-haut, nous ajouterons une méthode spéciale appelée **constructeur** à l'intérieur de la classe. C'est une **méthode publique sans le mot void qui porte le même nom que la classe**.

```
public class Rectangle {  
  
    private int longueur;  
    private int largeur;  
  
    public Rectangle(int longu, int larg) {  
        longueur = longu;  
        largeur = larg;  
    }  
    public int perimRectangle() {  
        //...  
    }  
    public int aireRectangle() {  
        return longueur * largeur;  
    }  
    public void dessineRectangle(char caractere) {  
        //...  
    }  
}
```

Constructeur



Construction des objets (2/6)

- La **construction**, aussi appelée **instanciation** ou **création**, d'un objet à partir d'une classe nécessite 2 étapes:
 - 1- la déclaration de l'objet
 - 2- l'initialisation de l'objet
- Exemple: construction d'un objet de la classe Rectangle

```
Rectangle rect1; // déclaration
rect1 = new Rectangle(5,3) // initialisation
```
- On peut rassembler ces deux étapes sur une seule ligne.

```
Rectangle rect1 = new Rectangle(5,3) // déclaration et initialisation
```
- Lorsqu'une variable est déclarée, mais n'est pas encore initialisée, elle contient la valeur de référence `null`.

Construction des objets (3/6)

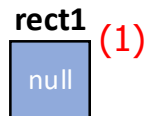
Les 2 étapes de construction d'un objet :

1- La déclaration de l'objet:

Elle consiste en la création d'une variable dont le type correspond au nom de la classe de l'objet (1),

```
Rectangle rect1;
```

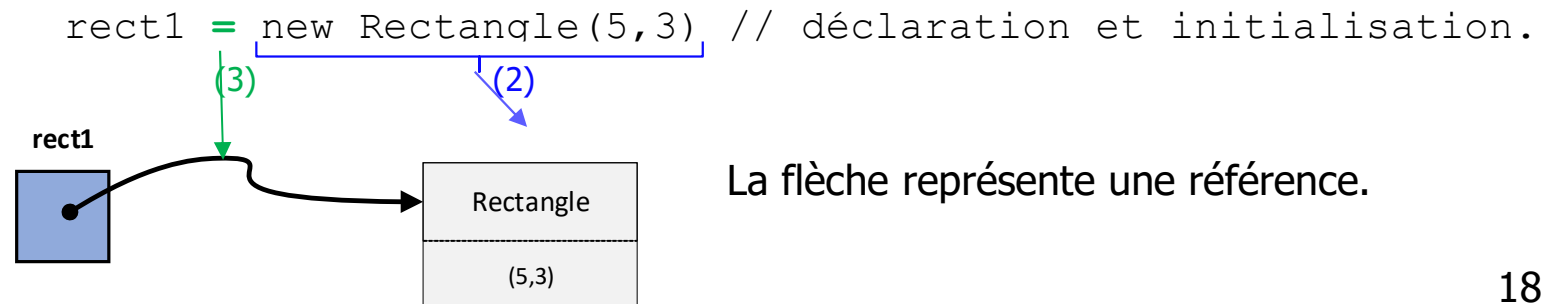
- Après exécution de cette ligne de code, une nouvelle variable est créée, mais ne contient aucune référence (elle contient la valeur `null`).



2- L'instanciation de l'objet en utilisant l'opérateur new.

```
rect1 = new Rectangle(5,3)
```

- L'exécution de cette instruction par la machine virtuelle Java se fait en deux temps:
 - L'opérateur new, suivi du constructeur à sa droite, crée un nouvel objet en mémoire (une instance de la classe Rectangle) (2)
 - L'opérateur new renvoie la référence (adresse mémoire) de l'objet créé, et celle-ci est affectée à la variable rect1 grâce à l'opérateur d'affectation = (3)

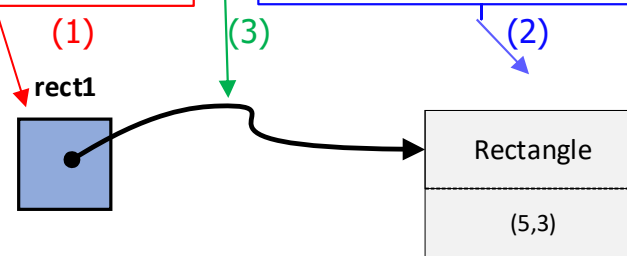


La flèche représente une référence.

Construction des objets (4/6)

Il est possible de déclarer et d'initialiser en une seule instruction comme pour les variables de type simple:

```
Rectangle rect1 = new Rectangle(5,3) // déclaration et initialisation.
```



La flèche représente une référence.

Construction des objets (5/6): exemple

```
public class Rectangle {  
  
    private int longueur;  
    private int largeur;  
  
    public Rectangle(int longu, int larg) {  
        longueur = longu;  
        largeur = larg;  
    }  
    public int perimRectangle() {  
        int perim;  
        perim = (longueur + largeur) * 2;  
        return perim;  
    }  
    public int aireRectangle() {  
        return longueur * largeur;  
    }  
    public void dessineRectangle(char caractere) {  
        System.out.println("Représentation graphique du rectangle:");  
        for (int i = 0; i < longueur; i++) {  
            for (int j = 0; j < largeur; j++) {  
                System.out.print(caractere);  
            }  
            System.out.println();  
        }  
    }  
}
```

■ On peut créer des instances de la classe Rectangle:
Rectangle rect1 = new Rectangle(9, 3);
Rectangle rect2 = new Rectangle(7, 2);

rect1 et rect2 sont 2
instances de la classe
Rectangle

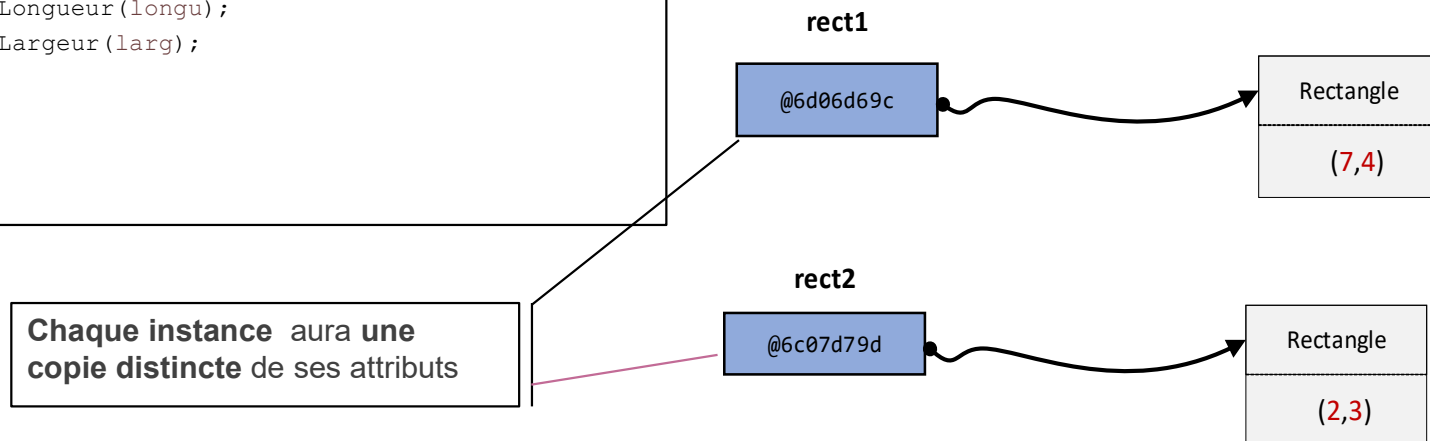
Classe Rectangle

Construction des objets (6/6)

- Chaque nouvel objet de la classe Rectangle initialisé **aura une copie distincte de ses attributs ou variables** d'instances.
- Dans l'exemple ci-dessous, `rect1` aura comme longueur la valeur 7 et comme largeur la valeur 4, alors que `rect2` aura comme longueur la valeur 2 et comme largeur la valeur 3.

```
public class Rectangle {  
  
    // Attributs(ou ou variables d'instances)  
    private int longueur;  
    private int largeur;  
  
    public Rectangle(int longu, int larg) {  
        setLongueur(longu);  
        setLargeur(larg);  
    }  
  
    ...  
}
```

```
public static void main(String[] args) {  
    Rectangle rect1, rect2;  
    rect1 = new Rectangle(7, 4);  
    rect2 = new Rectangle(2, 3);  
}
```



Exercice2

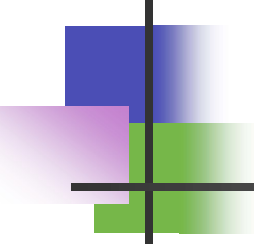
- Créez les objets Cours zc6 et zh5 ci-dessous.
- Voici les étapes à suivre:
 1. Créez la classe des objets. Donnez un nom significatif, par exemple **Cours**. Ajoutez les attributs et un constructeur.
 2. Créez deux instances zc6 et zh5 de cette classe en utilisant l'opérateur new.

zc6	zh5
Code: 420-ZC6-MO	Code: 420-ZH5-MO
Titre: Algorithmie et programmation	Titre: Base de données
Session: 1	Session: 3
Nombre d'heures par semaine : 6	Nombre d'heures par semaine: 5

- Ajoutez une méthode qui affiche les informations d'un cours à la console sous le format ci-dessous:

```
Informations du cours :420-ZH5-MO
*****

Titre : Base de données
Session : 3
Nombre d'heures par semaine : 5
```
- Ajoutez une méthode qui calcule le nombre d'heures d'un cours par session sachant qu'une session a 15 semaines.



Exercice2 : corrigé

```
public class Cours {
    private String code;
    private String titre;
    private int session;
    private int nbHeures;
    // constructeur
    public Cours(String pCode, String pTitre, int pSession, int pNbHeures) {
        code = pCode;
        titre = pTitre;
        session = pSession;
        nbHeures = pNbHeures;
    }
    public void afficher() {
        System.out.println("Informations du cours : " + code);
        System.out.println("*****");
        System.out.println("Titre : " + titre);
        System.out.println("Session : " + session);
        System.out.println("Nombre d'heures par semaine : " + nbHeures);
    }
    public int heuresParSession() {
        return 15 * nbHeures;
    }
}
```

```
public class GestionCours {

    public static void main(String[] args) {
        //création de l'objet zc6 en une seule ligne
        Cours zc6 = new Cours("420-ZC6-MO", "Algorithmie et programmation", 1, 6);
        //création de l'objet zc6 en deux lignes
        Cours zh5;
        zh5 = new Cours("420-ZH5-MO", "Base de données", 3, 5);
        zh5.afficher();
        System.out.println("Nombre d'heures par session : " + zh5.heuresParSession());
    }
}
```

Visibilité des membres d'une classe : public et private

La visibilité des membres (attributs et méthodes) d'une classe définit les endroits d'où ils peuvent être utilisés.

- Les méthodes et attributs **privés** (`private`) d'une classe ne sont accessibles que par les méthodes de la classe.
- Les méthodes et attributs **publics** (`public`) d'une classe sont accessibles de l'extérieur de la classe.
- Nous verrons plus tard dans le cours d'autres types de visibilité.

```
public class Rectangle {  
  
    private int longueur; // cet attribut n'est pas accessible à l'extérieur de la classe  
    private int largeur; //cet attribut n'est pas accessible à l'extérieur de la classe  
  
    public Rectangle(int longu, int larg) {  
        longueur = longu;  
        largeur = larg;  
    }  
  
    public int perimRectangle() { // méthode accessible à l'extérieur de la classe  
        int perim;  
        perim = (longueur + largeur) * 2;  
        return perim;  
    }  
  
    public int aireRectangle() { // méthode accessible à l'extérieur de la classe  
        return longueur * largeur;  
    }  
  
    private boolean isCarre() { // méthode non accessible à l'extérieur de la classe  
        //...  
    }  
    //...  
}
```

Syntaxe à point

- On peut accéder aux différents éléments accessibles d'un objet avec la syntaxe à point suivante:
nomObjet.nomElementAccessible.

Exemple: `rect1.aireRectangle()`

```
public class GestionRectangle {
    public static void main(String[] args) {
        //déclarer une variable de type rectangle
        Rectangle rect1;

        // construire un rectangle de longueur 7 et de largeur 4
        rect1 = new Rectangle(7, 4);

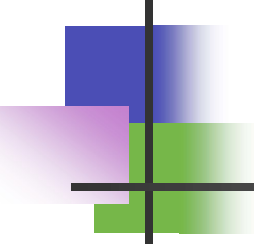
        // Afficher l'aire de rect1
        System.out.print("l'aire du rectangle 1: ");
        System.out.println(rect1.aireRectangle());

        // Afficher le périmètre de rect1
        System.out.print("le périmètre du rectangle 1: ");
        System.out.println(rect1.perimRectangle());

        // dessiner rect1 à la console avec le symbole *
        System.out.println("Représentation graphique du rectangle 1:");
        rect1.dessineRectangle('*');
    }
}
```

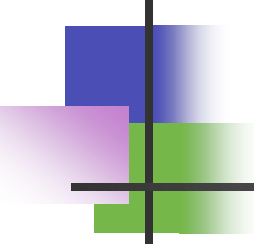
```
l'aire du rectangle 1: 28
le périmètre du rectangle 1: 22
Représentation graphique du
rectangle 1:
```

```
****
****
****
****
****
****
****
```



Principe d'encapsulation

- La programmation orientée objet repose sur trois piliers: l'encapsulation , l'héritage et le polymorphisme. L'héritage et le polymorphisme seront vus dans les prochains cours.
- **L'encapsulation est un mécanisme consistant à rassembler les données et les méthodes au sein d'une structure en cachant l'implémentation de l'objet au monde extérieur.** Ainsi, l'accès aux données ne doit se faire que par les méthodes (les services proposés). L'encapsulation permet donc de garantir l'intégrité des données contenues dans l'objet.
- Voici comment mettre en œuvre l'encapsulation :
 - **Privatiser les attributs** en utilisant le mot clé `private`. L'utilisation d'une autre visibilité doit être justifiée.
 - Si on a besoin d'accéder à un attribut à partir d'une autre classe, **créer une méthode d'accès** appelée accesseur (getter en anglais).
 - Si on a besoin de modifier un attribut à partir d'une autre classe, **créer une méthode appelée mutateur** (setter en anglais).
 - **Ne pas créer d'accesseurs/mutateurs si ce n'est pas nécessaire.**



Types de méthodes

Un objet a différents types de méthodes.

- Constructeur
 - Son rôle est d'initialiser l'objet.
- Accesseur (en anglais Getter)
 - Sous forme de `getNomDeVariable()` ou `isNomDeVariable()` pour les booléens.
 - Son rôle est d'accéder (en lecture) à un attribut de l'objet .
- Mutateur (en anglais Setter)
 - Sous forme `setNomDeVariable()`
 - Son rôle est de modifier un attribut d'un objet.
- Méthode métier (business method en anglais)
 - Effectue des calculs en fonction des données.

Accesseur (getter en anglais)

Si on a besoin d'accéder en lecture à un attribut protégé à partir d'une autre classe, on crée une méthode appelée accesseur (getter en anglais).

- Cette méthode a les caractéristiques suivantes:
 - Elle **doit être public** pour pouvoir être appelée à partir d'une autre classe.
 - Elle doit avoir comme **type de retour le type de la variable à renvoyer**.
 - Elle ne possède pas habituellement de paramètre.
 - Par convention, **son nom doit être préfixé par «get» et suivi par le nom de la variable**, comme getLargeur, getLongueur, getNote. Pour les booléens, le préfixe sera «is».

Exemple de la classe Rectangle:

```
public class Rectangle {  
    private int longueur;  
    private int largeur;  
  
    //accesseur de longueur  
    public int getLongueur() {  
        return longueur;  
    }  
  
    //accesseur de largeur  
    public int getLargeur() {  
        return largeur;  
    }  
    //...  
}
```

```
public class AutreClasse {  
    public static void main(String[] args) {  
        int lng, larg;  
        Rectangle rect = new Rectangle(5, 3);  
        // obtenir la longueur  
        lng = rect.getLongueur();  
        // obtenir la largeur  
        larg = rect.getLargeur();  
    }  
}
```

Les attributs longueur et largeur sont accessibles à partir de toutes les méthodes de la classe même s'ils sont privés (private). Par contre, ils **ne sont pas accessibles de l'extérieur de la classe**.

Mutateur (setter en anglais)

Si on a besoin de modifier un attribut protégé à partir d'une autre classe, on crée une méthode appelée mutateur (setter en anglais).

- Cette méthode a les caractéristiques suivantes:
 - Elle doit être **une méthode void** (elle ne retourne aucun résultat).
 - Elle **doit être public** pour pouvoir être appelée à partir d'une autre classe.
 - Elle doit **avoir comme paramètre la valeur à assigner à la donnée membre**. Le paramètre doit donc être du type de la donnée membre.
 - Par convention, **son nom doit être préfixé par «set» et suivi par le nom de la variable**, comme setLargeur, setLongueur, setNote.

Exemple de la classe Rectangle:

```
public class Rectangle {  
    private int longueur;  
    private int largeur;  
  
    public void setLongueur(int longu) {  
        longueur = longu;  
    }  
    public void setLargeur(int larg) {  
        largeur = larg;  
    }  
    //...  
}
```

```
public class AutreClasse {  
    public static void main(String[] args) {  
        Rectangle rect = new Rectangle(5, 3);  
        rect.setLongueur(10); //mofier la longueur de rect  
        rect.setLargeur(20); // mofier la largeur de rect  
    }  
}
```

Les attributs longueur et largeur sont accessibles à partir de toutes les méthodes de la classe même s'ils sont privés (private). Par contre, ils **ne sont pas accessibles de l'extérieur de la classe**.

Validation des données dans le mutateur

- On peut valider un attribut avant de le modifier dans un mutateur. Dans ce cas, il est intéressant de faire **appel à ce mutateur dans le constructeur pour initialiser l'attribut.**

```
public class Rectangle {  
  
    private int longueur;  
    private int largeur;  
  
    public Rectangle(int longu, int larg) {  
        setLongueur(longu); //appel du mutateur pour valider longueur  
        setLargeur(larg); //appel du mutateur pour valider largeur  
    }  
  
    public void setLongueur(int longu) {  
        if (longu > 0) { // sinon longueur = 0, valeur par défaut  
            longueur = longu;  
        }  
    }  
  
    public void setLargeur(int larg) {  
        if (larg > 0) { // sinon largeur = 0, valeur par défaut  
            largeur = larg;  
        }  
    }  
}
```

Référence this

- À l'intérieur d'une classe, le mot clé **this** fait référence à l'objet courant, c'est-à-dire l'objet qui est en train d'être manipulé.
- L'utilisation la plus courante de `this` est d'éliminer la confusion entre un attribut d'une classe et un paramètre d'une méthode ou d'un constructeur de mêmes noms.

```
public class Rectangle {  
    private int longueur;  
    private int largeur;  
  
    public void setLongueur(int lo) {  
        longueur = lo;  
    }  
  
    public void setLargeur(int largeur) {  
        this.largeur = largeur;  
    }  
    ...  
}
```

Ici, `this` est implicite (sous-entendu), il n'est pas nécessaire.
`longueur` est équivalent à `this.longueur`.

Ici, `this` est nécessaire, car l'attribut `largeur` est masqué par le paramètre `largeur` de la méthode `setLargeur()`.

- `this` peut également être utilisé dans d'autres situations qu'on verra plus loin, comme appeler le constructeur de la classe actuelle.

Référence this

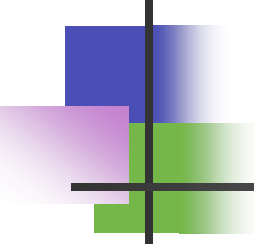
This implicite: Dans ce code, le this n'a pas besoin d'être spécifié, il est implicite. Cela permet d'avoir un code plus clair.

```
public class Rectangle {  
    private int longueur;  
    private int largeur;  
  
    public Rectangle(int pLongueur, int pLargeur) {  
        longueur = pLongueur;  
        largeur = pLargeur;  
    }  
  
    public int getLongueur() {  
        return longueur;  
    }  
  
    public void setLargeur(int pLargeur) {  
        largeur = pLargeur;  
    }  
  
    public int perimRectangle() {  
        int perim;  
        perim = (longueur + largeur) * 2;  
        return perim;  
    }  
    //...  
}
```

Ici, this est implicite (sous-entendu), il n'est pas nécessaire.

Ce code est équivalent à:

```
this.longueur = pLongueur  
this.largeur = pLargeur
```



Exercice3:

- Ajoutez le mot clé `this` aux endroits nécessaires

```
public class Rectangle {  
    private int longueur;  
    private int largeur;  
  
    public Rectangle(int largeur, int longueur) {  
        longueur = longueur;  
        largeur = largeur;  
    }  
  
    public int getLongueur() {  
        return longueur;  
    }  
  
    public void setLargeur(int largeur) {  
        largeur = largeur;  
    }  
  
    public int perimRectangle() {  
        int perim;  
        perim = (longueur + largeur) * 2;  
        return perim;  
    }  
    //...  
}
```

Exercice3: Corrigé

- Indiquez si le mot clé `this` est nécessaire devant les attributs `longueur` et `largeur` de la classe `Rectangle`.

```
public class Rectangle {
    private int longueur;
    private int largeur;

    public Rectangle(int largeur, int longueur) {
        this.longueur = longueur; // this est nécessaire
        this.largeur = largeur; // this est nécessaire
    }

    public int getLongueur() {
        return this.longueur; // this est inutile, Il est implicite
    }

    public void setLargeur(int largeur) {
        this.largeur = largeur; // this est nécessaire
    }

    public int perimRectangle() {
        int perim;
        perim = (this.longueur + this.largeur) * 2;
        //Les 2 this sont inutiles ici-haut. Ils sont implicites.
        return perim;
    }
    //...
}
```

Exercice4

1. On modifie les noms des paramètres des méthodes de la classe `Cours` comme indiqué ici-bas. Complétez les méthodes ainsi que le constructeur de cette classe. Téléchargez le nouveau code de la classe `Cours` dans Moodle.
2. Doit-on modifier la classe `GestionCours` à la suite de ces changements?
3. Une session est valide si elle est comprise entre 1 et 6. Modifiez le mutateur de session pour ajouter sa validation. Mettez à jour le constructeur.
4. Un code cours est valide s'il a 10 caractères, sinon la chaîne «INCONNU» est enregistrée dans la variable. Le code doit être converti en majuscule dans tous les cas. Modifiez le mutateur du code de cours pour ajouter sa validation.
5. Mettez à jour le constructeur.

```
public class Cours {
    private String code;
    private String titre;
    private int session;
    private int nbHeures;

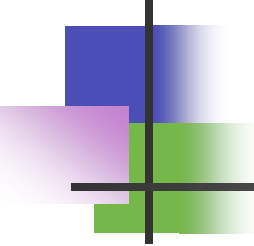
    // constructeur
    public Cours(String code, String titre, int session, int nbHeures) {
        // à compléter
    }

    // mutateurs
    public void setCode(String code) {
        // à compléter
    }
    public void setTitre(String titre) {
        // à compléter
    }
    public void setNbHeures(int nbHeures) {
        // à compléter
    }
    public void setSession(int session) {
        // à compléter
    }
}
```

```
// Accesseurs
public String getCode() {
    return code;
}
public int getSession() {
    return session;
}

//Méthodes métiers
public void afficher() {
    System.out.println("Informations du cours: " + code);
    System.out.println("*****");
    System.out.println("Titre: " + titre);
    System.out.println("Session: " + session);
    System.out.println("Nombre d'heures par semaine: " + nbHeures);
}

public int heuresSession() {
    return nbHeures * 15;
}
```

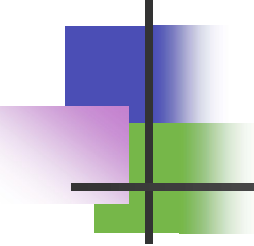


Exercice4: corrigé

Question1:

```
public class Cours {
    private String code;
    private String titre;
    private int session;
    private int nbHeures;
    // constructeur
    public Cours(String code, String titre, int session, int nbHeures) {
        this.code = code;
        this.titre = titre;
        this.session = session;
        this.nbHeures = nbHeures;
    }
    // mutateurs
    public void setCode(String code) {
        this.code = code;
    }
    public void setTitre(String titre) {
        this.titre = titre;
    }
    public void setNbHeures(int nbHeures) {
        this.nbHeures = nbHeures;
    }
    public void setSession(int session) {
        this.session = session;
    }
    // Accesseurs
    public String getCode() {
        return code;
    }

    public int getSession() {
        return session;
    }
}
```

Exercice4: corrigé

Question 2: non

Questions 3, 4 et 5

```
public Cours(String code, String titre, int session, int nbHeures) {
    setCode(code);          // setCode() valide le code avant de la modifier
    this.titre = titre;
    setSession(session);    // setSession() valide la session avant de la modifier
    this.nbHeures = nbHeures;
}

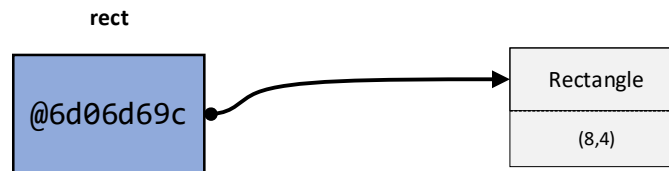
public void setSession(int session) {
    if (session <= 6 && session >= 1) {
        this.session = session;
    }
}

public void setCode(String code) {
    if (code.length() == 10) {
        this.code = code.toUpperCase();
    } else {
        this.code = "INCONNU";
    }
}
```

Afficher les informations d'un objet

- La variable `rect` ne stocke pas directement l'objet créé, mais une référence vers l'emplacement mémoire où se trouve cet objet. Par exemple:

```
Rectangle rect = new Rectangle(8, 4);  
System.out.println(rect); // affiche Rectangle@6d06d69c
```



- Si on veut afficher les informations de `rect`, on utilise les accesseurs et les autres méthodes comme suit:

```
System.out.println("Largeur:" + rect.getLargeur() + "\n" + "Longueur: " +  
rect.getLongueur() + "\n" + "périmètre:" + rect.perimRectangle());
```

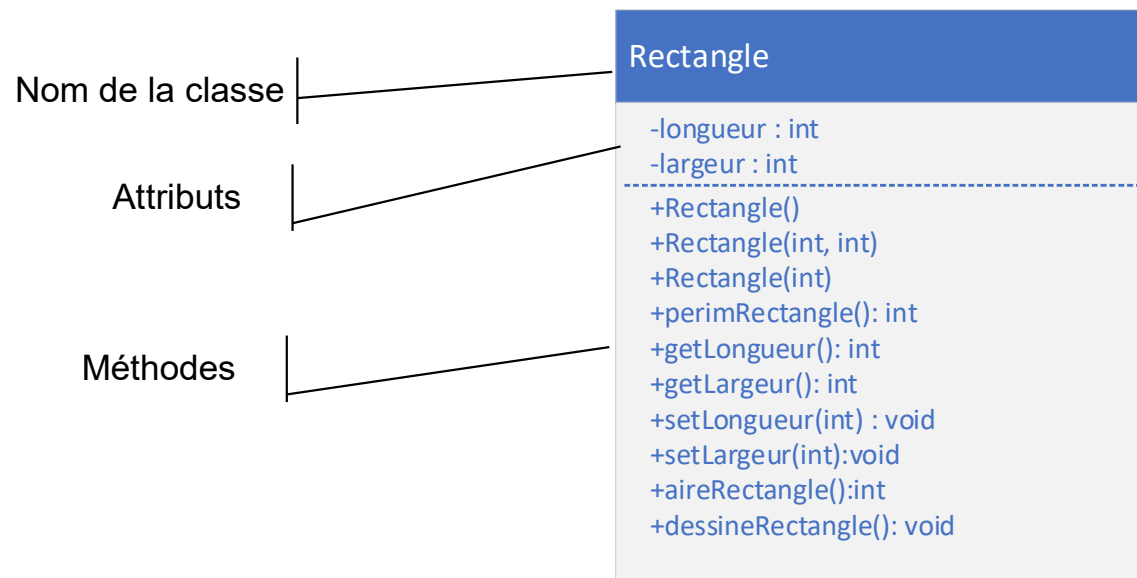
- L'affichage à la console est comme suit:

```
Largeur:4  
Longueur: 8  
périmètre:24
```

Modélisation d'une classe

Voici comment une classe peut être représentée en suivant les conventions de la modélisation objet **UML** (Unified Modeling Language).

- **On spécifie le nom de la classe, les attributs et les méthodes** dans un rectangle comme ici-bas.
- La visibilité des membres (attributs et méthodes) est représentée par les différents symboles suivants :
 - **private ou -** : le membre est accessible uniquement dans la classe en question.
 - **public ou +** : le membre est accessible dans toutes les autres classes du programme.
 - **protected ou #** : le membre est accessible uniquement dans les classes dérivées de la classe en question ainsi qu'aux classes se trouvant dans le même package. On verra la notion de classe dérivée dans les prochains cours.
 - **package ou ~ ou rien** : le membre est accessible uniquement dans les classes du même paquetage.
- Note: nous verrons plus en détail cette notion de visibilité dans les autres cours.

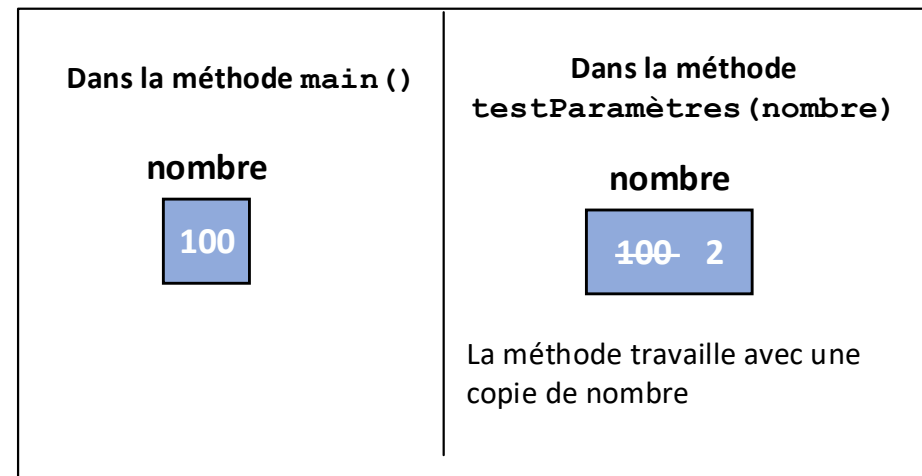


Paramètre par valeur

- Lorsqu'on passe un paramètre de type primitif (byte, short, int, long, float, double, boolean, char) à une méthode, ce paramètre est passé par valeur. C'est une copie de cette variable qui est utilisée par la méthode au moment de son appel.
- Par conséquent, **la variable initiale n'est pas accessible (donc non modifiable) par la méthode appelée.**

```
public class TestParametres {  
  
    public static void testParametres(int n) {  
        n = 2;  
        System.out.println("Dans la méthode");  
        System.out.println("nombre: " + n);  
    }  
  
    public static void main(String[] args) {  
        int nombre = 100;  
        System.out.println("Avant l'appel de la méthode");  
        System.out.println("nombre: " + nombre);  
        testParametres(nombre);  
        System.out.println("Après l'appel de la méthode");  
        System.out.println("nombre: " + nombre);  
    }  
}
```

Avant l'appel de la méthode
nombre: 100
Dans la méthode
nombre: 2
Après l'appel de la méthode
nombre: 100



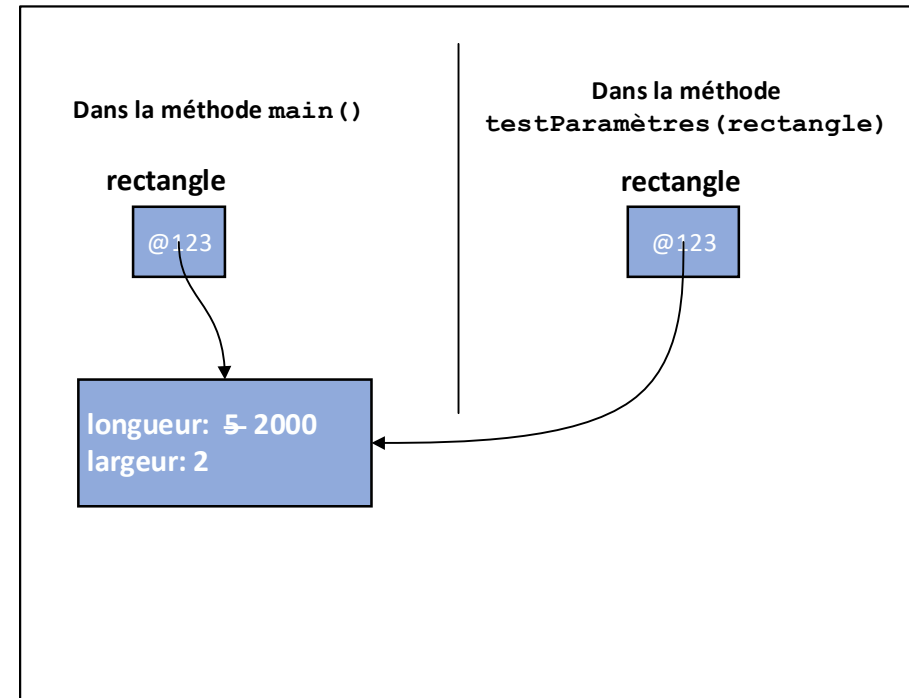
Paramètre par référence

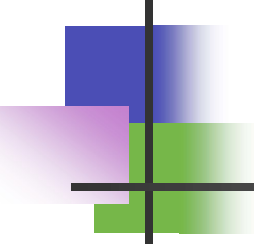
Lorsqu'on passe un paramètre de type objet à une méthode (Rectangle, tableau,...), ce paramètre est passé par référence: c'est l'adresse de l'objet placé en paramètre lors de l'appel qui est utilisée.

- Par conséquent, **l'objet initial est accessible (donc modifiable) par la méthode appelée.**

```
public class TestParametres {  
  
    public static void testParametres(Rectangle rect) {  
        rect.setLongueur(2000);  
    }  
  
    public static void main(String[] args) {  
        Rectangle rectangle = new Rectangle(5, 2);  
        System.out.println("Avant l'appel de la méthode");  
        System.out.println("longueur du rectangle: " +  
            rectangle.getLongueur());  
        testParametres(rectangle);  
        System.out.println("Après l'appel de la méthode");  
        System.out.println("longueur du rectangle: " +  
            rectangle.getLongueur());  
    }  
}
```

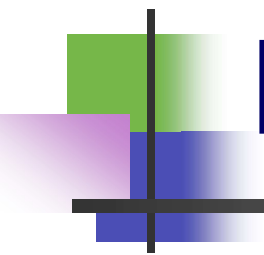
```
Avant l'appel de la méthode  
longueur du rectangle: 5  
Après l'appel de la méthode  
longueur du rectangle: 2000
```



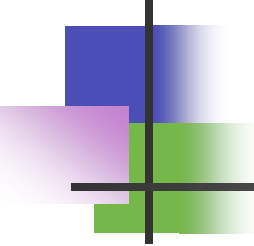


Sources

- Apprendre Java et la programmation orientée objet:
<https://www.ukonline.be/cours/java/apprendre-java>
- Tasso, A. (s.d.). Le livre de JAVA premier langage avec 109 exercices corrigés. Éditions EYROLLES.
- Les images:
<https://webcourses.ucf.edu/courses/1249560/pages/what-is-object-oriented-programming>



Les constructeurs



Constructeur: rappel

- Pour instancier une classe, c'est-à-dire créer un objet, on utilise l'opérateur `new` qui fait appel à une méthode spéciale de la classe appelée constructeur.
- **Le rôle principal d'un constructeur est d'initialiser les attributs lors de la création d'un objet.**
- **Un constructeur est une méthode spéciale qui porte le même nom que la classe** dans laquelle elle est définie, elle n'a pas de type de retour ni de mot clé `void`.
- Exemple:

```
public class Rectangle {  
    private int longueur;  
    private int largeur;  
  
    public Rectangle(int longueur, int largeur) {  
        this.longueur = longueur;  
        this.largeur = largeur;  
    }  
    //..  
}
```

Le nom du constructeur est le même que la classe (commence par une majuscule).
Pas de type retour ni `void`.

- Utilisation: si on veut construire un rectangle de longueur 7 et de largeur 4, on utilise l'instruction:
`Rectangle rect = new Rectangle (7,4);`

Plusieurs constructeurs

On peut définir **plusieurs constructeurs dans une classe**. Ces constructeurs **se différencient par le nombre de leurs paramètres ou les types de leurs paramètres**. On dit que le **constructeur est surchargé**.

- Pour déterminer quel constructeur doit être utilisé, l'interpréteur Java regarde, lors de son appel, la liste des paramètres définis dans chaque constructeur.
- Un constructeur sans paramètres est appelé **constructeur par défaut**.

```
public class Rectangle {  
    private int longueur;  
    private int largeur;  
  
    public Rectangle(int longueur, int largeur) {  
        this.longueur = longueur;  
        this.largeur = largeur;  
    }  
    //Construire un rectangle de type carré  
    // longueur = largeur  
    public Rectangle(int cote) {  
        longueur = cote;  
        largeur = cote;  
    }  
    // constructeur sans paramètres ou par défaut  
    public Rectangle() {  
        longueur = 1;  
        largeur = 1;  
    }  
    //...  
}
```

```
public class GestionRectangle {  
  
    public static void main(String[] args) {  
        //Construire un rectangle de longueur 1 et de largeur 1.  
        //Appel du troisième constructeur.  
  
        Rectangle rect1 = new Rectangle();  
  
        //Construire un rectangle de longueur 8 et de largeur 4.  
        //Appel du premier constructeur.  
  
        Rectangle rect2 = new Rectangle(8,4);  
  
        //Construire un rectangle de TYPE CARRÉ  
        //longueur = largeur = 5.  
        //Appel du deuxième constructeur.  
  
        Rectangle rect3 = new Rectangle(5);  
  
    }  
}
```

Constructeur par défaut

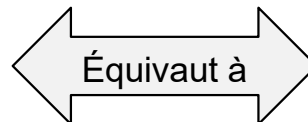
Si une classe ne contient aucun constructeur écrit par le programmeur, alors le compilateur Java lui fournit un constructeur (implicite) sans arguments qui consiste à initialiser tous les attributs avec leur valeur par défaut :

- byte, short, int, long $\leftarrow$ 0
- float, double $\leftarrow$ 0.0
- boolean $\leftarrow$ false
- char $\leftarrow$ '\u0000'
- objet quelconque (incluant String) $\leftarrow$ null

On suppose que les attributs ne sont pas initialisés avec des valeurs explicites

- C'est un constructeur par défaut, par défaut.
- **Attention! Dès que nous définissons un constructeur pour une classe, le constructeur par défaut n'existe plus.**
- Exemple: la classe Point2D n'a pas de constructeur. Java lui fournit un constructeur **implicite (par défaut)**.

```
class Point2D {  
    private int x;  
    private int y;  
  
    public String afficher() {  
        return "(" + x + "," + y + " ";  
    }  
}
```



```
class Point2D {  
    private int x;  
    private int y;  
  
    public Point2D() {  
        x = 0;  
        y = 0;  
    }  
  
    public String afficher() {  
        return "(" + x + "," + y + " ";  
    }  
}
```

- On peut alors écrire dans une méthode main d'une autre classe:

```
Point2D point = new Point2D();  
System.out.println(point.afficher()); // (0,0) s'affiche
```

Collaboration entre constructeurs (1/2)

- Un constructeur peut appeler un autre constructeur de la même classe. **this()** est utilisé pour faire appel à un autre constructeur de classe actuelle et doit être la première instruction.
- Exemple: Ces deux codes sont équivalents.

```
public class Rectangle {
    private int longueur;
    private int largeur;

    public Rectangle(int longueur, int largeur) {
        this.longueur = longueur;
        this.largeur = largeur;
    }
    //Construire un rectangle de type carré
    public Rectangle(int cote) {
        longueur = cote;
        largeur = cote;
    }
    // constructeur sans paramètres, ou par défaut
    public Rectangle() {
        longueur = 1;
        largeur = 1;
    }
    //...
}
```

Avec this()

```
public class Rectangle {
    private int longueur;
    private int largeur;

    public Rectangle(int longueur, int largeur) {
        this.longueur = longueur;
        this.largeur = largeur;
    }

    public Rectangle(int cote) {
        this(cote, cote);
        //Appel du constructeur avec 2 paramètres
    }

    public Rectangle() {
        this(1,1);
        //Appel du constructeur avec 2 paramètres
    }
    //...
}
```

Collaboration entre constructeurs (2/2)

- Supposons que l'on rajoute un attribut de type `boolean` qui vaut `true` si le rectangle est plein et `false` s'il est vide.

```
public class Rectangle {
    private int longueur;
    private int largeur;
    private boolean plein; // false par défaut

    public Rectangle(int longueur, int largeur) {
        this.longueur = longueur;
        this.largeur = largeur;
    }

    public Rectangle(int longueur, int largeur,
        boolean plein) {
        this(longueur, largeur);
        this.plein = plein;
    }

    //...
}
```

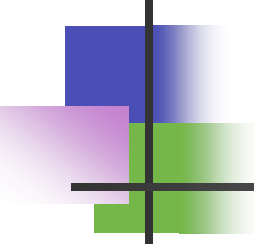
```
public class GestionRectangle {
    public static void main(String[] args) {

        /* Construire un rectangle de longueur 8 et de
        largeur 4. la variable plein prend la valeur par
        défaut : false */
        Rectangle rect0 = new Rectangle(8, 4);

        /* Construire un rectangle plein de longueur 10
        et de largeur 7 */
        Rectangle rect1 = new Rectangle(10, 7, true);

    }
}
```

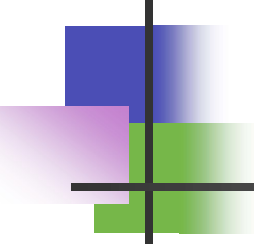
L'appel du constructeur, `this(...)`, doit être la première instruction



Exercice

Téléchargez le fichier Cours.java sur Moodle puis complétez la classe `Cours` comme demandé ici-bas.

1. Ajoutez un constructeur qui permet de construire un cours à partir du code et du titre (constructeur à 2 paramètres).
 2. Ajoutez un constructeur qui permet de construire un cours à partir du code, du titre et du nombre d'heures (constructeur à 3 paramètres). Utilisez le constructeur de la question 1 (constructeur à 2 paramètres).
 3. Modifiez le constructeur initial à 4 paramètres pour utiliser le constructeur de la question 2 (constructeur à 3 paramètres).
 4. Ajoutez un constructeur par défaut (sans paramètre) qui initialise les attributs comme suit:
 - Code «Inconnu»
 - Titre: chaîne vide
 - Nombre d'heures: 0
 - Session: -1
- Testez chaque constructeur.

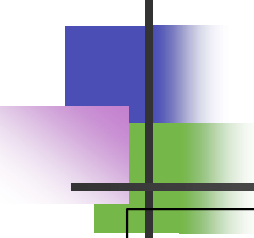


Exercice: La classe Cours

```
public class Cours {
    private String code;
    private String titre;
    private int session;
    private int nbHeures;

    // mutateurs
    public void setCode(String code) {
        if (code.length() == 10) {
            this.code = code.toUpperCase();
        } else {
            this.code = "INCONNU";
        }
    }
    public void setTitre(String titre) {
        this.titre = titre;
    }
    public void setNbHeures(int nbHeures) {
        this.nbHeures = nbHeures;
    }
    public void setSession(int session) {
        if (session <= 6 && session >= 1) {
            this.session = session;
        }
    }

    // Accesseurs
    public String getCode() {
        return code;
    }
    public int getSession() {
        return session;
    }
    public void afficher() {
        System.out.println("Informations du cours: " + code);
        System.out.println("*****");
        System.out.println("Titre: " + titre);
        System.out.println("Session: " + session);
        System.out.println("Nombre d'heures par semaine: " + nbHeures);
    }
    public int heuresSession() {
        return nbHeures * 15;
    }
}
```



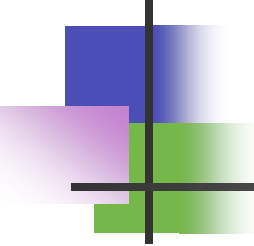
Exercice: corrigé

```
// question 1
// constructeur à 2 paramètres
public Cours(String code, String titre) {
    setCode(code);
    this.titre = titre;
    // session et nbHeures valent 0
}

// question 2
// constructeur à 3 paramètres
public Cours(String code, String titre, int nbHeures) {
    this(code, titre);
    this.nbHeures = nbHeures;
    // session vaut 0
}

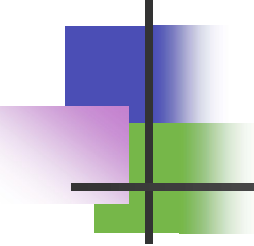
// question 3
public Cours(String code, String titre, int session, int nbHeures) {
    this(code, titre, nbHeures);
    setSession(session);
}

// question 4
public Cours() {
    this("Inconnu", "", 0, -1);
}
```



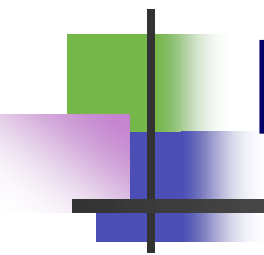
Exercice: corrigé

```
public static void main(String[] args) {  
    // 1- Test du constructeur à 2 paramètres  
    Cours c1 = new Cours("420-ZF5-M0", "Programmation structurée");  
    c1.afficher();  
  
    // 2- constructeur à 3 paramètres  
    Cours c2 = new Cours("420-ZC6-M0", "Algorithmie et programmation", 6);  
    c2.afficher();  
  
    // 3- constructeur à 4 paramètres  
    Cours c3 = new Cours("420-ZH5-M0", "Bases de données", 3, 5);  
    c3.afficher();  
  
    // 4- constructeur par défaut  
    Cours c4 = new Cours();  
    c4.afficher();  
}
```

Sources

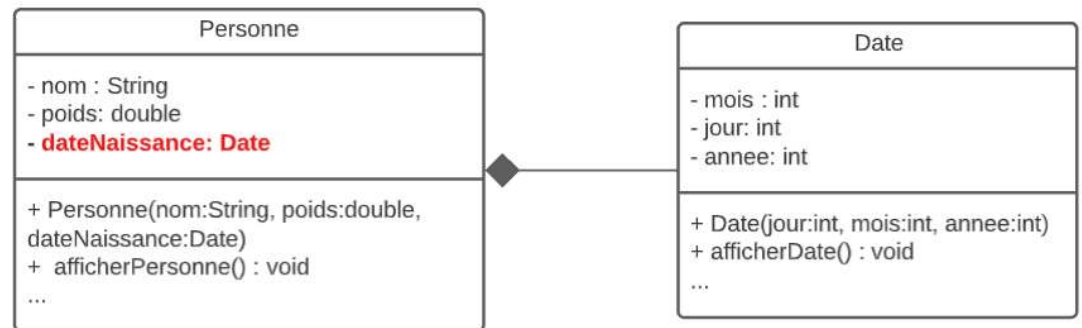
- Apprendre Java et la programmation orientée objet:
<https://www.ukonline.be/cours/java/apprendre-java>
- Tasso, A. (s.d.). Le livre de JAVA premier langage avec 109 exercices corrigés. Éditions EYROLLES.
- Les images:
<https://webcourses.ucf.edu/courses/1249560/pages/what-is-object-oriented-programming>



La relation de composition

La relation de composition

- La composition représente une forme de réutilisation de logiciel qui permet à une classe de posséder comme attributs des références à des objets d'autres classes.
- C'est une relation «a un» ou «est composé de» qui signifie qu'un objet peut contenir un ou plusieurs objets d'autres classes.
- La composition est indiquée par un lien débutant par un losange.
- Exemple: Une Personne a une Date



```
public class Personne {
    private String nom;
    private int poids;
    private Date dateNaissance;

    public Personne(String nom, int poids, Date dateNaissance) { ... }
    ...
}

public class Date {
    private int jour;
    private int mois;
    private int annee;
    public Date(int jour, int mois, int annee) {...}
    ...
}
```

An arrow points from the `dateNaissance` attribute in the **Personne** class to the **Date** class definition.

La relation de composition: création des objets

- 2 façons de créer un objet de type personne:

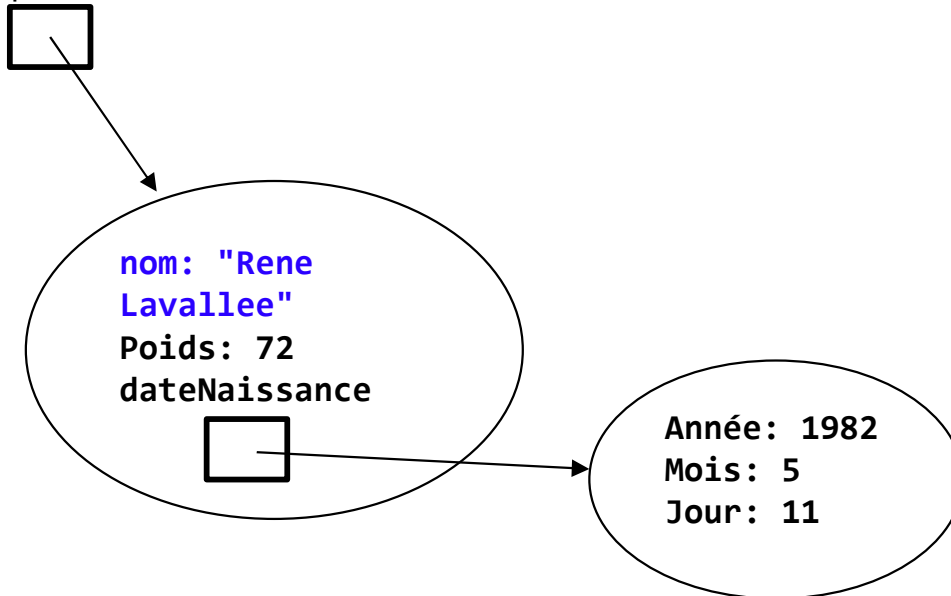
```
Personne pers1 = new Personne("Rene Lavallee" , 72, 11, 5, 1982);
```

- Ou bien

```
Date datenais = new Date(11, 5, 1982);
```

```
Personne pers1 = new Personne("Rene Lavallee" , 72, datenais);
```

pers1



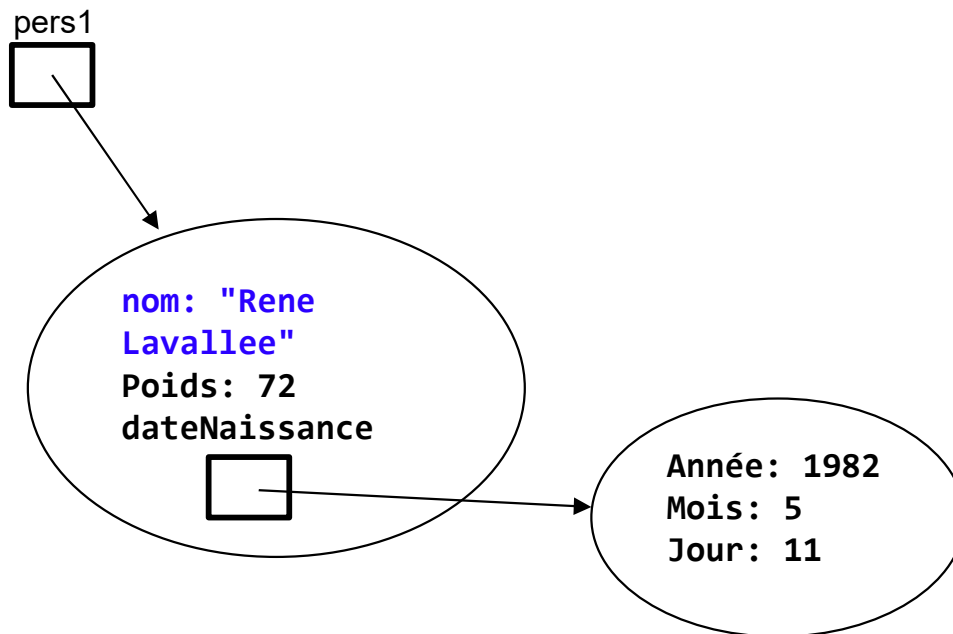
La relation de composition: accès

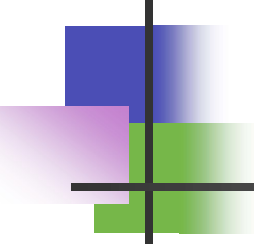
- Pour accéder au mois de naissance de pers1

```
pers1.getDateNaissance().getMois();
```

- Modifiez le mois de naissance de pers1

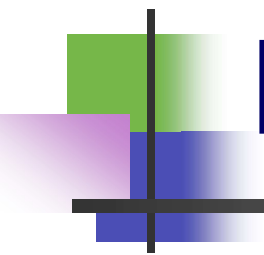
```
pers1.getDateNaissance().setMois(12);
```



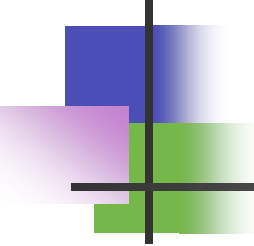


Sources

- Introduction a la programmation orientée objet:
Sotira Kyprianou



La classe Object



La classe Object

- En Java, **toutes les classes héritent de la classe Object**. La classe `Object` est donc la classe parente de toutes les classes. Les deux déclarations ci-dessous sont équivalentes.

```
public class Rectangle{  
}
```

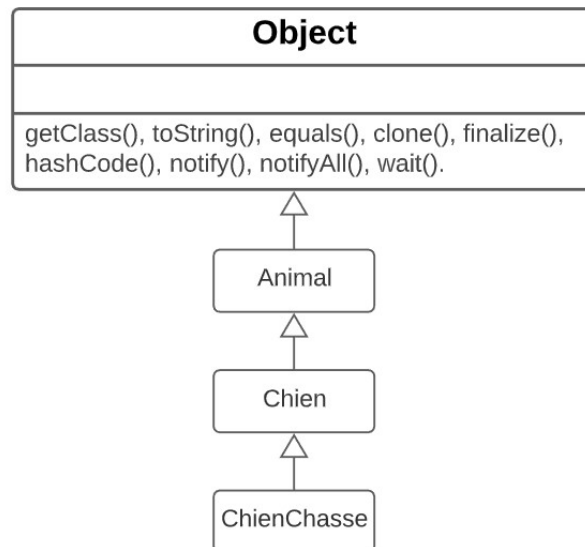


```
public class Rectangle extends Object{  
}
```

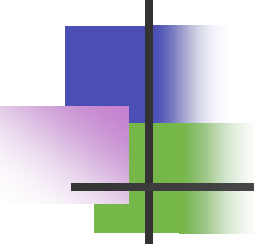
- La classe `Object` permet donc de présenter le comportement minimal de tout objet ainsi que le code par défaut pour ce comportement minimal.
- Les méthodes de la classe `Object` sont: **`getClass()`, `toString()`, `equals()`**, `clone()`, `finalize()`, `hashCode()`, `notify()`, `notifyAll()` et `wait()`.
- Dans ce cours, on s'intéresse aux méthodes `getClass()`, `toString()` et `equals()`.

La classe Object

- Toutes les méthodes contenues dans la classe `Object` sont accessibles à partir de n'importe quelle classe, car par héritage successif, toutes les classes héritent de la classe `Object`.



```
ChienChasse chien = new ChienChasse("Pitou");
chien.éléments accessibles de la classe Object
chien.getClass(), chien.toString(), chien.equals(),...
```



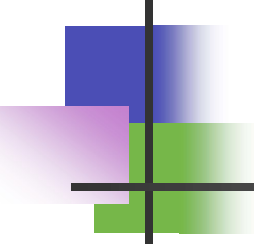
La classe Object

- On peut déclarer une variable de type `Object` et lui affecter un objet de n'importe quelle classe puisque toutes les classes héritent de la classe `Object`.

```
Object obj1, obj2;  
obj1 = new Produit( ... );  
obj2 = new chien( ... );  
obj1.éléments accessibles de Object  
obj2.éléments accessibles de Object
```

- On peut également convertir une variable de type `Object` en une variable d'une autre classe.

```
( (Produit) obj1 ).éléments accessibles de Produit et de Object  
( (Chien) obj2 ).éléments accessibles de chien et de Object
```



La méthode equals() de la classe Object

- La méthode `equals()` de la classe `Object` permet de comparer deux objets. Elle retourne `true` si les deux objets pointent vers la même zone mémoire, et `false` dans le cas contraire.
- On doit redéfinir (remplacer) cette méthode dans nos propres classes afin d'effectuer une comparaison plus appropriée.
- Pour savoir si deux objets sont égaux, on utilise la méthode `equals()` et non pas l'opérateur `==`

```
if ( obj1.equals( obj2 ) ) {  
    // obj1 est égal à obj2.  
}
```

```
if ( obj1==obj2 ) {  
    // obj1 et obj2 pointent vers la même  
    // zone mémoire.  
}
```

La méthode equals() de la classe Object

- **Exemple:** on redéfinit la méthode `equals()` dans la classe `Produit`. On suppose que deux objets de cette classe sont égaux (équivalents) s'ils ont le même numéro produit.

```
@Override
public boolean equals(Object autreObjet) {
    boolean egalite = false;
    // Si l'objet courant et l'autre objet pointent sur la même zone mémoire.
    if (this == autreObjet) {
        egalite = true;
    } else if (autreObjet != null && autreObjet instanceof Produit) {
        // Convertir le type de l'autre objet en type Produit.
        Produit autreProduit = (Produit) autreObjet;
        // L'objet courant et l'autre objet sont identiques si les numéros
        // des produits sont identiques.
        if (this.getNumero() == autreProduit.getNumero()) {
            egalite = true;
        }
    }
    return egalite;
}
```

La méthode equals() de la classe Object

- On peut alors comparer des objets de la classe Produit

```
Produit produit1 = new Produit(10, "Chaise", 132.56);  
Produit produit2 = new Produit(20, "Table", 98.50);  
Produit produit3 = new Produit(10, "Bureau", 120.00);
```

```
if (produit1.equals(produit3)) {  
    System.out.println("Le produit " + produit1 + " est identiques au produit " +  
        produit3);  
}
```

- Affichage à la console:

```
Le produit 10: Chaise: 132,56$ est identique au produit 10: Bureau: 120,00$
```

La méthode toString de la classe Object

- La méthode `toString()` définie dans la classe `Object` renvoi **le nom de la classe** suivi du **caractère @** et de **l'adresse mémoire** de cet objet en hexadécimale .
Exemple: supposons que `toString()` ne soit pas redéfinie dans la classe `Rectangle`
`Rectangle rect1 = new Rectangle(4,5);`
`System.out.println(rect); // affiche Rectangle@6d06d69c`
- Lorsqu'on définit une classe, il peut être très utile de redéfinir la méthode `toString()` afin de donner une **description satisfaisante des objets de cette classe.**
- Lorsque la méthode `System.out.print()` reçoit en paramètre un objet de n'importe quel type, celle-ci fait appel à la méthode `toString()` de l'objet pour savoir comment l'afficher. Si cette méthode n'est pas redéfinie, c'est la méthode `toString()` de la classe `Object` qui est appelée.
- Exemple de redéfinition de méthode `toString()`.

```
public class Rectangle {  
    private int longueur;  
    private int largeur;  
    ...  
    @Override  
    public String toString() {  
        return this.getClass().getName() + "[" +  
            this.getLargeur() + "," +  
            this.getLongueur() + "];"  
    }  
}
```

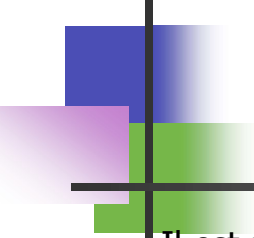
```
public static void main(String[] args) {  
    Rectangle rect = new Rectangle(45, 12);  
    System.out.println(rect);  
}  
■ Affiche à la console  
Rectangle[12,45]
```

La méthode getClass() de la classe Object

- La méthode `getClass()` retourne la classe d'un objet de type `Class`. Il existe donc une classe `Class` qui modélise les classes en Java.
- On peut connaître les caractéristiques de la classe en utilisant les méthodes de la classe `Class` :
 - `getName()` : retourne le nom de la classe incluant le package.
 - `isInterface()` : retourne `true` si la classe est une interface, `false` si non.
 - `isArray()` : retourne `true` si la classe est un tableau.
 - `getSuperClass()` : retourne les informations sur la classe mère de la classe (de type `class`).

- **Exemple :**

```
Produit produit1 = new Produit(10, "Chaise", 132.56);
Produit produit2 = new Produit(20, "Table", 98.50);
System.out.println(produit1.getClass()); // class Produit
System.out.println(produit1.getClass().getName()); // Produit
System.out.println(produit1.getClass().isInterface()); // false
System.out.println(produit1.getClass().isArray()); // false
System.out.println(produit1.getClass().getSuperclass()); // class java.lang.Object
System.out.println((produit1.getClass() == produit2.getClass())); // true
```



La méthode getClass() de la classe Object

- Il est possible d'utiliser `getClass()` pour **vérifier si la classe d'un objet est la même que celle d'un autre objet**. Si les deux classes ne sont pas dans la même arborescence d'héritage (hiérarchie), cela provoque une erreur à la compilation.

```
if (objet1.getClass()==objet2.getClass()) {  
    //traitement quand la classe de obj1 = classe de obj2  
}
```



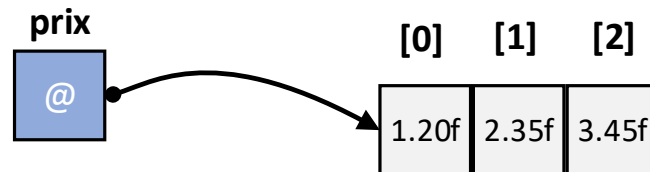

Les tableaux d'objets

Les tableaux unidimensionnels d'objets

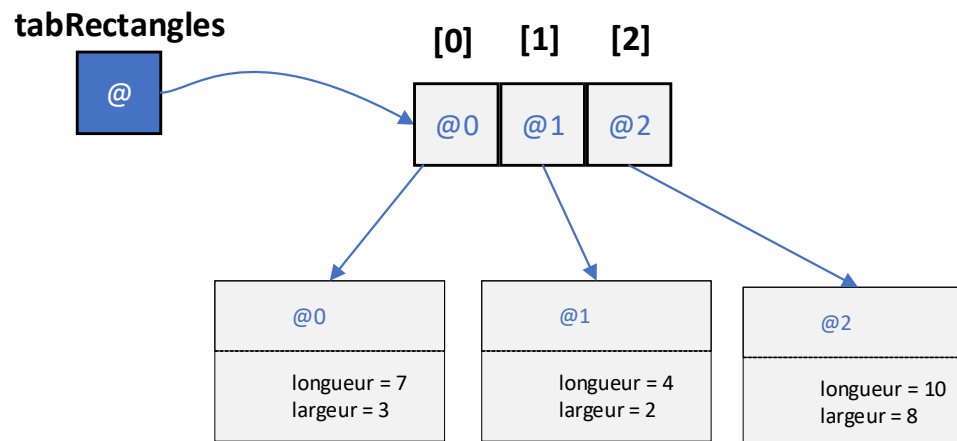
- Un tableau peut contenir n'importe quel type de données. On peut donc créer des tableaux dont le type d'éléments est une classe. Ce sont des tableaux d'objets, par exemple un tableau de type `Rectangle`, un tableau de type `Client` et un tableau de type `Produit`.
- Déclaration et initialisation d'un tableau de 5 objets de type `Rectangle`
`Rectangle[] tabRectangle1 = new Rectangle[5];`
- Initialisation d'un tableau de type `Rectangle` en donnant ses éléments lors de sa déclaration
`Rectangle[] tabRectangle2 = { new Rectangle(7, 3),
new Rectangle(4, 2), new Rectangle(10, 8) };`
- Comme pour un tableau de type simple, une fois que l'opérateur `new` est exécuté, la taille d'un tableau d'objets est fixée et on ne peut plus la modifier.

Représentation en mémoire d'un tableau unidimensionnel d'objets

- Dans un tableau de type simple, chaque case contient une valeur de ce type.
- Exemple : `float[] prix = { 1.20f, 2.35f, 3.45f };`

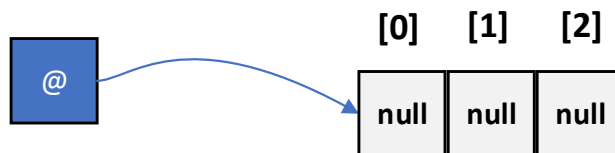


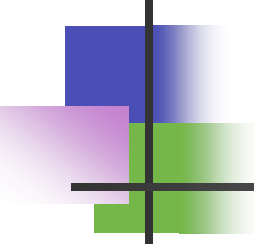
- Par contre, dans un tableau d'objets, chaque case du tableau contient la référence vers un objet.
`Rectangle[] tabRectangles = { new Rectangle(7, 3), new Rectangle(4, 2), new Rectangle(10, 8) };`



Représentation en mémoire d'un tableau unidimensionnel d'objets

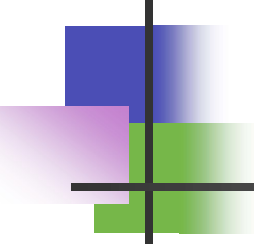
- L'opérateur `new` réserve le nombre de cases mémoires consécutives indiqué entre crochets. De plus, il initialise le tableau selon le type:
 - `byte, short, int, long` -> 0
 - `float, double` -> 0.0
 - `boolean` -> false
 - `char` -> `'\u0000'`
 - objet quelconque (incluant `String`) -> null
- Pour un tableau d'objets, les éléments sont initialisés à null.
 - Exemple:
 - `Rectangle[] tabRectangle1 = new Rectangle[3];`





Accès aux éléments d'un tableau unidimensionnel d'objets

- Pour faire appel à une méthode d'un objet se trouvant à la case d'indice `i` d'un tableau d'objets, on écrit `nomTableau[i].nomMethode()`.
- Exemple: pour avoir la largeur de l'objet qui se trouve à l'indice 2 de `tabRectangle`, on écrit:
`tabRectangle[i].getLargeur();`



Exercice1

- Complétez la méthode main de la classe Exercice1 fournie dans Moodle

```
public static void main(String[] args) {  
    // déclarer et initialiser un tableau de 3 rectangles nommé tabRectangle  
    //à compléter  
  
    // afficher la largeur du dernier rectangle  
    System.out.println("La largeur du dernier rectangle: " + "à compléter");  
  
    // modifier la longueur du premier rectangle , la nouvelle longueur est 50  
    //à compléter  
  
    // afficher le périmètre du premier rectangle  
    System.out.println("perimetre du premier rectangle: " + "à compléter");  
  
    // afficher les caractéristiques du premier rectangle  
    //à compléter  
  
    //Quel Rectangle a la plus grande superficie entre le premier et le deuxième  
    //à compléter
```

Parcourir les éléments d'un tableau unidimensionnel d'objets

- Exemple: Parcourir le tableau `tabRectangle` Pour afficher ses éléments.

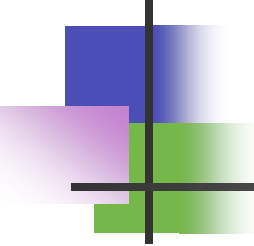
```
Rectangle[] tabRectangle = { new Rectangle(7, 3), new Rectangle(4, 2),  
                             new Rectangle(10, 8) };
```

```
for (int i = 0; i < tabRectangle.length; i++) {  
    tabRectangle[i].afficher();  
}
```

Ou

```
for (Rectangle unRectangle : tabRectangle) {  
    unRectangle.afficher();  
}
```

```
Console ×  
<terminated> test [Java Application] C:\P  
La longueur:      7  
La largeur:       3  
L'aire:           21  
Le perimetre:     20  
  
La longueur:      4  
La largeur:       2  
L'aire:           8  
Le perimetre:     12  
  
La longueur:     10  
La largeur:      8  
L'aire:          80  
Le perimetre:    36
```



Parcourir les éléments d'un tableau unidimensionnel d'objets

- Note: L'instruction d'affichage `System.out.println(tabRectangle[i])` affiche la référence de l'objet contenu dans la case `i`.

```
Rectangle[] tabRectangle = { new Rectangle(7, 3), new Rectangle(4, 2),  
                             new Rectangle(10, 8) };  
for (int i = 0; i < tabRectangle.length; i++) {  
    System.out.println(tabRectangle[i]);  
  
}
```

```
<terminated> test [Java Application] C:\Program Files\Java  
tabobjets.Rectangle@4617c264  
tabobjets.Rectangle@36baf30c  
tabobjets.Rectangle@7a81197d
```




Exercice2

Exercice2: complétez la classe Exercice2

```
public class Exercice2 {
    public static Scanner clavier = new Scanner(System.in);
    public static void main(String[] args) {
        Rectangle[] tabRectangle = new Rectangle[3];
        // Appelez initialiserTabRectangle pour initialiser (remplir) tabRectangle
        // Appelez afficherTabRectangle pour afficher les elements de tabRectangle
        // Affichez l'aire totale des rectangles: appelez aireTotale(tabRectangle)
    }
    // Initialise les elements du tableau fourni en parametre avec des données entrees au clavier.
    public static void initialiserTabRectangle(Rectangle[] tabRectangle) {
        int larg, longu;
        for (int i = 0; i < tabRectangle.length; i++) {
            System.out.print("Entrez la longueur du rectangle #" + (i+1) + ": ");
            longu = clavier.nextInt();
            System.out.print("Entrez la largeur du rectangle #" + (i+1) + ": ");
            larg = clavier.nextInt();
            //à completer
        }
    }
    //Affiche les éléments du tableau
    public static void afficherTabRectangle(Rectangle[] tabRectangle) {
        //à completer
    }
    //Retourne l'aire totale des rectangles du tableau fourni en paramètre
    public static int aireTotale(Rectangle[] tabRectangle) {
        //à completer
    }
}
```

Tableau à 2 dimensions d'objets

- Syntaxe de déclaration et d'initialisation d'un tableau à deux dimensions d'objets.
`NomType[][] nomTableau = new NomType[NB_LIGNES][NB_COLONNES]`

Exemple: un tableau de type `Rectangle` de 3 lignes et 2 colonnes.

```
Rectangle[][] tab2DRectangle1 = new Rectangle[3][2];
```

- Initialisation d'un tableau 2D de type `Rectangle` en donnant ses éléments lors de la déclaration.

```
Rectangle[][] tab2DRectangle2 = {  
    { new Rectangle(8, 2), new Rectangle(7, 5) },  
    { new Rectangle(10, 3), new Rectangle(3, 1) },  
    { new Rectangle(5, 3), new Rectangle(5, 2) } };
```

Parcourir un tableau à 2 dimensions d'objets

- Pour parcourir un tableau à deux dimensions, on utilise 2 boucles imbriquées.

```
NomType[][] nomTableau = new NomType[NB_LIGNES][NB_COLONNES];
//parcourir les lignes
for (int ligne = 0; ligne < nomTableau.length; ligne++) {
    //parcourir les colonnes
    for (int colonne = 0; colonne < nomTableau[ligne].length; colonne++) {
        tabRectangle[ligne][colonne].afficher();
    }
}
```

- Note: généralement, on utilise le nom de la variable `i` pour une `ligne` et le nom `j` pour une `colonne`.

Parcourir un tableau à 2 dimensions d'objets

```
public class Tableau2DRectangle {
    public static void main(String[] args) {
        Rectangle[][] tab2DRectangle1 = new Rectangle[3][2];
        Rectangle[][] tab2DRectangle2 = {
            { new Rectangle(8, 2), new Rectangle(7, 5) },
            { new Rectangle(10, 3), new Rectangle(3, 1) },
            { new Rectangle(5, 3), new Rectangle(5, 2) }
        };
        affichertabRectangle(tab2DRectangle2);
    }

    public static void affichertabRectangle(Rectangle[][] tabRectangle) {
        for (int i = 0; i < tabRectangle.length; i++) {
            for (int j = 0; j < tabRectangle[i].length; j++) {
                tabRectangle[i][j].afficher();
            }
        }
    }
}
```



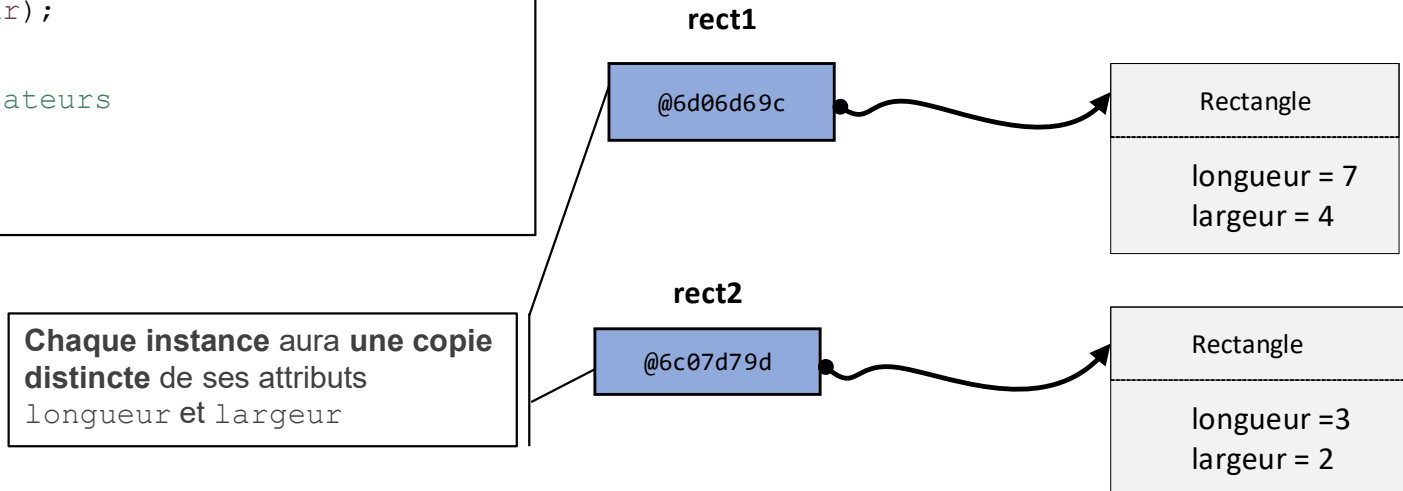
Les attributs et méthodes statiques

Rappel: attribut ou champ d'instance

- On a vu dans le cours précédent que chaque nouvel objet initialisé aura une copie distincte de ses attributs (ou variables d'instance).
- Exemple: Si on crée 2 objets de la classe `Rectangle` `rect1` et `rect2` comme ici-bas, alors `rect1` aura comme longueur 7 et comme largeur 4, alors que `rect2` aura comme longueur 3 et comme largeur 2.

```
public class Rectangle {  
    // Attributs(ou variables d'instances)  
    private int longueur;  
    private int largeur;  
  
    public Rectangle(int longueur, int largeur) {  
        setLongueur(longueur);  
        setLargeur(largeur);  
    }  
  
    // Accesseurs et mutateurs  
}
```

```
public static void main(String[] args) {  
    Rectangle rect1, rect2;  
    rect1 = new Rectangle(7, 4);  
    rect2 = new Rectangle(3, 2);  
}
```



Attribut statique (ou variable de classe)

Si on veut partager un attribut entre toutes les instances d'une classe, on doit le définir comme statique à l'aide du mot clé `static`. **Une copie unique de cet attribut sera alors créée et partagée entre toutes les instances de cette classe.**

- Exemple: Supposons que nous voulions créer une variable qui va compter le nombre d'objets de type Rectangle qui ont été initialisés. Cette variable devra être partagée entre toutes les instances de Rectangle, permettant ainsi à chacune d'y accéder et de l'incrémenter lors de son initialisation.

```
public class Rectangle {
    // Attribut statique ou variable de classe
    public static int compteurRectangle = 0;

    // Attributs (ou champs d'instance)
    private int longueur;
    private int largeur;

    // Constructeur
    public Rectangle(int longueur, int largeur) {
        setLongueur(longueur);
        setLargeur(largeur);
        compteurRectangle++; //une instance de plus
    }

    // Accesseurs et mutateurs
    ...
}
```

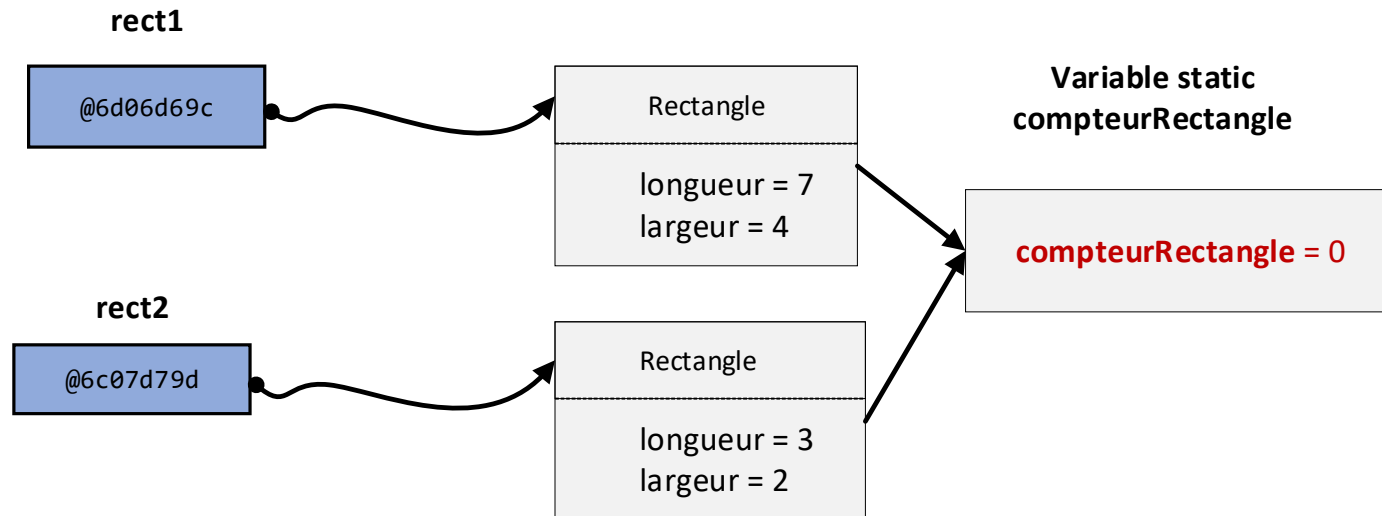
```
public static void main(String[] args) {
    Rectangle rect1, rect2;
    rect1 = new Rectangle(7, 4);
    //incrémente compteurRectangle
    rect2 = new Rectangle(3, 2);
    //incrémente compteurRectangle
    System.out.println(
        Rectangle.compteurRectangle);
    //affiche 2
}
```

- En dehors de la classe, il faut préciser le nom de la classe suivi de la variable statique.
- Dans la classe, on référence directement la variable statique.
- `compteurRectangle` est incrémentée à chaque instantiation de la classe Rectangle.

Attribut statique (ou variable de classe)

Il y aura un attribut `longueur` et un attribut `largeur` pour chaque instance (objet) de la classe `Rectangle`, mais il n'y a qu'un seul champ statique `compteurRectangle`.

- L'attribut statique `compteurRectangle` appartient à la classe `Rectangle` et non pas aux objets (instances) de la classe `Rectangle`. **Il existe un seul exemplaire** de cet attribut statique que toutes les instances de la classe `Rectangle` se partagent. Cet attribut aura la même valeur pour tous les objets `Rectangle`.
- La création de cette variable statique permet d'économiser l'espace mémoire. En effet, on a une copie unique pour toutes les instances de `Rectangle`.





Accès à un attribut statique

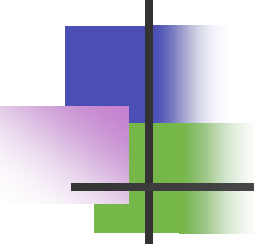
- Les attributs statiques sont accessibles sans initialisation d'objet. **On y accède avec le nom de la classe suivi du point suivi du nom de l'attribut**, s'il n'est pas protégé.

Rectangle.compteurRectangle;

- Même si aucun objet n'est encore instancié, le champ statique est présent et peut être utilisé. **Un champ statique est disponible aussitôt que la classe est chargée en mémoire.**
- On peut accéder à un attribut statique en utilisant une référence d'objet, mais il n'est pas conseillé. En effet, il devient difficile dans ce cas de déterminer s'il s'agit d'une variable d'instance ou de classe.
- Exemple:

rect1.compteurRectangle = 45;

- Attention! Bien que `compteurRectangle` soit modifié par l'instance `rect1`, **cette valeur est modifiée pour tous les objets de la classe puisqu'elle est unique.**



Accès à un attribut statique

- Dans le cas où l'attribut statique est privé, on lui ajoute un accesseur comme pour les attributs d'instance.
- Exemple:

```
public class Rectangle {  
    // Attribut statique ou variable de classe  
    private static int compteurRectangle = 0;  
  
    // Attributs ou variables d'instance de la classe Rectangle  
    private int longueur;  
    private int largeur;  
  
    // Constructeur  
    public Rectangle(int longueur, int largeur) {  
        setLongueur(longueur);  
        setLargeur(largeur);  
        compteurRectangle++; // une instance de plus  
    }  
  
    // Accesseur de l'attribut statique compteurRectangle  
    public static int getCompteurRectangle() {  
        return compteurRectangle;  
    }  
}
```

```
public static void main(String[] args) {  
    Rectangle rect1, rect2;  
    rect1 = new Rectangle(7, 4);  
    rect2 = new Rectangle(3, 2);  
    System.out.print("Nombre d'occurrences: "  
+ Rectangle.getCompteurRectangle());  
}
```

Méthode statique

Une méthode statique, également appelée méthode de classe, est une méthode qui a un rôle indépendant d'une quelconque instance. **Elle appartient à la classe et non à une instance quelconque de la classe.**

- On utilise une méthode statique dans les cas suivants:
 - La méthode n'a **pas besoin d'accéder à l'état d'un objet** (attributs d'instance).
 - la méthode **n'accède qu'à des variables statiques** de la classe.
- Nous n'avons pas besoin de créer un objet pour appeler une méthode statique. On invoque une méthode statique à l'aide du nom de la classe suivi de l'opérateur « . » suivi du nom de la méthode.
- Exemple:

```
public class Rectangle {  
    // Attribut statique ou variable de classe  
    private static int compteurRectangle = 0;  
  
    // Attributs d'instance  
    private int longueur;  
    private int largeur;  
  
    public Rectangle(int longueur, int largeur) {  
        setLongueur(longueur);  
        setLargeur(largeur);  
        compteurRectangle++; // une instance de plus  
    }  
  
    // méthode statique  
    public static int getCompteurRectangle() {  
        return compteurRectangle;  
    }  
}
```

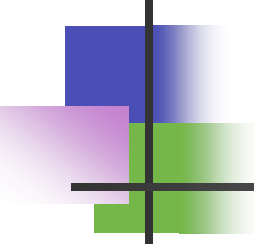
```
public static void main(String[] args) {  
    Rectangle rect1, rect2;  
    rect1 = new Rectangle(7, 4);  
    rect2 = new Rectangle(3, 2);  
    System.out.print("Nombre d'occurrences: "  
+ Rectangle.getCompteurRectangle());  
}
```

Méthode statique

- **Une méthode statique ne peut accéder aux attributs et méthodes qui ne sont pas statiques.**
Puisqu'une méthode statique n'appartient à aucune instance de la classe, elle ne peut invoquer ni un attribut d'instance ni une méthode d'instance.
- **Une méthode statique ne possède aucune référence `this`**, puisqu'elle n'est rattachée à aucun objet de la classe. D'ailleurs, elle existe avant même l'instanciation du tout premier objet de la classe.
- **Un constructeur n'est jamais statique**

```
12 public static Rectangle(int longueur, int largeur) {  
13     setLongueur(longueur);  
14     setLargeur(largeur);  
15     compteurRectangle++;  
16 }  
17  
18 public static int testMethodeStatic() {  
19     longueur = 3;  
20     this.compteurRectangle = 10;  
21 }
```

- Le message d'erreur de la ligne 12
«Illegal modifier for the constructor in type Rectangle; only public, protected & private are permitted»
- Le message d'erreur de la ligne 19
«Cannot make a static reference to the non-static field longueur»
- Le message d'erreur de la ligne 20
«Cannot use this in a static context»



Méthode statique

- **Les méthodes statiques sont souvent utilisées dans des classes utilitaires** ou auxiliaires afin de pouvoir les utiliser sans créer d'objet.
Exemple: les méthodes `Math.sqrt()` et `Math.pow()` sont des méthodes statiques.
- La méthode `main()` est la méthode principale ou le point d'entrée d'un programme. Elle est invoquée automatiquement lors de l'exécution du programme.
- La méthode `main()` est une méthode statique. Elle n'a besoin d'aucun objet pour être appelée. Lorsqu'un programme démarre, il n'existe aucun objet dont le programme a besoin.

Une constante statique

- C'est une constante commune à tous les objets de la classe.
- C'est un attribut déclaré avec le mot `static` et `final` (il ne peut être initialisé qu'une seule fois).
- Exemple: `TYPE_FORME = "Rectangle"` est une constante commune à tous les objets `Rectangle`.

`public static final String TYPE_FORME = "Rectangle";`

```
public class Rectangle {
    // Constante publique et statique (de classe)
    public static final String TYPE_FORME = "Rectangle";

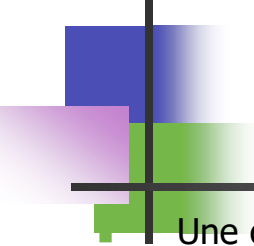
    // Variable statique ou de classe
    private static int compteurRectangle = 0;

    // Attributs de classe
    private int longueur;
    private int largeur;

    // Constructeur
    public Rectangle(int longueur, int largeur) {
        setLongueur(longueur);
        setLargeur(largeur);
        compteurRectangle++; // une instance de plus
    }

    public void afficher() {
        System.out.println(TYPE_FORME + " de Longueur "+
            longueur + " et de largeur "+ largeur);
    }
}
```

```
public static void main(String[] args) {
    Rectangle rect1;
    System.out.println(
        Rectangle.TYPE_FORME );
    // affiche Rectangle
    rect1 = new Rectangle(7, 4);
    rect1.afficher();
    // affiche: Rectangle de longueur 7
    // et de largeur 3
    ...
}
```

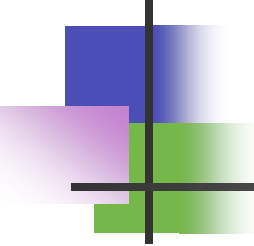


Classe utilitaire

Une classe utilitaire regroupe des fonctionnalités utiles au programme en cours de développement. Elle possède les caractéristiques suivantes:

- Elle ne contient aucun attribut, ce qui implique l'absence de getters et de setters.
- Elle se compose exclusivement de constantes et de méthodes statiques, permettant ainsi leur invocation depuis une autre classe sans nécessiter d'instancier un objet.
- Elle ne contient généralement pas de constructeur public. Cependant, la classe peut avoir un constructeur privé pour empêcher explicitement toute instanciation depuis l'extérieur.

```
public class NomDeLaClasseUtilitaire {  
    // Constantes, le cas échéant  
    public static final int UNE_CONSTANTE = 42;  
  
    // Constructeur privé pour empêcher l'instanciation  
    private NomDeLaClasseUtilitaire() {  
        // Constructeur privé pour empêcher l'instanciation  
    }  
  
    // Méthode utilitaire 1  
    public static TypeRetour methodeUtilitaire1(TypeParametre1 parametre1, TypeParametre2  
parametre2, ...) {...}  
  
    // Méthode utilitaire 2  
    public static TypeRetour methodeUtilitaire2(TypeParametre1 parametre1, TypeParametre2  
parametre2, ...) {...}  
  
    // Autres méthodes utiles  
}
```



La classe utilitaire Math

- La classe `java.lang.Math` est une classe utilitaire en Java qui fournit des méthodes statiques pour effectuer des opérations mathématiques.
- Elle inclut des constantes et des méthodes telles que :
 - `public static final double PI`
 - `public static double sqrt(double a){ ... }`
 - `public static double pow(double a, double b){...}`
- La classe `Math` a un constructeur qui n'est pas visible (privé)
- <https://docs.oracle.com/javase/8/docs/api/java/lang/Math.html>

- Exemples d'utilisation de la classe `Math`:

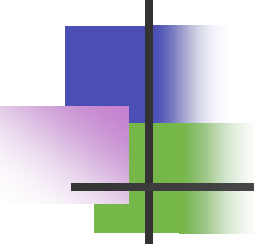
```
public double perimetre() {  
    return Math.PI * rayon * 2;  
}
```

```
boolean estSurCercle(double x, double y) {  
    return Math.pow((x - h), 2) + Math.pow((y - k), 2) == Math.pow(rayon, 2);  
}
```



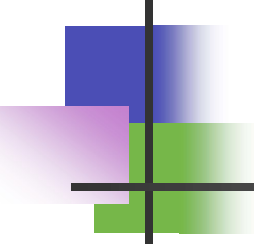

Résumé

- Tout ce qui est déclaré **static**, appartient à la classe et non à une instance particulière de cette classe.
- Il est possible d'avoir des attributs statiques et des méthodes statiques.



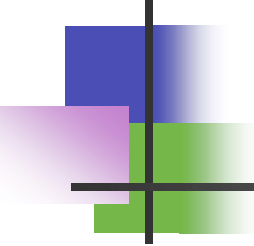
Exercice

- Créez une classe utilitaire nommée `UtilPuissance`
- Ajoutez un constructeur privé par défaut qui ne fait rien
- Ajoutez une méthode publique statique `double calculerCarre(double nombre)` qui calcule et retourne le carré d'un nombre de type `double` reçu en paramètre.
- Ajoutez une méthode publique statique `long calculerCarre(int nombre)` qui calcule et retourne le carré d'un nombre de type `int` reçu en paramètre.
- Ajoutez une méthode publique statique `double calculerCube(double nombre)` qui calcule et retourne le cube d'un nombre de type `double` reçu en paramètre.
- Créez une classe nommée `MainUtilPuissance` comportant une méthode `main()` pour tester les 3 méthodes.
 - Calculez le carré de 78,5
 - Calculez le carré de 200
 - Calculez le cube de 15



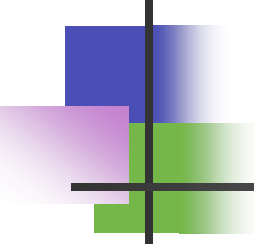
Corrigé de l'exercice (1/2)

```
public class UtilPuissance {  
    // Constructeur privé pour empêcher l'instanciation  
    private UtilPuissance() {  
    }  
  
    // Méthode utilitaire 1 - Carré d'un double  
    public static double calculerCarre(double nombre) {  
        return nombre * nombre;  
    }  
  
    // Méthode surchargée 1 - Carré d'un int  
    public static long calculerCarre(int nombre) {  
        return nombre * nombre;  
    }  
  
    // Méthode utilitaire 2 - Cube d'un double  
    public static double calculerCube(double nombre) {  
        // return nombre * nombre * nombre;  
        return Math.pow(nombre, 3);  
    }  
}
```



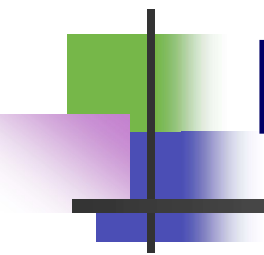
Corrigé de l'exercice (2/2)

```
public class MainUtilPuissance {  
  
    public static void main(String[] args) {  
        System.out.println("le carre de 78.5 est " + UtilPuissance.calculerCarre(78.5));  
        System.out.println("le carre de 200 est " + UtilPuissance.calculerCarre(200));  
        System.out.println("le cube de 15 est " + UtilPuissance.calculerCube(15));  
    }  
}
```



Sources

- Kypriano, S. Le modificateur static
- Apprendre Java et la programmation orientée objet:
<https://www.ukonline.be/cours/java/apprendre-java>
- Tasso, A. (s.d.). Le livre de JAVA premier langage avec 109 exercices corrigés. Éditions EYROLLES.



Les interfaces



Les interfaces

- Une interface permet de créer un sous-type sans créer de classe de base. **Une interface n'est pas une classe.**
- Une interface est un peu comme une classe abstraite, mais en version minimaliste: elle peut contenir des constantes et des méthodes abstraites, mais ne peut pas avoir de méthodes d'instance concrètes, de constructeurs ou d'attributs.
- Une interface permet donc de définir un ensemble de méthodes à implémenter. Elle établit ainsi un contrat que certaines classes peuvent s'engager à respecter.
- La syntaxe de déclaration d'une interface est:

```
visibilité interface NomInterface {  
    // définition des constantes publiques  
    // Il ne peut pas y avoir de champs d'instance.  
    // Il ne peut pas y avoir de constructeurs.  
    // définition des méthodes abstraites  
}
```

- Exemple:

```
public interface Amical {  
    public final int NB_CALIN_MIN = 3;  
    public abstract void faireCalin();  
    public abstract int nombreCalin();  
}
```

Les interfaces

- Une interface contient :
 - Éventuellement des constantes publiques et statiques.
 - Les mots-clés `public`, `static` et `final` sont **implicites et facultatifs**, ils peuvent être omis puisque les attributs d'instance ne sont pas autorisés.
 - Éventuellement des méthodes publiques et abstraites.
 - Les mots-clés `public` et `abstract` sont donc également **implicites et facultatifs**.
 - Depuis Java 8, des méthodes par défaut (`public default`), des méthodes statiques (`public static`) et depuis Java 9, des méthodes privées (`private`) et des méthodes privées statiques (`private static`). Dans ce cours, nous traiterons des interfaces qui contiennent des constantes et des méthodes abstraites.
 - Et rien d'autre

```
interface Amical {  
    int NB_CALIN_MIN = 3;  
    void faireCalin();  
    int nombreCalins();  
}
```



```
public interface Amical {  
    public static final int NB_CALIN_MIN = 3;  
    public abstract void faireCalin();  
    public abstract int nombreCalin();  
}
```


Implémentation d'une interface

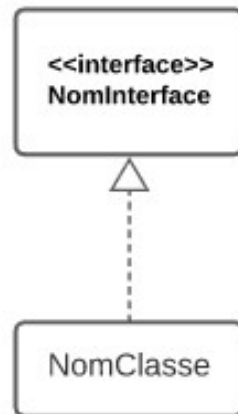
- On dit **qu'une classe implémente une interface**, si elle **défini les méthodes de l'interface**.
- En java, on déclare qu'une classe implémente une interface avec le mot clé `implements`.
- **Si la classe n'implémente pas toutes les méthodes abstraites de l'interface, elle devient alors une classe abstraite** (`abstract`).
- Une classe qui implémente une interface peut utiliser ses constantes directement sans avoir à spécifier le nom de l'interface.

- Syntaxe

```
public class NomClasse implements NomInterface {  
    // On peut utiliser une constante de l'interface NomInterface  
    // directement sans avoir à spécifier le nom de l'interface.  
  
    // Implémentation de toutes les méthodes abstraites  
    // de l'interface NomInterface.  
  
    @Override  
    public typeRetour nomMethode( typePar1 nomPar1, typePar2 nomPar2, ... ) {  
        . . .  
    }  
}
```

Représentation graphique d'une interface

- Une flèche pointillée avec un bout vide pointe de la classe vers l'interface.



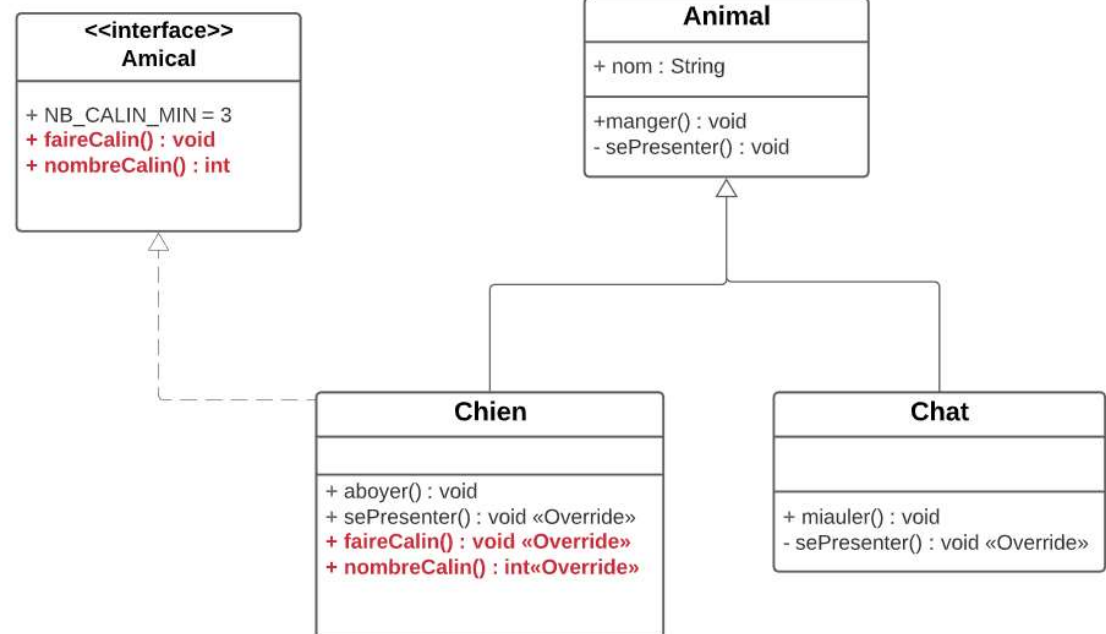
Implémentation d'une interface

Une classe peut hériter d'une autre classe et implémenter une interface. Dans ce cas, on doit **respecter l'ordre des déclarations**: l'expression d'héritage (`extends`) doit être avant l'expression d'implémentation (`implements`), sinon le code ne compilera pas.

- Exemple: La classe `Chien` hérite de la classe `Animal` et implémente l'interface `Amical`.
- La classe `Chien` doit implémenter toutes les méthodes de l'interface `Animal`. Sinon, la classe `Chien` deviendra abstraite.

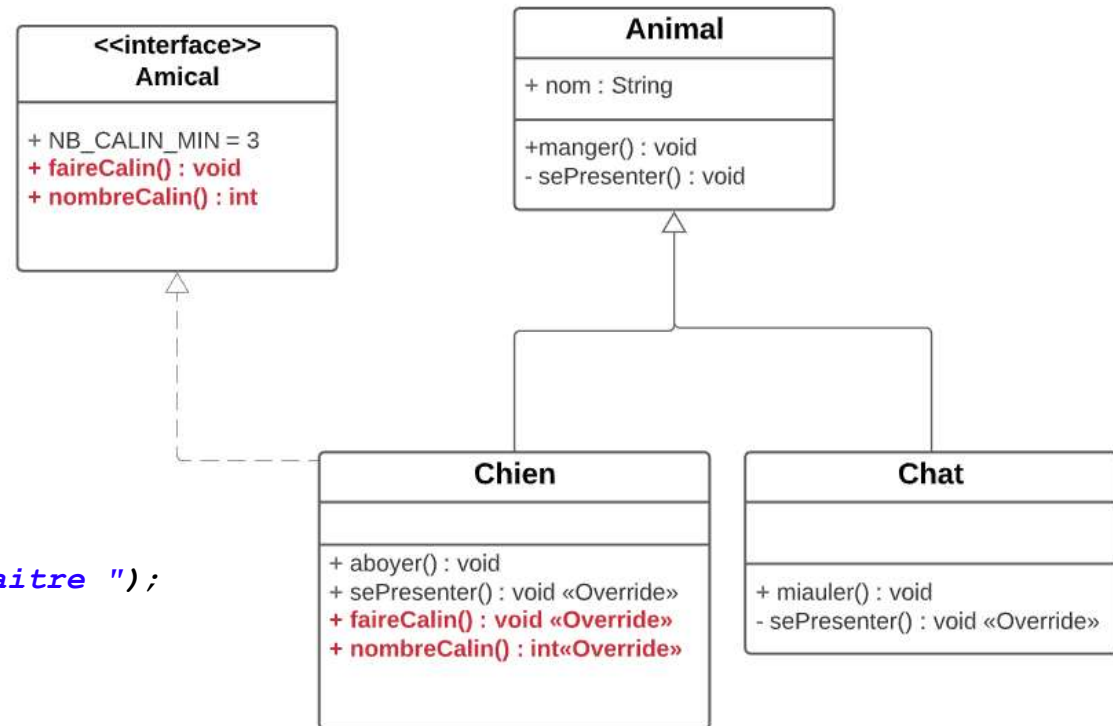
```
public class Chien extends Animal implements Amical {  
    //contenu de la classe Chien
```

```
    ...  
    // Méthode de l'interface Amical  
    @Override  
    public void faireCalin() {  
        ...  
    }  
    @Override  
    public int nombreCalin() {  
        ...  
    }  
    ...  
    // Méthode de la super-classe Animal  
    @Override  
    public void sePresenter() {  
        ...  
    }  
}
```



Implémentation d'une interface

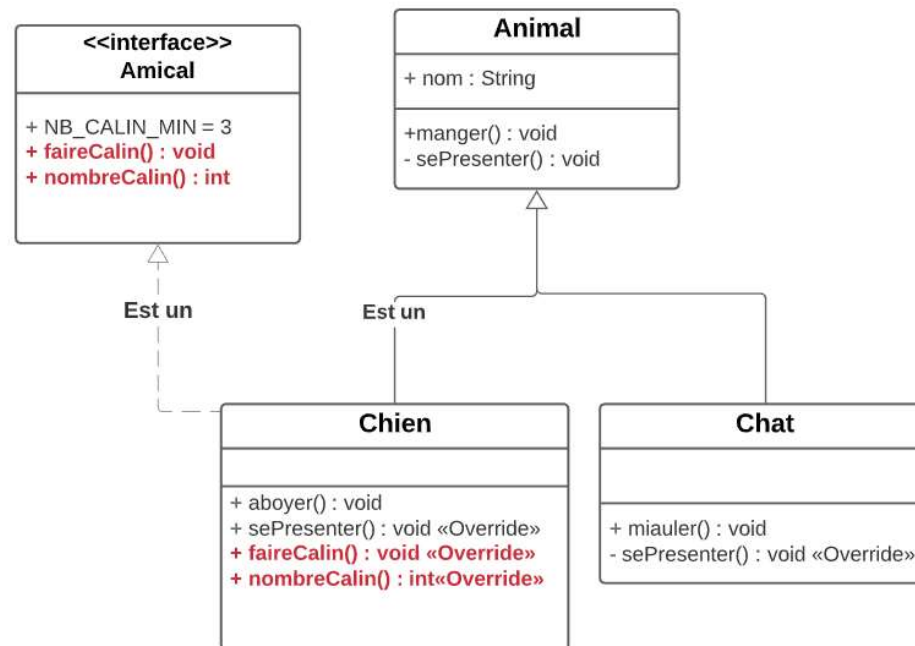
```
public class Chien extends Animal implements Amical {  
    public Chien(String nom) {  
        super(nom);  
    }  
    public void aboyer() {  
        System.out.println("Wof");  
    }  
    @Override  
    // Méthode de la superclasse Animal  
    public void sePresenter() {  
        System.out.println("Je suis le chien "  
            + getNom());  
    }  
    //Méthode de l'interface Amical  
    @Override  
    public void faireCalin() {  
        System.out.println("Je me colle à mon maitre ");  
    }  
    //Méthode de l'interface Amical  
    @Override  
    public int nombreCalin() {  
        return NB_CALIN_MIN * 2;  
        // Accès à la constante de l'interface directement  
    }  
}
```

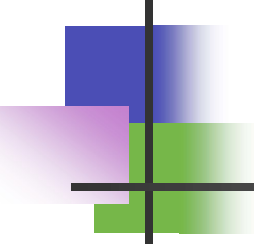


Implémentation d'une interface

Une interface permet d'attribuer un type supplémentaire à une classe .

- C'est la solution de Java **pour imposer un contenu commun à des classes en dehors d'une relation d'héritage**.
- **Java ne permet pas l'héritage multiple**, l'utilisation des interfaces permet de pallier cette carence, dans la mesure où une classe peut implémenter plusieurs interfaces.
- **Exemple:** Un objet `Chien` est de type `Animal`, il est également de type `Amical`.





Une classe peut implémenter plusieurs interfaces

- Une classe peut implémenter plusieurs interfaces. Dans ce cas, elle doit implémenter toutes les méthodes abstraites de toutes les interfaces sinon, elle devient une classe abstraite.

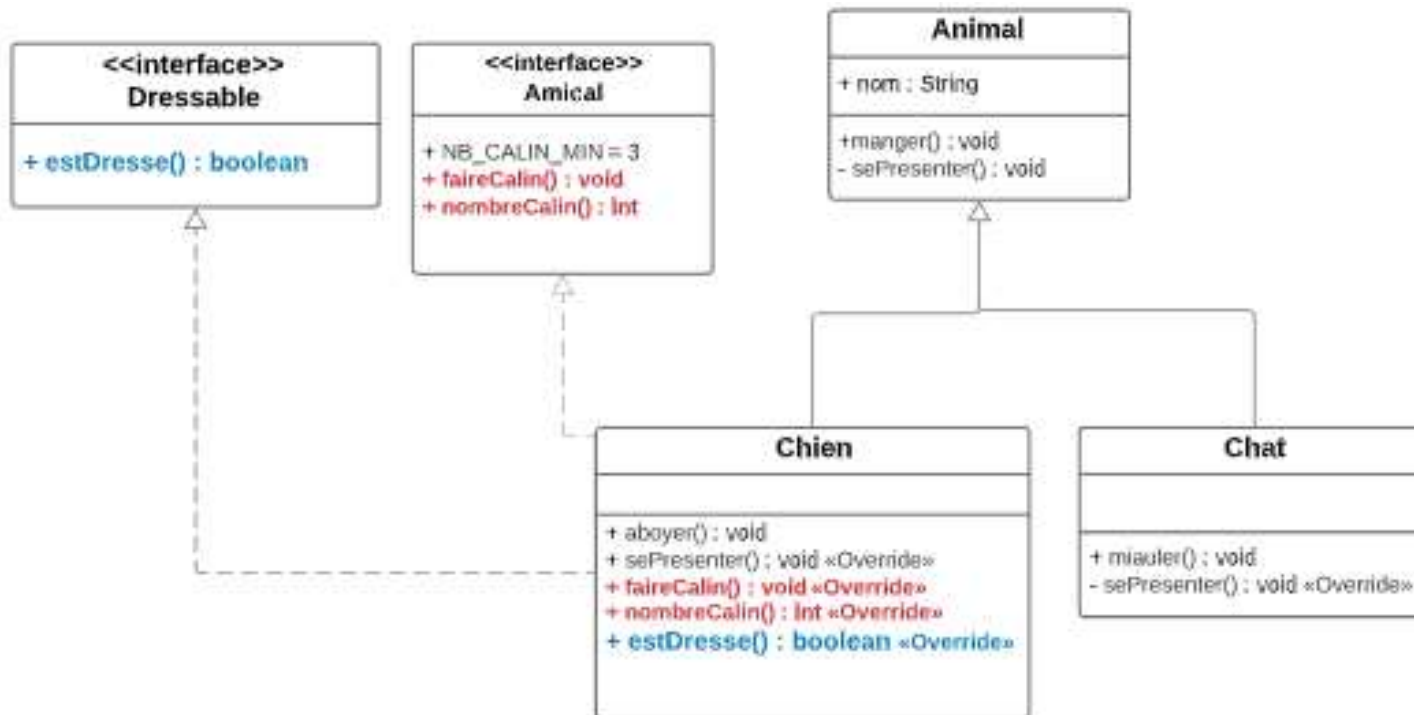
```
public class NomClasse implements NomInterface1, NomInterface2, ... {  
    // Implémentation de toutes les méthodes abstraites  
    // des interfaces NomInterface1, NomInterface2, ...  
  
    @Override  
    public typeRetour nomMethode( typePar1 nomPar1, typePar2 nomPar2, ... ) {  
        . . .  
    }  
}
```

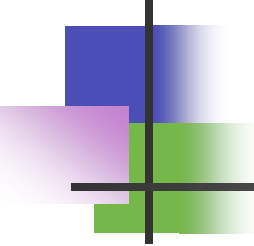
- Une classe qui n'implémente pas toutes les méthodes des interfaces devient abstraite

```
public abstract class NomClasse implements NomInterface1, NomInterface2, ... {  
    // Les méthodes abstraites des interfaces  
    // NomInterface1, NomInterface2, ...  
    // ne sont pas toutes implémentées.  
    . . .  
}
```

Une classe peut implémenter plusieurs interfaces

- Exemple:





Héritage entre les interfaces

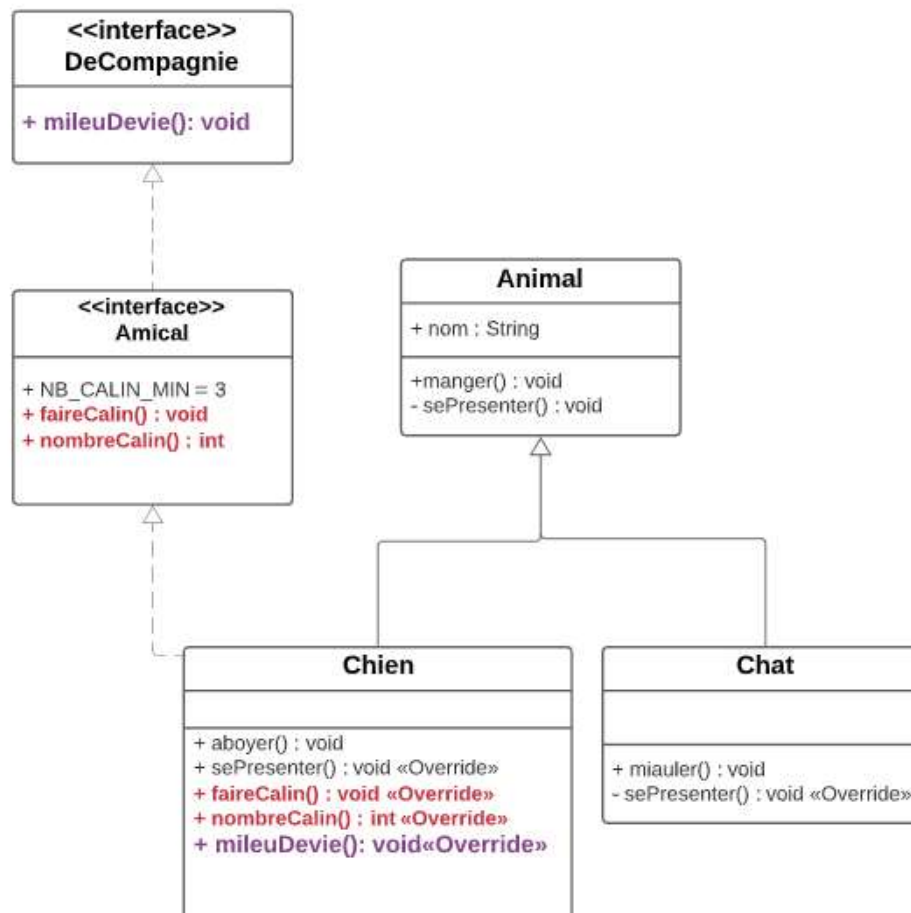
- Une **interface peut hériter d'une autre interface**.
- On a vu, dans les précédents cours, qu'une classe ne peut pas hériter de plusieurs classes. Contrairement à une classe, une interface peut hériter de plusieurs interfaces. Dans la déclaration de l'interface, le mot-clé `extends` sera suivi par la liste des noms d'interfaces séparés par des virgules.

```
public interface NomInterfaceFille extends NomInterfaceMere1, NomInterfaceMere2, ... {  
    //constantes et méthodes de l'interface NomInterfaceFille  
}
```

- Une classe qui implémente `NomInterfaceFille` doit implémenter toutes les méthodes abstraites des deux interfaces `NomInterfaceFille` et `NomInterfaceMere`. Sinon, elle devient une classe abstraite.
- Voir un exemple dans la diapositive suivante.

Héritage entre les interfaces

- La classe `Chien` doit implémenter toutes les méthodes des interfaces `Amical` et `DeCompagnie`. Sinon, elle sera abstraite.



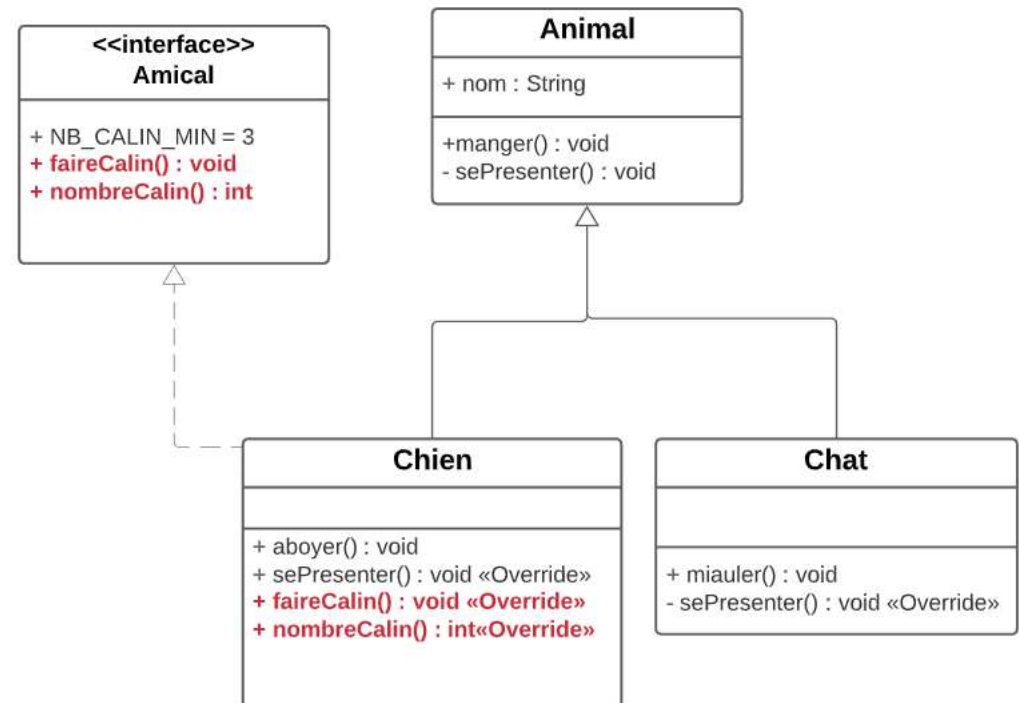
Déclaration d'une variable objet d'une interface

- **On ne peut pas instancier une interface avec l'opérateur `new`** puisqu'une interface n'est pas une classe.
- Par contre, on peut :
 - Déclarer une variable de type interface
 - Y affecter un objet d'une classe qui implémente l'interface
- Il est possible d'utiliser l'opérateur `instanceof` pour vérifier si un objet implémente une interface.
- Exemples:

```
Amical chien1 = new Amical();
// impossible puisque Amical n'a pas de
// constructeur: c'est une interface.
```

```
Amical amical = new Chien("Pitou");
// Ok. Chien est une classe non abstraite
// qui implémente l'interface Amical
```

```
Chien chien = new Chien("Adolph");
if (chien instanceof Amical) { // vrai
    System.out.println
        ("L'objet chien implémente Amical");
}
```



Déclaration d'une variable objet d'une interface

■ Exemples :

Lavable lavable

```
= new Lavable(...); // Pas Ok.  
= new Fenetre(...); // Ok.  
= new Voiture(...); // Ok.  
= new Tasse(...); // Ok.  
= new Chien(...); // Ok.  
= new TasseCafe(...); // Ok.
```

lavable.éléments accessibles de Lavable

lavable.laver(); // Méthode laver() de la classe appropriée.

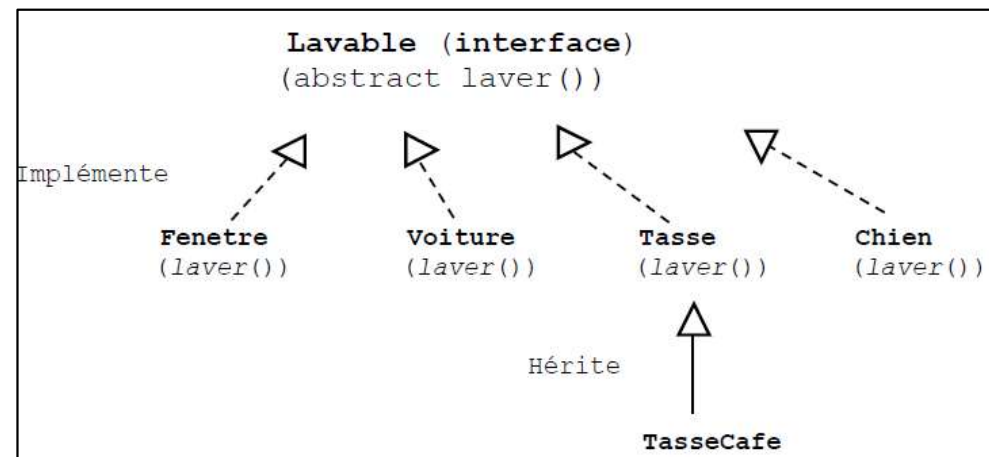
// Polymorphisme.

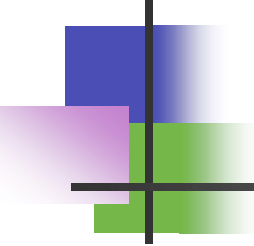
Tasse tasse = new Tasse(...);

tasse.éléments accessibles de Tasse et de Lavable

TasseCafe tasseCafe = new TasseCafe(...);

tasseCafe.éléments accessibles de TasseCafe, de Tasse et de Lavable





L'interface Comparable

- L'interface Comparable est utilisée pour classer les objets d'une classe définie par l'utilisateur. Cette interface contient une seule méthode nommée `compareTo (Object)`.
- Ainsi, pour comparer des objets d'une classe, on utilise la méthode `compareTo()`. La classe doit implémenter l'interface Comparable. <https://docs.oracle.com/javase/8/docs/api/java/lang/Comparable.html>

```
interface Comparable <T>
    int compareTo(T o);
}
```
- La méthode `compareTo(T o)`
 - Reçoit en paramètre un autre objet du même type que l'objet avec lequel on veut le comparer.
 - Elle retourne :
 - **0** si l'objet reçu en paramètre est **égal** à l'objet en cours (`this`).
 - un entier **négalif** (<0) si l'objet courant (`this`) est **inférieur** à l'objet spécifié en paramètre.
 - un entier **positif** (>0) si l'objet courant (`this`) est **supérieur** à l'objet spécifié en paramètre.

L'interface Comparable

■ Pour vérifier si un objet1 est plus petit qu'un autre objet2, on écrit :

```
if ( obj1.compareTo(obj2) < 0 ) {  
    // obj1 est plus petit que obj2.  
}
```

Et non pas

```
if ( obj1 < obj2 ) {  
// L'adresse obj1 est plus petite que l'adresse obj2.  
}
```

■ Pour vérifier si un objet1 est plus grand qu'un autre objet2, on écrit :

```
if ( obj1.compareTo(obj2) > 0 ) {  
    // obj1 est plus grand que obj2.  
}
```

Et non pas

```
if ( obj1 > obj2 ) {  
// L'adresse obj1 est plus grand que l'adresse obj2.  
}
```

■ Pour vérifier si un objet1 est égal à un autre objet2, on écrit :

```
if ( obj1.compareTo(obj2) == 0 ) {  
    // obj1 est égal (équivalent) à obj2.  
}
```

Et non pas

```
if ( obj1 == obj2 ) {  
// L'adresse obj1 est égale à l'adresse obj2.  
}
```



L'interface Comparable:

Exemple: en redéfinissant la méthode `compareTo()` de la classe `Produit`, on peut établir une relation d'ordre entre les objets `Produit` basée sur l'attribut `numero`.

```
public class Produit implements Comparable<Produit> {
    private int numero;
    private String description;    private double prix;
    public Produit(int numero, String description, double prix) {
        this.numero = numero;
        this.prix = prix;
        this.description = description;
    }
    @Override
    public int compareTo(Produit autreObjet) {
        int compNumero;
        if (this.numero < autreObjet.numero) {
            compNumero = -1;
        } else if (this.numero > autreObjet.numero) {
            compNumero = 1;
        } else {
            compNumero = 0;
        }
        return compNumero;
    }
}
```

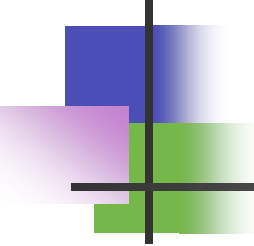
L'interface Comparable: exemple

- On peut comparer ainsi des objets de la classe Produit

```
public static void main(String[] args) {  
    Produit produit1 = new Produit(10, "Chaise", 132.56);  
    Produit produit2 = new Produit(20, "Table", 98.50);  
    Produit produit3 = new Produit(3, "Bureau", 120.00);  
  
    if (produit1.compareTo(produit2) < 0) {  
        System.out.println("produit1 est classé avant produit2 selon la relation d'ordre ");  
    } else if (produit1.compareTo(produit2) > 0) {  
        System.out.println("produit1 est classé après produit2 selon la relation d'ordre ");  
    } else {  
        System.out.println("produit1 et produit2 ont le même classement selon la relation  
            d'ordre ");  
    }  
}
```

- Affichage à la console:

```
produit1 est classé avant produit2 selon la relation d'ordre
```



Sources

- RÉSUMÉ DOCUMENT 7 INTERFACES par Soti Kyprianou et Christian Mongeon



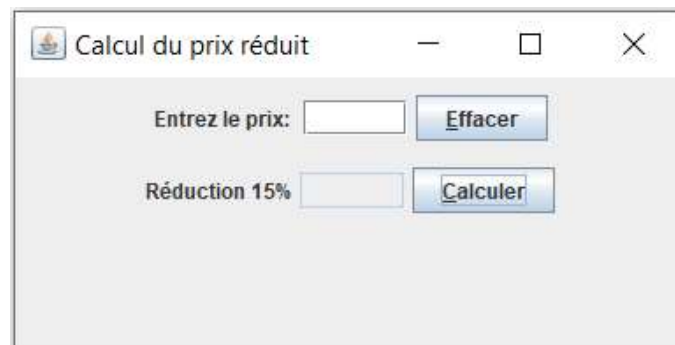
LES INTERFACES GRAPHIQUES EN JAVA

Partiel

Introduction

- Jusqu'à présent, nous n'avons implémenté que des programmes à **interface console**. Dans cette partie du cours, nous allons développer des programmes simples à interfaces graphiques.
- Dans les programmes à interface console, **c'est le programme qui pilote l'utilisateur** en le sollicitant au moment voulu pour qu'il fournisse des informations **sous forme de texte** dans la console.
- Dans un programme à interfaces graphiques, c'est **l'utilisateur qui a l'impression de piloter le programme**. L'utilisateur exprime des demandes en cliquant sur des boutons, en sélectionnant des options de menu, ou en remplissant des boîtes de dialogue et le programme réagit à cette demande. Ce type de programmation s'appelle « **la programmation événementielle** »,

Exemple : Calcul du prix après une réduction de 15%



Les bibliothèques

- Pour développer des interfaces graphiques en java, nous nous servons des classes et interfaces du package **java.awt** (Abstract Window Toolkit) et du package **javax.swing**.
- Le package Swing est plus riche que le package AWT, mais **tous les composants de Swing dérivent d'une classe de l'AWT**
- Swing ne remplace pas AWT, mais ne fait que la compléter.

• AWT

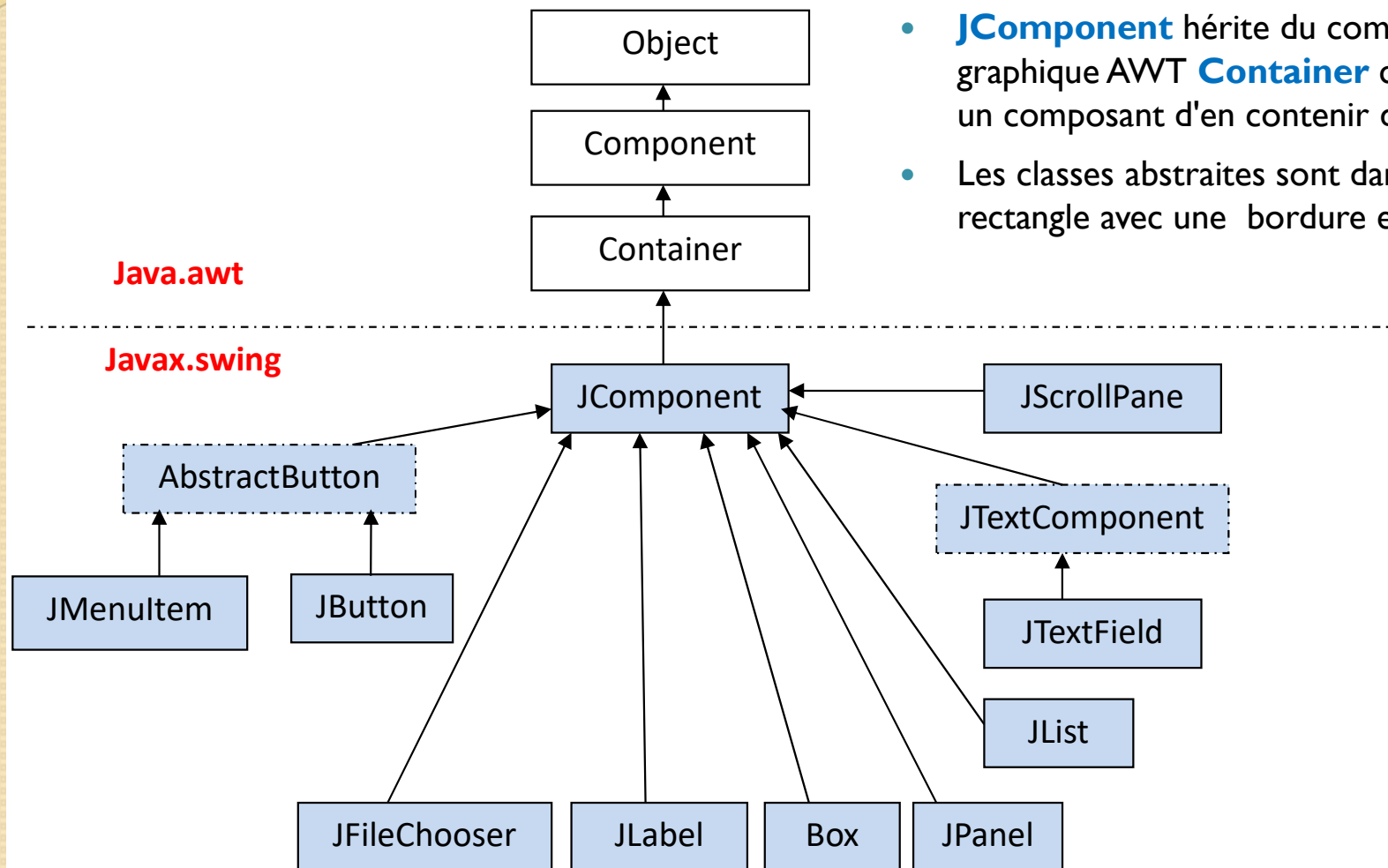
- Elle constitue le package **java.awt** dès le JDK 1.0.
- Les composants sont dessinés et contrôlés par le système d'exploitation. On dit qu'ils sont des **composants lourds**
- Inconvénients:
 - **L'apparence des composants change d'un système à un autre.**
- Avantages:
 - AWT nécessite moins de temps CPU, puisqu'elle utilise directement le système d'exploitation.

• SWING

- Constitue le package **javax.swing** (à partir du JDK 1.2)
- La plupart des composants Swing sont complètement pris en charge par la machine virtuelle (JVM). On dit qu'ils sont des **composants légers**.
- Avantages:
 - L'exécution est indépendante de la plateforme = portabilité totale pour la plupart des composantes.
- Inconvénients:
 - la bibliothèque Swing nécessite plus de temps CPU, car la machine virtuelle doit dessiner les moindres détails des composants.

Hiérarchie partielle des composants Swing

- les noms de composants Swing commencent généralement par **J**
- Plusieurs composants héritent du composant graphique **JComponent**
- **JComponent** hérite du composant graphique AWT **Container** qui permet à un composant d'en contenir d'autres
- Les classes abstraites sont dans un rectangle avec une bordure en pointillés.



Les d'objets graphiques

- Swing dispose de 3 types d'objets graphiques
- Les objets de haut niveau qui permettent de définir une fenêtre.
 - **JFrame**: fenêtre normale
 - **JWindow**: fenêtre non décorée
 - **JDialog**: boîte de dialogue
 - **JApplet**: destiné à être téléchargé et exécuté par un navigateur du poste client

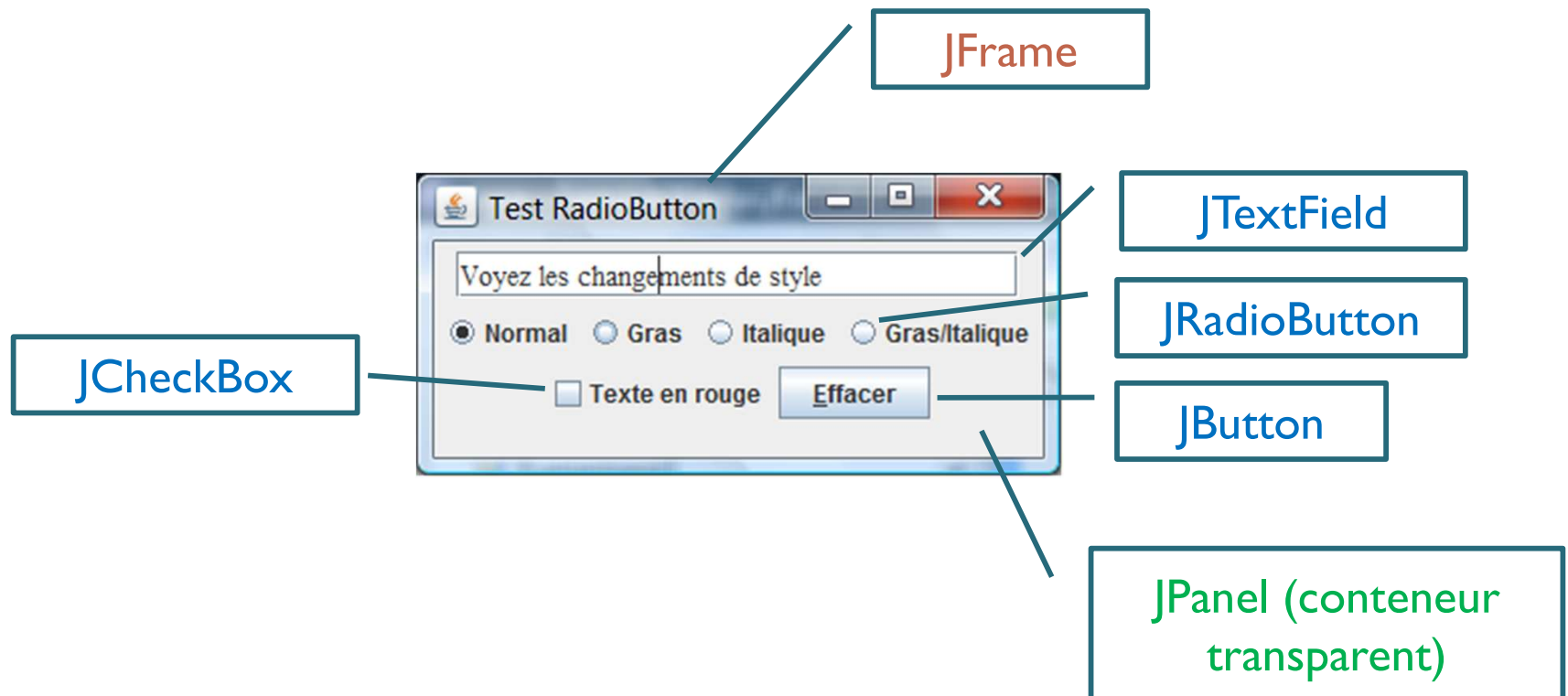
Les fenêtres Swing sont les seuls composants lourds, puisqu'elles sont essentiellement liées au système d'exploitation.

- Les objets de niveau intermédiaire: pour composer la fenêtre
 - **JPanel**: conteneur transparent
 - **JScrollPane**: barre de défilement
 - **JSplitPane**: pour diviser deux Components.
 - ...
- Les objets de niveau inférieur: les composants de base
 - **JButton**,
 - **JCheckBox**,
 - **JLabel**,
 - **JTextField**,
 - **JTextArea**,



Les d'objets graphiques

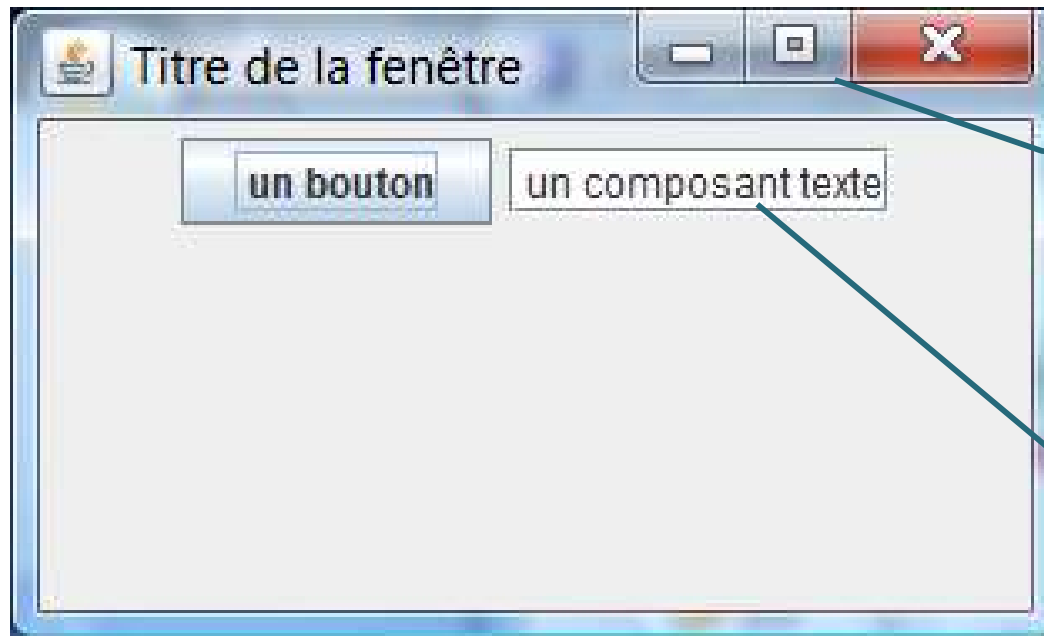
- Objet de haut niveau
- Objet de niveau intermédiaire
- Objet de niveau inférieur





La fenêtre de type JFrame

Le composant JFrame



L'affichage de la fenêtre dépend du système d'exploitation: **composant lourd**.

L'affichage du bouton et de la zone de texte est du style de Swing : identique sur tous les systèmes d'exploitation. Ce sont des **composants légers**.

Le composant **JFrame** possède un **titre**, une **taille** modifiable, une **icône**, des **boutons de redimensionnement** et éventuellement une **barre de menu**.

Création d'une fenêtre

- Pour créer une fenêtre, nous pouvons simplement créer un objet de type **JFrame**. Puis, nous utiliserons les fonctionnalités présentes dans cette classe.

```
import javax.swing.JFrame;

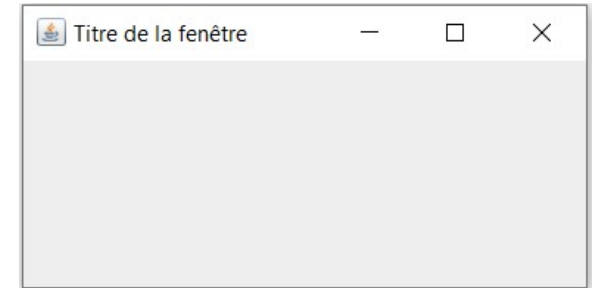
public class Fenetre0 {

    public static void main(String arg[]) {

        //construire la fenêtre en définissant son titre
        JFrame fenetre = new JFrame("Titre de la fen\u00Eatre");
        // préciser sa taille: largeur et hauteur. Par défaut 0,0
        fenetre.setSize(400, 200);
        // préciser le comportement de la fenêtre lors de la fermeture
        fenetre.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // rendre la fenêtre visible. Par défaut, elle est invisible
        fenetre.setVisible(true);
        // Ne pas modifier la fenêtre après qu'elle soit visible

    }

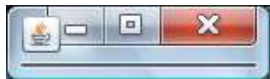
}
```



Création d'une fenêtre

- Quelques constructeurs de JFrame

Constructeur	Exemple
<code>JFrame()</code>	<code>JFrame fen = new JFrame();</code>
<code>JFrame(String)</code>	<code>JFrame fen = new JFrame("Un titre");</code>

- Pour afficher la fenêtre à l'écran, on doit **spécifier sa taille** avec la méthode ***setSize(int largeur, int hauteur)***.
 - Par défaut largeur = hauteur = 0. 
 - Exemple: pour une fenêtre de 400 pixels de largeur et 200 pixels de hauteur, on écrit:
`fenetre.setSize(400, 200);`
- Il faut **demandeur l'affichage** de la fenêtre en appelant la méthode ***setVisible(boolean b)***.
 - Par défaut une fenêtre est invisible.
 - Exemple: `fenetre.setVisible(true);`

Création d'une fenêtre

- On doit fermer la fenêtre. On utilise la méthode:
`setDefaultCloseOperation(int operation)`.
 - Par défaut, la fenêtre est simplement cachée quand elle est fermée (si on clique sur le bouton X).

- Exemples:

//fermer l'application

```
fenetre.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

//rend le bouton X inactif

```
fenetre.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
```

Création d'une classe fenêtre personnalisée

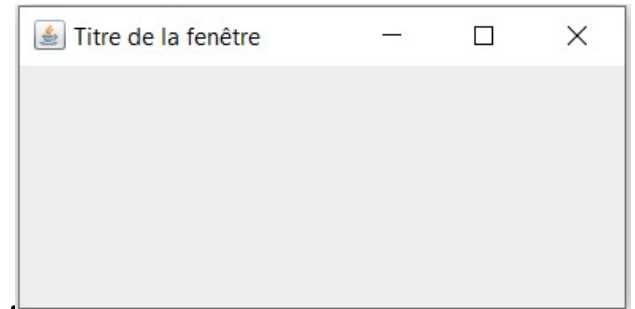
- Pour construire une fenêtre, on crée une classe dérivée de **JFrame**, puis on crée un objet de ce nouveau type.

```
import javax.swing.JFrame;

public class Fenetre1 extends JFrame {

    // Constructeur
    public Fenetre1() {
        super("Titre de la fen\u00EAtre ");
        setSize(400, 100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

public class Application1 {
    public static void main (String arg[]){
        Fenetre1 fenetre = new Fenetre1();
        fenetre.setVisible(true);
    }
}
```

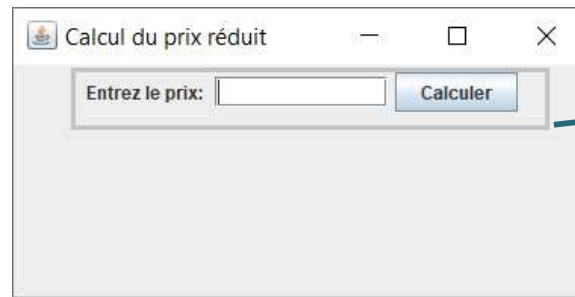


- On peut écrire autrement le code de Fenetre1. On peut utiliser le constructeur sans paramètre et la méthode **setTitle()** pour attribuer le titre.

```
public Fenetre1() {
    super();
    setTitle("Titre de la fen\u00EAtre");
    setSize(400, 100);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
```

Ajouter des composants à une fenêtre

- Les étapes à suivre pour ajouter des composants à une fenêtre sont:
 1. Créer des composants intermédiaires, exemple un `JPanel`
 2. Créer des composants de bases, exemples `JButton`, `TextField`,...
 3. Insérer les composants de bases dans les composants intermédiaires
 4. insérer les composants intermédiaires dans la fenêtre.
- Exemple:
 1. Créer un panneau
 2. Créer le bouton(calculer), l'étiquette (entrez le prix) et la zone de texte (vide)
 3. Ajoutez le bouton, l'étiquette (entrez le prix) et la zone de texte (vide) au panneau
 4. Ajoutez le panneau à la fenêtre.



Panneau transparent

Ajouter des composants à une fenêtre

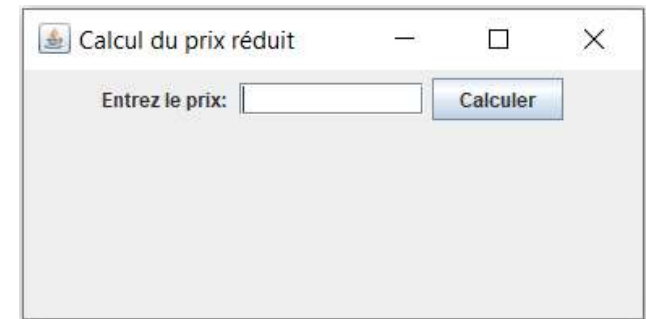
- Exemple: package applicationI

```
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;

public class FrmPrixReduit extends JFrame {
    // créer les composants de base
    private JButton btnCalculer = new JButton("Calculer");
    private JTextField txtPrix = new JTextField(10); // largeur 10
    private JLabel lblPrix = new JLabel("Entrez le prix: ");
    // créer le composant intermédiaire: le panneau
    private JPanel panell = new JPanel(); // créer un panneau: conteneur vide transparent

    public FrmPrixReduit () {
        super("Calcul du prix réduit");
        setSize(400, 200);
        //ajouter les composants au panneau
        panell.add(lblPrix);
        panell.add(txtPrix);
        panell.add(btnCalculer);
        //ajouter le panneau à la fenêtre
        add(panell);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

public class ApplicationPrixReduit { //la classe principale de l'application
    public static void main (String arg[]){
        FrmPrixReduit fenetre= new FrmPrixReduit ();
        fenetre.setVisible(true);
    }
}
```



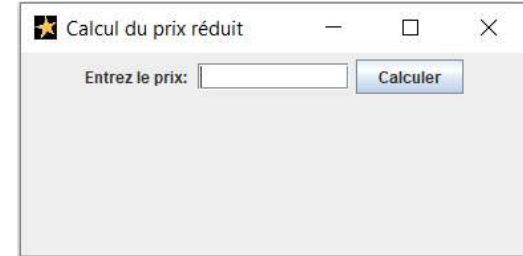
La personnalisation de l'icône de la fenêtre

- Exemple: package application2
- La méthode `setIconImage(Image image)` permet de modifier l'icône de la JFrame.
- Si l'image est trop grande, elle est redimensionnée.
- Exemple:

```
public class FrmPrixReduit extends JFrame {
    // créer les composants de base
    private JButton btnCalculer = new JButton("Calculer");
    private JTextField txtPrix = new JTextField(10); // largeur 10
    private JLabel lblPrix = new JLabel("Entrez le prix: ");
    // créer le composant intermédiaire: le panneau
    private JPanel panell = new JPanel(); // créer un panneau: conteneur

    public FrmPrixReduit () {
        super("Calcul du prix réduit");
        setSize(400, 200);
        ImageIcon iconeFenetre = new ImageIcon(FrmPrixReduit.class.getResource("/images/etoile.jpg"));
        // chemin relatif de l'image
        setIconImage(iconeFenetre.getImage());

        //ajouter les composants au panneau
        panell.add(lblPrix);
        panell.add(txtPrix);
        panell.add(btnCalculer);
        //ajouter le panneau à la fenêtre
        add(panell);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```





les composants de base

La classe JLabel (étiquette): constructeurs

- Un JLabel est un texte **non éditable**. Il peut être accompagné d'une icône.
- Quelques constructeurs

Constructeurs	Rôle
JLabel()	Création d'une instance sans texte ni image.
JLabel(Icon image)	Création d'une instance en précisant l'image.
JLabel(String text)	Création d'une instance en précisant le texte

- Exemples de construction de composants JLabel.

```
JLabel label1 = new JLabel("label1 ");  
ImageIcon icone = new ImageIcon(Etiquettes.class.getResource("images/etoile.jpg"));  
JLabel label2 = new JLabel(icone);
```



La classe JLabel (étiquette): quelques méthodes

Méthode	Rôle
setText()	Permet d'initialiser ou de modifier le texte affiché
setOpaque(boolean)	Indique si le composant est transparent (paramètre false) ou opaque (true)
setBackground()	Indique la couleur de fond du composant (setOpaque doit être à true)
setFont()	Permet de préciser la police du texte
setForeground()	Permet de préciser la couleur du texte
setHorizontalAlignment()	Permet de modifier l' alignement horizontal du texte et de l'icône
setIcon()	Permet d'assigner une icône

Exemples: modifier les caractéristiques d'un objet JLabel après sa construction.

- Lors de la construction

```
JLabel label1 = new JLabel("Ancienne étiquette");
```

Ancienne étiquette

- Modification du texte et de la couleur de texte

```
label1.setText("Nouvelle étiquette");  
label1.setForeground(Color.RED);  
label1.setOpaque(true);  
label1.setBackground(Color.CYAN);
```

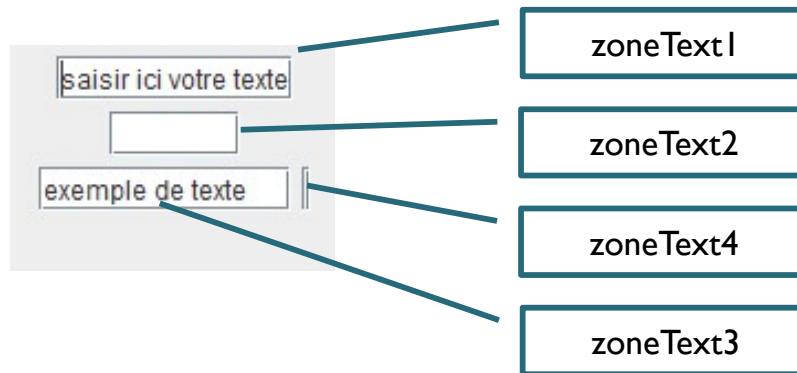
Nouvelle étiquette

La classe JTextField

- Permet la saisie d'une seule ligne de texte.

- Quelques exemples de construction

```
JTextField zoneText1 = new JTextField("saisir ici votre texte");  
JTextField zoneText2 = new JTextField(5); //longueur 5  
JTextField zoneText3 = new JTextField("exemple de texte",10);  
JTextField zoneText4 = new JTextField(); //longueur 0
```



- Pour modifier le texte

```
setText(String texte);
```

- Exemple:

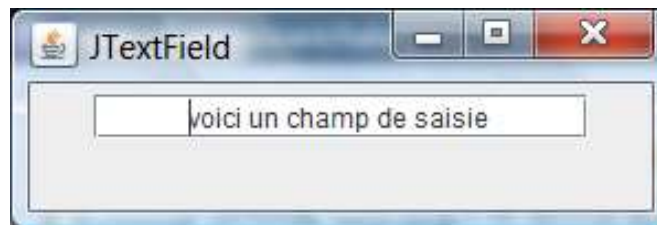
```
zoneText.setText("Nouveau texte");
```

La classe JTextField

- **Une zone de texte est par défaut éditable** (on peut écrire à l'intérieur). Pour rendre une zone de texte non éditable, on utilise `setEditable(false)`
- La méthode `setHorizontalAlignment(int alignment)` permet de préciser l'alignement du texte dans le composant en utilisant les constantes:
 - `JTextField.LEFT` ,
 - `JTextField.CENTER`
 - `JTextField.RIGHT`.

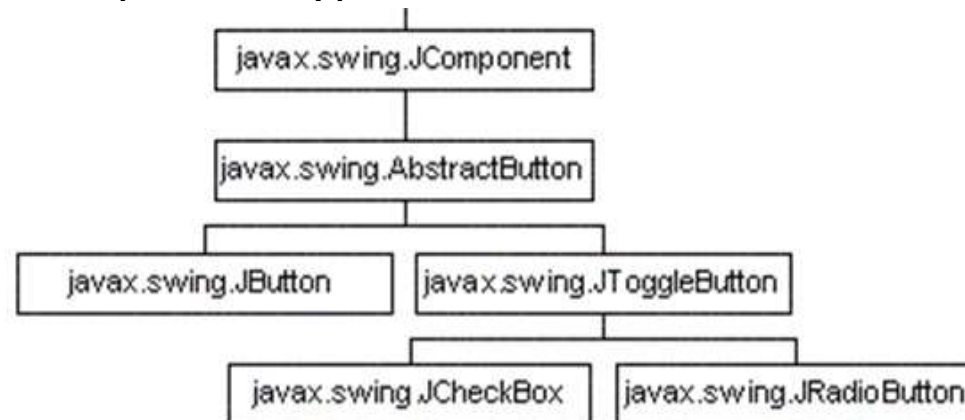
- Exemple

```
JTextField message = new JTextField(20);  
message.setHorizontalAlignment(JTextField.CENTER);  
String phrase = new String("voici un champ de saisie");  
message.setText(phrase);
```



Les boutons

- Il existe plusieurs types de boutons définis dans Swing



• **JButton**: bouton classique
• **JToggleButton**: bouton qui reste enfoncé:

- Toutes les classes de boutons héritent de la classe abstraite `AbstractButton`.** Cette classe définit de nombreuses méthodes communes à tous les boutons. Voir la diapositive suivante.
- Quelques méthodes de la classe **`AbstractButton`**

Méthode	Rôle
<code>doClick()</code>	Déclencher un clic par programmation
<code>getText()</code>	Obtenir le texte affiché par le composant
<code>setText()</code>	Mettre à jour le texte du composant
<code>isSelected()</code>	Indiquer si le composant est dans l'état sélectionné
<code>setSelected()</code>	Mettre à jour l'état sélectionné du composant

La classe JButton

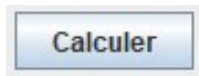
- Un bouton peut contenir un texte et/ou une icône.
- Quelques constructeurs

Constructeur	Rôle
<code>JButton()</code>	Construire un bouton sans texte ni icône
<code>JButton(String text)</code>	Préciser le texte du bouton
<code>JButton(Icon image)</code>	Préciser une icône
<code>JButton(String text, Icon image)</code>	Préciser un texte et une icône

- Toutes les méthodes concernant le contenu du composant `JLabel` sont valables pour le composant `JButton`.

- Exemple:

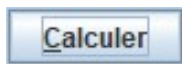
```
JButton btnCalculer = new JButton("Calculer");
```



- Si on veut ajouter un raccourci clavier sur la lettre 'C' de sorte à actionner le bouton avec la combinaison des touches ALT + 'C', on utilise la méthode `setMnemonic(int mnemonic)`.

- Exemple: pour un raccourci avec la lettre 'C'.

```
btnCalculer.setMnemonic(KeyEvent.VK_C);
```



La classe JCheckBox



- Quelques constructeurs

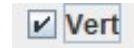
Constructeur	Rôle
<code>JCheckBox()</code>	créer une case à cocher vierge
<code>JCheckBox(String text)</code>	précise l'intitulé
<code>JCheckBox(String text, boolean selected)</code>	précise l'intitulé et l'état (sélectionné ou non)

- Exemples

```
private JCheckBox checkRouge = new JCheckBox("Rouge");
```

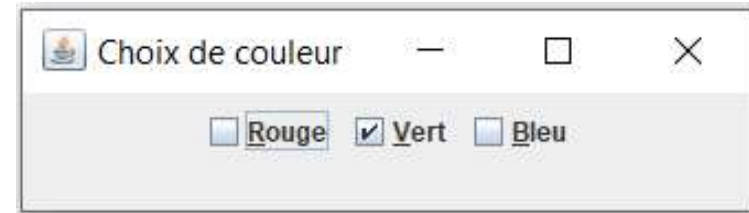


```
private JCheckBox checkVert = new JCheckBox("Vert", true);
```



La classe JCheckBox

- Exemple: FrmCaseACocher.java



```
public class FrmCaseACocher extends JFrame {
    private JCheckBox checkRouge = new JCheckBox("Rouge");
    private JCheckBox checkVert = new JCheckBox("Vert", true);
    private JCheckBox checkBleu = new JCheckBox("Bleu");

    public FrmCaseACocher() {
        super();
        setTitle("Choix de couleur");
        setSize(350, 100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JPanel panneau = new JPanel();

        // Les mnémonique
        checkRouge.setMnemonic(KeyEvent.VK_R);
        checkVert.setMnemonic(KeyEvent.VK_V);
        checkBleu.setMnemonic(KeyEvent.VK_B);

        panneau.add(checkRouge);
        panneau.add(checkVert);
        panneau.add(checkBleu);
        add(panneau);
    }

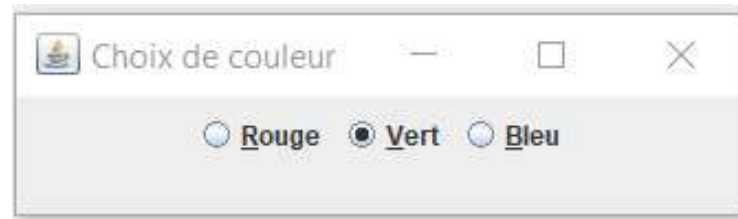
    public static void main(String args[]) {
        FrmCaseACocher fenetre = new FrmCaseACocher();
        fenetre.setVisible(true);
    }
}
```


La classe JCheckBox

- Quelques méthodes intéressantes héritées de **AbstractButton** :
- `public void setText(String text)`
 - Pour spécifier le texte de la case à cocher.
- `public void setIcon (Icon icon)`
 - Pour spécifier l'image de la case à cocher.
- `public void setSelected (boolean b)`
 - Pour spécifier l'état de la case à cocher.
- `public boolean isSelected ()` :
 - Retourne l'état de la case à cocher.
- `public void setMnemonic(int mnemonic)`
 - Associe un mnémonique (combinaison alt + touche)

La classe JRadioButton

- Les paramètres des constructeurs sont similaires à ceux de la classe **JCheckBox**.



- Exemple:

```
private JRadioButton radioRouge = new JRadioButton("Rouge");
```



```
private JRadioButton radioVert = new JRadioButton("Vert", true);
```



Les groupes de boutons

- La classe **ButtonGroup** nous permet de créer un groupe de boutons **pour n'avoir qu'un bouton sélectionné à la fois.**
- **ButtonGroup** peut être utilisée avec
 - `JToggleButton` (ne seront pas vus dans ce cours)
 - `JCheckBox`
 - `JRadioButton`
- On ajoute un bouton au groupe avec la méthode **`add()`**
- **ButtonGroup** est un conteneur logique **et ne doit pas être ajouté** à la fenêtre.
- **On ne peut pas** désélectionner un bouton se trouvant dans un **ButtonGroup** en cliquant dessus (voir l'exemple de la diapositive suivante).

Les groupes de boutons

- Exemple: FrmRadio.java

```
public class FrmRadio extends JFrame {
    private JRadioButton radioRouge = new JRadioButton("Rouge");
    private JRadioButton radioVert = new JRadioButton("Vert");
    private JRadioButton radioBleu = new JRadioButton("Bleu");
    private ButtonGroup groupe = new ButtonGroup();
    private JPanel panneau = new JPanel();

    public FrmRadio() {
        super();
        setTitle("Choix de couleur");
        setSize(300, 100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Les mnémonique
        radioRouge.setMnemonic(KeyEvent.VK_R);
        radioVert.setMnemonic(KeyEvent.VK_V);
        radioBleu.setMnemonic(KeyEvent.VK_B);

        // Selectionner le bouton rougeRadio
        radioRouge.setSelected(true);
        // ajouter les bouton au groupe
        groupe.add(radioRouge);
        groupe.add(radioVert);
        groupe.add(radioBleu);

        //ajouter les boutons au panneau
        panneau.add(radioRouge);
        panneau.add(radioVert);
        panneau.add(radioBleu);

        // ajouter le panneau à la fenêtre
        add(panneau);
    }
}
```

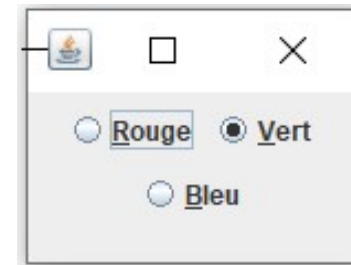




Les gestionnaires de disposition (layout manager)

La disposition des composants dans la fenêtre

- Les fenêtres utilisent un "layout manager" (ou gestionnaire de mise en forme) pour positionner leurs composantes.
- Il existe plusieurs layouts:
 - BorderLayout: place aux 4 coins cardinaux
 - FlowLayout: place les composants un à la suite de l'autre comme dans un panneau
 - GridLayout: place dans une grille
 - GridBagLayout: placements complexes
 - BoxLayout: place les composants horizontalement ou verticalement
 - ...
- **Le gestionnaire par défaut d'un composant JPanel est FlowLayout:** les composants sont placés un à la suite de l'autre. S'il n'y a pas d'espace, le composant se place à la ligne suivante.



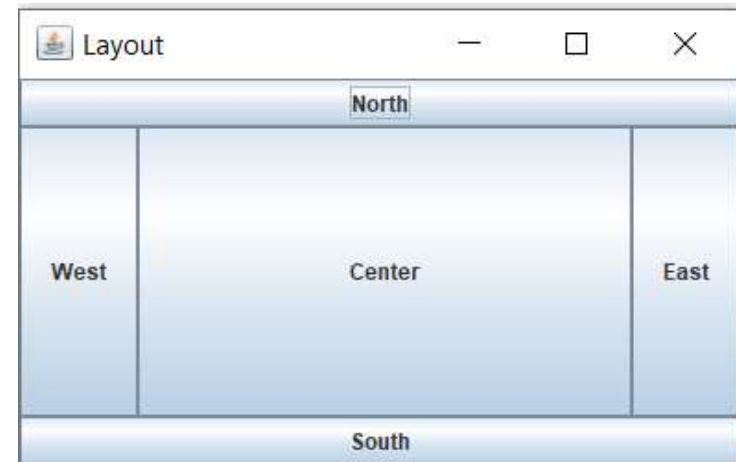
- **Le gestionnaire utilisé par défaut par un JFrame est le BorderLayout** (voir la diapositive suivante).

La disposition des composants dans la fenêtre

- Le gestionnaire utilisé par défaut par un `JFrame` est le `BorderLayout`.
- Lorsqu'on fait appel à la méthode `add(composant)` pour ajouter un composant à la fenêtre, celui-ci est **ajouté au centre**.
- On peut utiliser la fonction `setLayout()` pour changer le layout utilisé. Exemple
`setLayout(new FlowLayout());`

- Exemple: `FrmLayout.java`

```
public class FrmLayout extends JFrame {  
    JButton button1 = new JButton("North");  
    JButton button2 = new JButton("South");  
    JButton button3 = new JButton("East");  
    JButton button4 = new JButton("West");  
    JButton button5 = new JButton("Center");  
  
    public FrmLayout() {  
        super("Layout");  
        add(button1, BorderLayout.NORTH);  
        add(button2, BorderLayout.SOUTH);  
        add(button3, BorderLayout.EAST);  
        add(button4, BorderLayout.WEST);  
        add(button5, BorderLayout.CENTER);  
        setSize(256, 256);  
        setVisible(true);  
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    }  
}
```





LA GESTION DES ÉVÈNEMENTS

La gestion des évènements

- L'interaction entre l'utilisateur et une interface graphique engendre des **évènements**. Par exemple, cliquer sur un bouton, appuyer sur une touche du clavier et fermer une fenêtre engendrent des évènements.
- Tous les composants graphiques peuvent engendrer des évènements.
- Les évènements sont représentés par des instances des sous-classes de `java.util.EventObject`
<http://docs.oracle.com/javase/7/docs/api/java/util/EventObject.html>
- Exemples d'évènements
 - `ActionEvent`: lorsque le composant est actionné (exemple clic sur un bouton)
 - `MouseEvent`: lorsqu'on a cliqué ou déplacé la souris sur un composant.
 - `FocusEvent`: lorsqu'un composant gagne ou perd le focus
 - `ComponentEvent`: lorsqu'un composant est déplacé, redimensionné, rendu visible ou invisible
 - `TextEvent`: lorsque le texte(comme un `textField`) est modifié.

La gestion des évènements

- En langage Java, la gestion des évènements est réalisée par l'intermédiaire d'objets spécifiques appelés **écouteurs** (en anglais **listener**). De façon simplifiée, on peut dire que lorsque l'utilisateur clique sur un bouton, ce dernier émet un évènement à l'attention de l'écouteur.
- Les **écouteurs** sont des interfaces qui héritent de l'interface **EventListener** comme `ActionListener`, `WindowListener`, `MouseListener`, ...
- **Exemple**
 - Lorsqu'on actionne un bouton (en utilisant la souris ou le clavier), il se produit un évènement **ActionEvent**.
 - Pour être capable d'écouter et de traiter un tel évènement, un objet écouteur(sa classe) se doit d'implémenter une interface nommée **ActionListener**.
 - L'interface `ActionListener` ne contient qu'une unique méthode à implémenter : `actionPerformed(ActionEvent e)`. Cette méthode doit donc contenir le code à exécuter si jamais on clique sur ce bouton.
 - On doit ensuite enregistrer l'écouteur auprès du bouton qu'il est censé écouter (surveiller l'évènement action).
- **L'interface `ActionListener` :**
 - <https://docs.oracle.com/javase/8/docs/api/java/awt/event/ActionListener.html>

```
public interface ActionListener extends EventListener{  
    void actionPerformed(ActionEvent e);  
}
```

Mise en place d'un gestionnaire d'évènement

- La mise en place d'un écouteur, ou gestionnaire d'évènement nécessite 3 étapes. Nous allons expliquer ces étapes pour un écouteur qui gère des évènements d'action sur un bouton, soit un ActionListener.
- **Étape 1: Créer l'écouteur (gestionnaire d'évènement)**

On peut créer une classe interne à la classe graphique qui implémente l'interface ActionListener. Par exemple, CalculerListener. **Il existe d'autres possibilités** qu'on ne verra pas dans ce cours.

```
class CalculerListener implements ActionListener {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        // écrire le code qui doit s'exécuter quand un bouton est actionné  
    }  
}
```

- **Étape 2: Instancier l'écouteur (le gestionnaire d'évènement)**

```
ActionListener ecouteur = new CalculerListener();
```

- **Étape 3: Enregistrer l'écouteur (le gestionnaire d'évènement) auprès du bouton qui génère l'évènement avec la méthode addActionListener().**

```
btnCalculer.addActionListener(ecouteur); // on met le bouton sur écoute
```

Ou bien

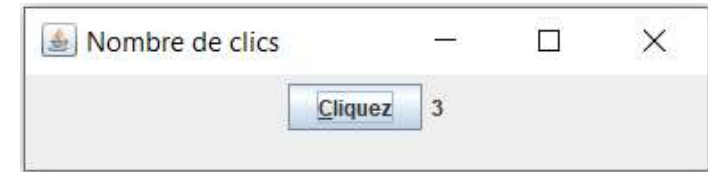
```
btnCalculer.addActionListener(new CalculerListener());
```

Mise en place d'un gestionnaire d'évènement: exemple I

- On affiche le nombre de clics sur le bouton «Cliquez».

```
public class FrmNombreClics extends JFrame {  
    private JButton btnClic = new JButton("Cliquez");  
    private JLabel lblNbClics = new JLabel("0");  
    private int compteurClics = 0;
```

```
    public FrmNombreClics() {  
        super("Nombre de clics");  
        setLayout(new FlowLayout()); // modifier le layout de la fenêtre  
        setSize(400, 100);  
        btnClic.setMnemonic(KeyEvent.VK_C);  
  
        // mettre btnClic sur écoute  
        btnClic.addActionListener(new ClicListener());  
  
        add(btnClic);  
        add(lblNbClics);  
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    }
```



Chaque fois que le bouton est actionné, il cherche à invoquer la méthode `actionPerformed()` sur chacun de ses écouteurs. Ici on a un seul écouteur.

```
class ClicListener implements ActionListener {  
  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        compteurClics++;  
        lblNbClics.setText(String.valueOf(compteurClics));  
    }  
}
```

Le traitement qui sera exécuté quand le bouton est actionné.
Le paramètre `e` qualifie l'évènement qui survient

C'est une classe interne définie dans la classe `FrmNombreClics`. Elle accède à tous les attributs même ceux privés, de cette classe.

Mise en place d'un gestionnaire d'évènement: exemple2

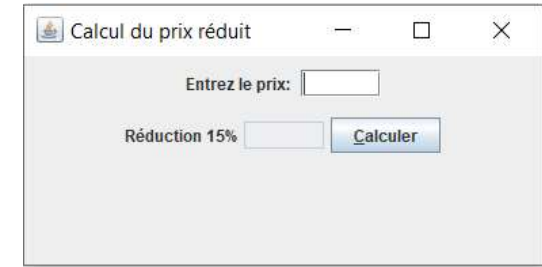
```
public class FrmCalculer extends JFrame {
    private JButton btnCalculer = new JButton("Calculer");
    private JTextField txtPrix = new JTextField(5);
    private JLabel lblPrix = new JLabel("Entrez le prix: ");
    private JLabel lblReduction = new JLabel("Réduction 15%");
    private JTextField txtPrixReduit = new JTextField(5);
    private JPanel panel1 = new JPanel();
    private JPanel panel2 = new JPanel();
```

```
public FrmCalculer() {
    super("Calcul du prix réduit");
    setLayout(new FlowLayout());
    setSize(400, 200);
    txtPrixReduit.setEditable(false);
    btnCalculer.setMnemonic(KeyEvent.VK_C);
```

```
    // mettre btnCalculer sur écoute
    btnCalculer.addActionListener(new CalculerListener());
```

```
    panel1.add(lblPrix);
    panel1.add(txtPrix);
    panel2.add(lblReduction);
    panel2.add(txtPrixReduit);
    panel2.add(btnCalculer);
    add(panel1);
    add(panel2);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
```

```
class CalculerListener implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent e) {
        String prixTxt = txtPrix.getText();
        double prixDouble = Double.parseDouble(prixTxt); // convertir le texte en double
        double prixReduit = prixDouble - prixDouble * 0.15;
        //txtPrixReduit.setText(String.valueOf(prixReduit));
        txtPrixReduit.setText(String.format("%.2f$", prixReduit));
    }
}
```



À chaque fois que le bouton est actionné, il cherche à invoquer la méthode `actionPerformed()` sur chacun de ses écouteurs. Ici on a un seul écouteur.

Le traitement qui sera exécuté quand le bouton est actionné.
Le paramètre `e` qualifie l'évènement qui survient

C'est une classe **interne** définie dans la classe `FrmCalculer`. Elle accède à tous les attributs même ceux privés, de cette classe.

Connaitre la source de l'évènement: getSource()

- Un écouteur (ou gestionnaire d'évènements) peut écouter plusieurs sources d'évènements. Une source d'évènement peut alerter plusieurs écouteurs.
- Comment reconnaître l'objet source de l'évènement?
 - L'objet évènement *e* de la méthode *actionPerformed(ActionEvent e)* permet d'identifier le composant source de l'évènement en appelant *getSource()* :
e.getSource() = composant source de l'évènement
 - La méthode *getSource()* fournit une référence de type *Object*.
 - La méthode *getSource()* est présente dans toutes les classes évènements.

- Exemple:

```
@Override
public void actionPerformed (ActionEvent e)
{
    if (e.getSource() == bouton1)
        action1
    else if (e.getSource() == bouton2)
        action2
    ...
}
```


Connaitre la source de l'évènement: getSource()

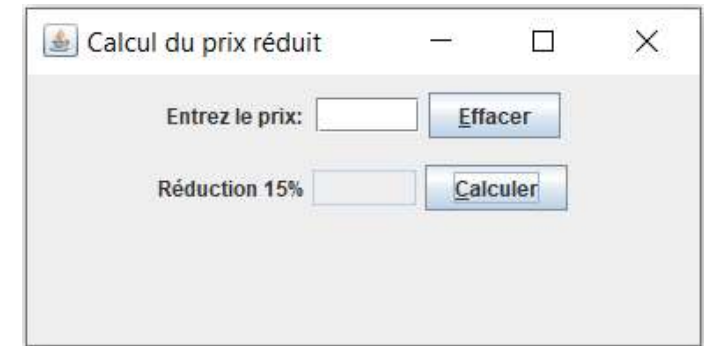
- Exemple: On ajoute un deuxième bouton qui consiste à effacer les 2 zones de texte. Si on utilise le même écouteur pour les boutons **btnCalculer** et **btnEffacer**, pour savoir quel bouton a été actionné, on utilise **e.getSource()**.

```
public class FrmCalculer2 extends JFrame {
    // créer les composants de base
    private JButton btnCalculer = new JButton("Calculer");
    private JButton btnEffacer = new JButton("Effacer");
    private JTextField txtPrix = new JTextField(5);
    private JLabel lblPrix = new JLabel("Entrez le prix: ");
    private JLabel lblReduction = new JLabel("Réduction 15%");
    private JTextField txtPrixReduit = new JTextField(5);
    // créer le composant intermédiaire: le panneau
    private JPanel panel1 = new JPanel();
    private JPanel panel2 = new JPanel();

    public FrmCalculer2() {
        super("Calcul du prix réduit");
        setLayout(new FlowLayout());
        setSize(400, 200);
        txtPrixReduit.setEditable(false);
        btnEffacer.setMnemonic(KeyEvent.VK_E);
        btnCalculer.setMnemonic(KeyEvent.VK_C);

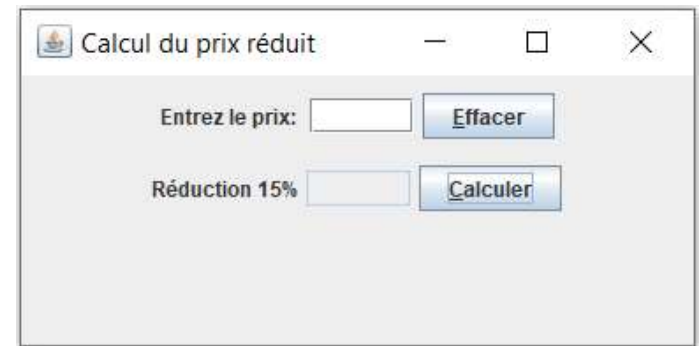
        // mettre les 2 boutons sur écoute
        btnEffacer.addActionListener(new CalculerListener());
        btnCalculer.addActionListener(new CalculerListener());

        panel1.add(lblPrix);
        panel1.add(txtPrix);
        panel1.add(btnEffacer);
        panel2.add(lblReduction);
        panel2.add(txtPrixReduit);
        panel2.add(btnCalculer);
        // ajouter le panneau à la fenêtre
        add(panel1);
        add(panel2);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```



Connaitre la source de l'évènement: getSource()

```
class CalculerListener implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == btnCalculer) {
            String prixTxt = txtPrix.getText();
            double prixDouble = Double.parseDouble(prixTxt);
            double prixReduit = prixDouble - prixDouble * 0.15;
            // txtPrixReduit.setText(String.valueOf(prixReduit));
            txtPrixReduit.setText(String.format("%.2f$", prixReduit));
            txtPrix.requestFocus(); // déplacer le curseur sur la zone de texte
            txtPrix.selectAll(); // sélectionner le contenu de la zone de texte
        } else { // la source est btnEffacer
            txtPrix.setText("");
            txtPrixReduit.setText("");
            txtPrix.requestFocus(); // déplacer le curseur sur zone de texte
        }
    }
}
```

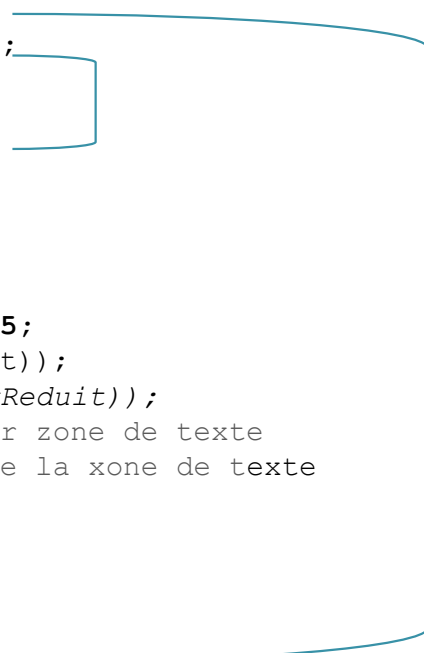


Utiliser plusieurs écouteurs

- Dans l'exemple précédent, on peut **utiliser deux gestionnaires (écouteurs)**:
 - Un gestionnaire pour effacer les zones de texte: **EffacerListener**
 - Un gestionnaire qui calcule et affiche le prix réduit: **CalculerListener**

```
public class FrmCalculer3 extends JFrame {
    private JButton btnCalculer = new JButton("Calculer");
    private JButton btnEffacer = new JButton("Effacer");
    ...
    public FrmCalculer3() {
        ...
        // mettre les 2 boutons sur écoute
        btnEffacer.addActionListener(new EffacerListener());
        btnCalculer.addActionListener(new CalculerListener());
        ...
    }
    class CalculerListener implements ActionListener {
        @Override
        public void actionPerformed(ActionEvent e) {
            String prixTxt = txtPrix.getText();
            double prixDouble = Double.parseDouble(prixTxt);
            double prixReduit = prixDouble - prixDouble * 0.15;
            // txtPrixReduit.setText(String.valueOf(prixReduit));
            txtPrixReduit.setText(String.format("%.2f$", prixReduit));
            txtPrix.requestFocus(); // déplacer le curseur sur zone de texte
            txtPrix.selectAll(); // sélectionner le contenu de la zone de texte
        }
    }

    class EffacerListener implements ActionListener {
        @Override
        public void actionPerformed(ActionEvent e) {
            txtPrix.setText("");
            txtPrixReduit.setText("");
            txtPrix.requestFocus(); // déplacer le curseur sur zone de texte
        }
    }
}
```



Traiter l'exception: le nombre saisi n'est pas du bon format

- Si on ne veut pas que le programme plante quand l'utilisateur saisit un prix qui n'est pas du bon format, on intègre la gestion des exceptions.
- L'instruction `Double.parseDouble(prixStr);` convertit la chaîne **prixStr** fournie comme argument en un nombre de type double. Cette instruction peut déclencher une exception de type **NumberFormatException** si l'argument n'est pas du bon format. Pour traiter cette exception et éviter ainsi que le programme plante, on l'entoure d'un bloc

```
try ..catch (NumberFormatException ex).
```

Exemple: FrmCalculer4.java

```
class CalculerListener implements ActionListener {
```

```
@Override
```

```
public void actionPerformed(ActionEvent e) {
```

```
    if (e.getSource() == btnCalculer) {
```

```
        String prixStr = txtPrix.getText();
```

```
        String messageErreur = " Entrez un prix valide. Exemple 12.35";
```

```
        try {
```

```
            double prixDouble = Double.parseDouble(prixStr);
```

```
            double prixReduit = prixDouble - prixDouble * 0.15;
```

```
            // txtPrixReduit.setText(String.valueOf(txtPrix));
```

```
            txtPrixReduit.setText(String.format("%.2f$", prixReduit));
```

```
            txtPrix.requestFocus(); // déplacer le curseur sur zone de texte
```

```
            txtPrix.selectAll(); // sélectionner le contenu de la zone de texte
```

```
        } catch (NumberFormatException ex) {
```

```
            JOptionPane.showMessageDialog(FrmCalculer4.this, messageErreur);
```

```
            txtPrix.requestFocus();
```

```
        }
```

```
    } else { // la source est btnEffacer
```

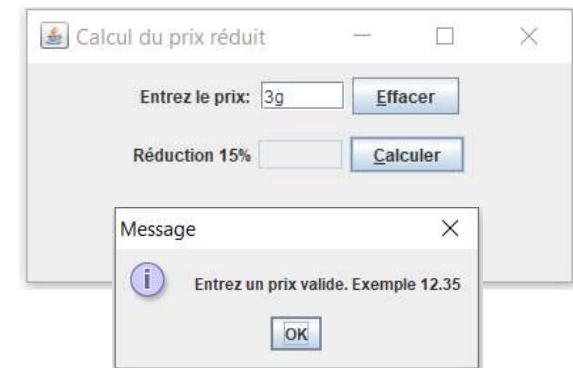
```
        txtPrix.setText("");
```

```
        txtPrixReduit.setText("");
```

```
        txtPrix.requestFocus();
```

```
    }
```

```
}
```



Si la conversion plante, le bloc catch est exécuté,

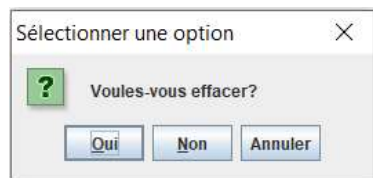
Les boîtes de dialogue

- Une boîte de dialogue est une fenêtre affichée par un programme ou par le système d'exploitation pour établir le dialogue avec l'utilisateur pour:

- informer l'utilisateur d'un évènement



- obtenir une information de l'utilisateur



- La classe **JOptionPane** et ses différentes **méthodes statiques** permettent de créer des boîtes de dialogue fournies par Java.

Les boîtes de dialogue: la boîte de message

- `static void showMessageDialog(Component parentComponent, Object message)`
 - **parentComponent** est le parent de la boîte de dialogue. Si le paramètre est null, la boîte de dialogue sera centrée à l'écran.
 - **message**: le message à afficher sur la boîte de dialogue.
- Exemple:

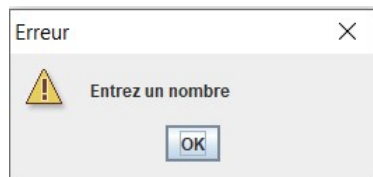
```
String messageErreur = " Entrez un prix valide. Exemple 12.35";
JOptionPane.showMessageDialog(null, messageErreur);
// la fenêtre s'affiche au milieu de l'écran.
```



- Si on veut que la boîte de dialogue s'affiche au milieu d'une fenêtre, on utilise:

```
JOptionPane.showMessageDialog(nomFenetre.this, messageErreur);
```
- La méthode `showMessageDialog` est surchargée et offre des paramètres supplémentaires qui permettent de personnaliser la boîte de dialogue.
- Exemple:

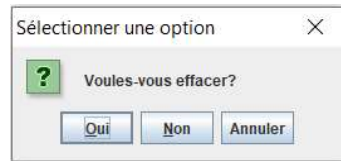
```
JOptionPane.showMessageDialog(FrmCalculer4.this, "Entrez un nombre", "Erreur" ,
JOptionPane.WARNING_MESSAGE);
```



Les boîtes de dialogue: la boîte de confirmation

- static **int** **showConfirmDialog**(**Component** parentComponent, **Object** message),
- La méthode **showConfirmDialog** retourne un entier qui indique sur quel bouton on a cliqué.
- Exemple: CalculerListener.java

```
int choix = JOptionPane.showConfirmDialog(null, "Voulez-vous effacer?");
//on récupère l'entier correspondant au bouton cliqué(yes, no, cancel, X)
if (choix == JOptionPane.YES_OPTION) {
    txtPrix.setText("");
    txtPrixReduit.setText("");
    txtPrix.requestFocus();
}
```



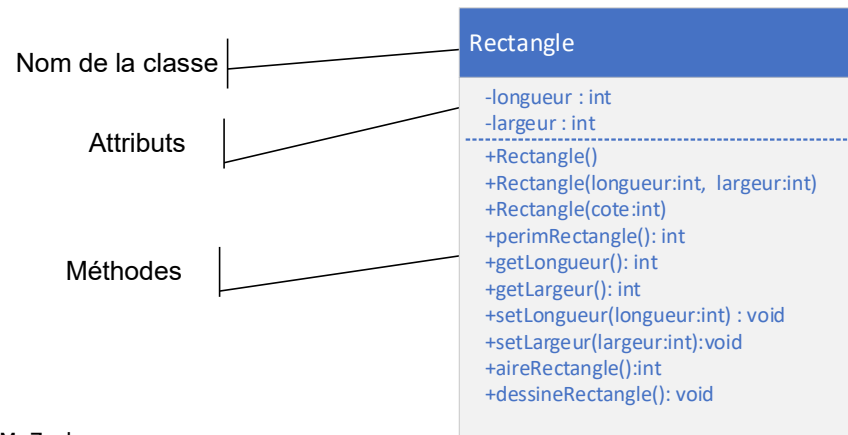
- Une fois la fenêtre de confirmation fermée, la méthode **showConfirmDialog** renvoie une valeur entière représentant le choix de l'utilisateur.
 - **JOptionPane.YES_OPTION**: le bouton **Oui** a été cliqué
 - **JOptionPane.NO_OPTION**: le bouton **Non** a été cliqué
 - **JOptionPane.CANCEL_OPTION**: le bouton **Annuler** a été cliqué
 - **JOptionPane.CLOSED_OPTION**: le bouton de fermeture (X) a été cliqué
-



La relation de composition

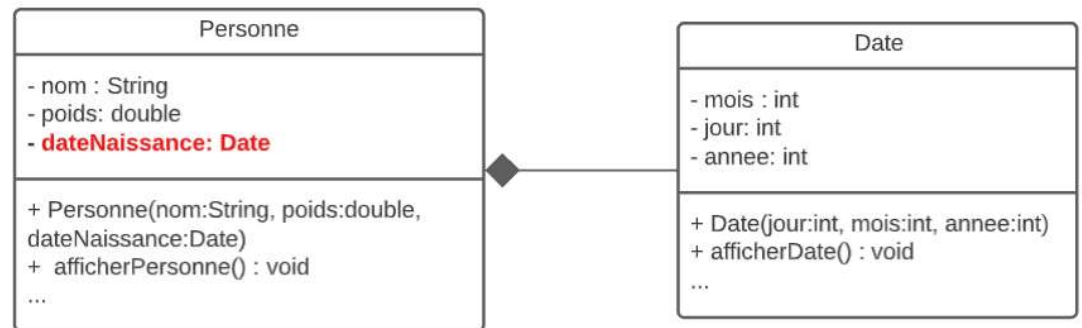
Rappel: modélisation d'une classe

- Voici comment une classe peut être représentée en suivant les conventions de la modélisation objet **UML** (Unified Modeling Language).
- **On spécifie le nom de la classe, les attributs et les méthodes** dans un rectangle comme ici-bas.
- La visibilité des membres (attributs et méthodes) est représentée par les différents symboles suivants :
 - **private ou -** : le membre est accessible uniquement dans la classe en question.
 - **public ou +** : le membre est accessible dans toutes les autres classes du programme.
 - **protected ou #** : le membre est accessible uniquement dans les classes dérivées de la classe en question ainsi qu'aux classes se trouvant dans le même package.
 - **package ou ~ ou rien** : le membre est accessible uniquement dans les classes du même paquetage.



La relation de composition

- La composition représente une forme de réutilisation de logiciel qui permet à une classe de posséder comme attributs des références à des objets d'autres classes.
 - C'est une relation «a un» ou «est composé de» qui signifie qu'un objet peut contenir un ou plusieurs objets d'autres classes.
 - Exemples de relations de composition:
 - Dans le tp1, la classe `Compte` a un champ d'instance de la classe `Client`
 - Dans l'exemple de la semaine 3, la classe `Personne` a un champ d'instance de type `Date`
 - La composition est indiquée par un lien débutant par un losange.
- Exemple: Une Personne a une Date



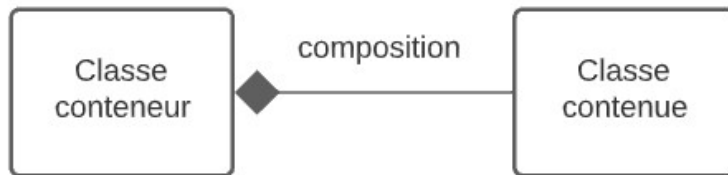
```
public class Personne {
    private String nom;
    private int poids;
    private Date dateNaissance;

    public Personne(String nom, int poids, dateNaissance Date) { ... }
    ...
}

public class Date {
    private int jour;
    private int mois;
    private int annee;
    public Date(int jour, int mois, int annee) {...}
    ...
}
```


composition et agrégation

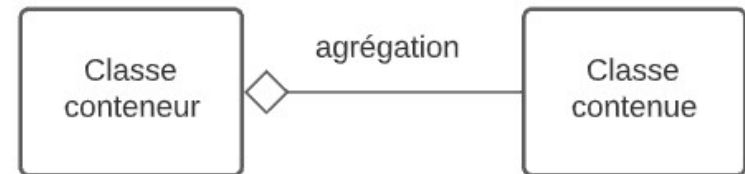
- Le **losange plein** représente la **composition** d'une classe. La classe contenue ne peut pas exister sans la classe conteneur,



- Exemple: La classe `Train` est composée de la classe `Siege`. La classe `Siege` ne peut exister sans la classe `Train`.



- Le **losange vide** représente l'**agrégation** d'une classe. La classe contenue peut exister sans la classe conteneur.

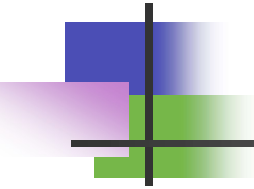


- Exemple: La classe `Bibliotheque` est composée de la classe `Livre`. La classe `Livre` peut exister sans la classe `Bibliotheque`.





La relation d'héritage



L'héritage

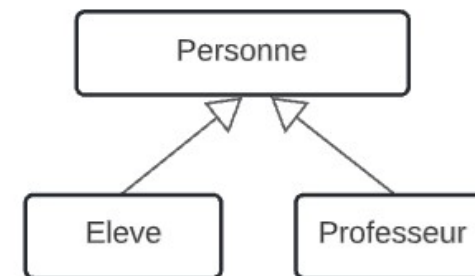
- L'héritage est une autre forme de réutilisation de logiciel qui consiste à créer de nouvelles classes à partir d'une autre classe déjà existante et ainsi récupérer les attributs et les méthodes, sans avoir à la réécrire complètement.
- Lorsqu'une classe hérite d'une autre classe, elle hérite (utilise) ses attributs et ses méthodes et on peut lui ajouter de nouveaux attributs et de nouvelles méthodes pour adapter la classe à la nouvelle situation (la spécialiser).
- Terminologie :
 - La classe définie à partir d'une autre classe est appelée la **classe fille** ou **sous-classe**. Celle qui sert à définir la classe fille est appelée la **classe mère** ou **super-classe**.
 - On dit que la classe fille **dérive** (ou **hérite** de ou **étend** la classe mère).

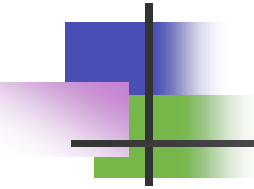
La relation d'héritage

Exemples de relations d'héritage

SuperClasse	Sous-classe	Sous-classe
Personne	Eleve Professeur	
Forme	Quadrilatere Cercle Triangle	Rectangle Carre Losange
Prêt	PretAutomobile PretHypothcaire PretPresonnel	
Vehicule	Moto Velo Voiture Bateau	

- La relation d'héritage est une relation «**est un**» : chaque objet d'une sous classe est un objet de la super classe. Par exemple, un `Eleve` est une `Personne`.
- L'héritage est aussi une relation de **spécialisation**. un `Eleve` est une `Personne` particulière.
- L'ensemble de tous les objets de sous-classe (comme `Eleve`) est un sous ensemble de tous les objets de la super-classe (comme `Personne`).
- L'héritage est indiqué par une flèche au bout vide dirigée de la sous-classe vers la superclasse. La superclasse est au-dessus de la sous-classe.





L'héritage en Java

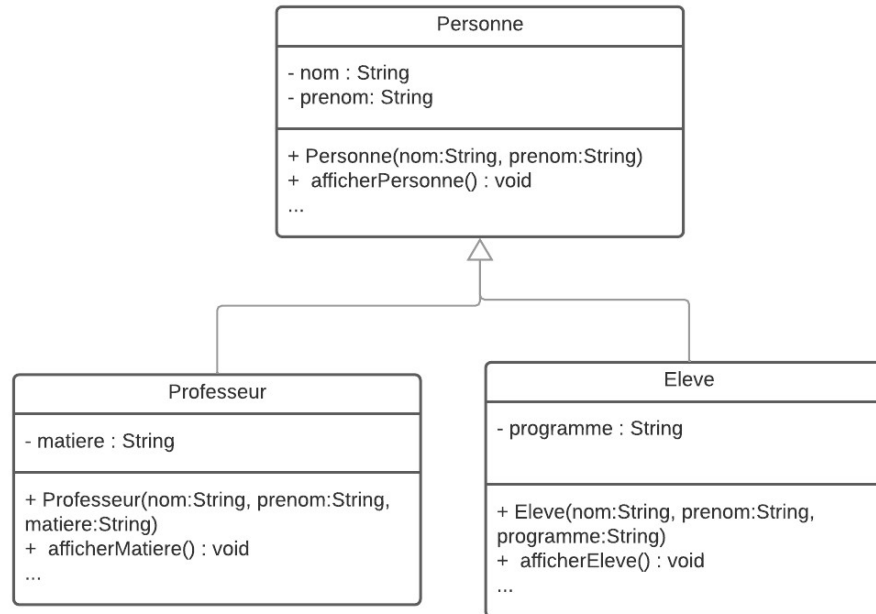
- On utilise le mot clé **extends** pour faire hériter une classe d'une superclasse.

```
Public class NomSousClasse extends NomSuperClasse {  
    /* Déclaration des attributs et méthodes spécifiques à la  
       sous-classe */  
}
```

Exemple :

```
public class Professeur extends Personne {  
    ...  
}
```

La relation d'héritage: exemple (1/3)



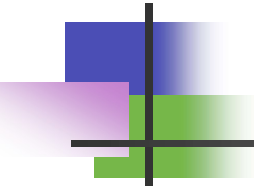
La relation d'héritage: exemple (2/3)

```
12 public Personne(String prenom, String nom) {
13     this.prenom = prenom;
14     this.nom = nom;
15 }
16
17 public String getPrenom() {}
20
21 public void setPrenom(String prenom) {}
24
25 public String getNom() {}
28
29 public void setNom(String nom) {}
32
33 public void afficherPersonne() {
34     System.out.println("Prenom:" + getPrenom());
35     System.out.println("Nom:" + getNom());
36 }
37
38 public void afficher() {
39     System.out.println("Renseignements de la personne.");
40     afficherPersonne();
41 }
42 }
```

```
11 public class Professeur extends Personne {
12     private String matiere;
13
14 public Professeur(String prenom, String nom, String matiere) {
15     // Appel du constructeur de la super classe Personne
16     super(prenom, nom);
17     this.matiere = matiere;
18 }
19
20 public String getMatiere() {}
23
24 public void setMatiere(String matieres) {}
27
28 public void afficherProfesseur() {
29     System.out.println("Renseignement de la personne.");
30     // Appel de la méthode afficherPersonne() de la superclasse Personne
31     super.afficherPersonne();
32     System.out.println("Matière:" + getMatiere());
33 }
34 }
```

La relation d'héritage: exemple (3/3)

```
10 public class Eleve extends Personne {
11     private String programme;
12
13     public Eleve(String prenom, String nom, String programme) {
14         super(prenom, nom);
15         this.programme = programme;
16     }
17
18     public String getProgramme() {
19
20     }
21
22     public void setProgramme(String programme) {
23
24     }
25
26     public void afficherEleve() {
27         System.out.println("Renseignements de l'élève:");
28         // Appel de la méthode afficherPersonne() de la superclasse Personne
29         super.afficherPersonne();
30         System.out.println("Programme: " + getProgramme());
31     }
32
33 }
```

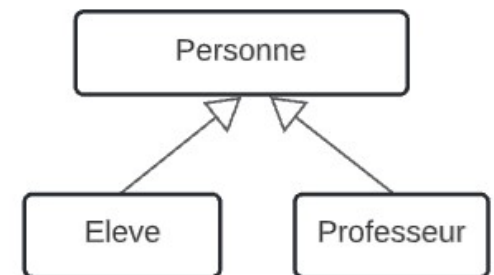



L'organisation des classes et l'héritage

- Quand on développe en orienté objet, on commence par concevoir les **classes les plus générales** que l'on **spécialise** au fur et à mesure en dérivant des **classes plus spécialisées**.
 - Exemple : on commence par concevoir la classe Personne (générale) ensuite les classes Professeur et Eleve (spécialisées)
- Il faut organiser les classes de façon à ce que les propriétés et méthodes communes à plusieurs classes soient placées dans une superclasse.
- Une classe parent peut avoir plusieurs classes enfants.
- Cependant, une classe ne peut pas hériter de plusieurs classes en Java. L'héritage multiple en java n'est pas possible.

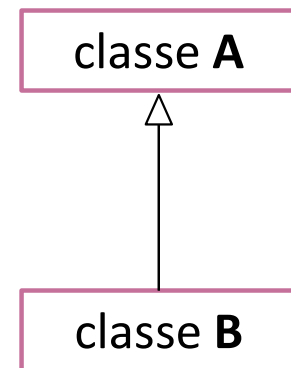
Particularisation-généralisation

- Particularisation-généralisation :
 - Un Professeur **est une** Personne, mais c'est une personne particulière.
 - Un Eleve est une Personne, mais c'est une personne particulière.
- On peut voir une Personne comme étant la **généralisation** de Eleve et de Professeur
- **Une classe enfant** (comme Eleve) **offre de nouveaux services** (méthodes). Elle peut redéfinir **les services rendus par la superclasse** (Personne).



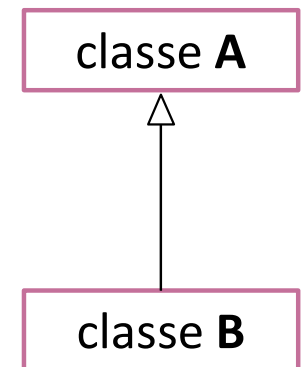
Vocabulaire

- On dit de la classe A:
 - La classe A est la superclasse de la classe B.
 - La classe A est la classe mère de la classe B.
 - La classe A est la classe parent de la classe B.
- On dit pour la classe B:
 - La classe B hérite de la classe A.
 - La classe B dérive de la classe A.
 - La classe B étend (extends) la classe A.
 - La classe B est une classe enfant de la classe A.
 - La classe B est une sous-classe de la classe A.



Réutilisation dans l'héritage

- Le type est hérité : un objet de la classe B est aussi un objet de classe A. La classe B hérite des attributs et des méthodes de la classe A, sauf les constructeurs.
- Qu'est ce qui change entre A et B ?
 - Le code source de B ne comporte que ce qui a changé par rapport au code de A.
 - On n'a pas besoin de recopier tout le code qui est dans A pour le mettre dans B.
- On peut, par exemple, faire les modifications suivantes:
 - Ajouter de nouvelles méthodes dans B .
 - Ajouter de nouveaux attributs dans B.
 - Modifier certaines méthodes de A qui se retrouvent ainsi dans B : d'autres fonctionnalités



Spécialisation

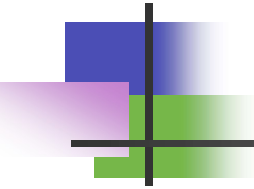
Enrichissement : redéfinition

Le mot-clé super

```
public class Personne {  
    private String prenom;  
    private String nom;  
  
    public Personne(String prenom, String nom) {  
        this.prenom = prenom;  
        this.nom = nom;  
    }  
    ..  
}
```

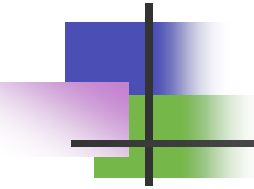
```
public class Professeur extends Personne {  
    private String matiere;  
  
    public Professeur(String prenom, String nom, String matiere) {  
        // Appel du constructeur de la super classe Personne  
        super(prenom, nom);  
        this.matiere = matiere;  
    }  
    ...  
}
```

Attention: si on omet d'appeler le constructeur de la superclasse (dans la première ligne), le constructeur par défaut est appelé implicitement. Donc, il devrait être défini dans la superclasse sinon on aura un message d'erreur.



Le mot-clé **super**

- Le mot-clé ***super*** est utilisé pour désigner la **superclasse**.
- ***super*** permet d'accéder aux membres de la superclasse: ***super.nomMethode()*** de la même manière que l'on accède aux membres de la classe elle-même (comme le mot-clé ***this***).
- Le mot clé ***super*** indique que la recherche de la méthode doit commencer dans la superclasse plutôt que dans la classe courante.
- ***super()*** permet de faire appel au constructeur de la superclasse. Il permet de profiter du constructeur de la superclasse et évite la duplication de code.



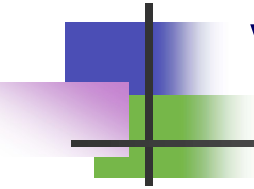
Héritage et constructeur

- **Les constructeurs de la superclasse ne sont jamais hérités dans la sous-classe**, c'est pourquoi il faut les appeler à l'aide du mot-clé ***super***.
- La syntaxe pour appeler ce constructeur est ***super()*** ou ***super(parametres)***.
- Cette instruction doit être **la première ligne** dans la définition du constructeur de la sous-classe.
- **Si *super()* n'est pas explicitement utilisé , alors le constructeur par défaut (constructeur sans paramètre) de la superclasse est appelé.**

Exemple 1

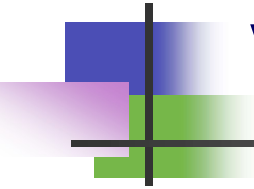
```
public class Forme {
    private int posX;
    private int posY;
    //constructeur par défaut
    public Forme() {
        posX = 0; posY = 0;}
    public Forme(int x, int y) {
        setPosX(x);
        setPosY(y);
    }
    public void setPosY(int y) { posY=y; }
    public void setPosX(int x) { posX=x; }
    public int getPosX() { return posX; }
    public int getPosY() { return posY; }
}
```

```
public class Rectangle extends Forme {
    private double largeur = 3.0;
    private double hauteur = 4.0;
    /*le constructeur par défaut fait appel au
    constructeur par défaut de la classe Forme. On
    a pu omettre le mot clé super car il le fera
    implicitement(pas conseillé)*/
    public Rectangle() {
        super();
        setHauteur(0);
        setLargeur(0);
    }
    /*le constructeur avec paramètres de la classe
    Rectangle fait appel au constructeur avec
    paramètres de la superclasse Forme en utilisant
    super(x,y)*/
    public Rectangle(double h, double l, int x, int
    y) {
        super(x,y);
        setHauteur(h);
        setLargeur(l);
    }
    public double getLargeur() {return largeur;}
    public double getHauteur() {return hauteur;}
    public void setLargeur(double l) {largeur=l;}
    public void setHauteur(double h) {hauteur=h;}
}
```

Visibilité des membres

- Rappel : l'accès aux membres (attributs et méthodes) d'une classe pouvait être :
 - **public** : visibilité totale à l'intérieur et à l'extérieur de la classe (mot-clé **public**)
 - **privé** : visibilité uniquement à l'intérieur de la classe (mot-clé **private**)
 - **par défaut** (aucun modificateur) : visibilité depuis toutes les classes du même paquetage (est aussi valable pour le paquetage par défaut)



Visibilité dans le cas de l'héritage

- Il n'est pas possible d'utiliser un attribut ou méthode privés d'une superclasse dans la classe héritée.
- Et on ne peut pas les rendre public pour pouvoir les utiliser car c'est contre le principe de l'encapsulation.
- **Solution : Modificateur protected**
- Un attribut **protected** ou une méthode **protected** dans une classe **public**, peut être accessible dans *n'importe quelle classe du même package ou dans ses sous-classes*, même si les sous-classes sont dans des packages différents.
- Comparé au modificateur **public**, **protected** permet une meilleure encapsulation mais n'est pas aussi strict que **private**.

Exemple 2 : Modificateur protected

```
public class Forme {  
    protected int posX;  
    protected int posY;  
  
    public Forme(int x, int y) {  
        setPosX(x);  
        setPosY(y);  
    }  
    //setter et getter de posX et posY  
}
```

```
public class Rectangle extends Forme {  
    private double largeur = 3.0;  
    private double hauteur = 4.0;  
    //....  
    public Rectangle(double h, double l, int x, int y) {  
        super(x,y);  
        setHauteur(h);  
        setLargeur(l);  
    }  
    //on ajoute la méthode coindDS à Rectangle  
    public void coindDS() {  
        System.out.println("Le coin sup droit est :  
        " + (posX + largeur) + ", " + posY);  
    }  
    //setter et getter de largeur et hauteur  
}
```

Exemple 3

On a enrichi la classe Forme et la classe Rectangle
On a rajouté les classes Cercle et Cylindre

```
public class Forme {
    protected int posX=0;
    protected int posY=0;
    protected static int nbObjets = 0;

    public Forme(int x, int y){
        setPosX(x);
        setPosY(y);
        nbObjets++;
    }
    public void deplaceDe( int dx, int dy){
        setPosX(posX+dx); setPosY(posY+dy);
    }
    public static int getNbObjets() {
        return nbObjets;
    }
    //les getters et les setters ...
}
```

```
public class Rectangle extends Forme {
    private double largeur = 3.0;
    private double hauteur = 4.0;
    public Rectangle(double h, double l, int x, int y) {
        super(x,y);
        setHauteur(h);
        setLargeur(l);
    }
    public void coinDS() {
        System.out.println("Le coin sup droit est : " +
            (posX + largeur) + ", " + posY);
    }
    public double surface() {
        return largeur*hauteur;
    }
    public double perimetre() {
        return (largeur+hauteur)*2;
    }
    public double getLargeur() {return largeur;}
    public double getHauteur() {return hauteur;}
    public void setLargeur(double l) {largeur=l;}
    public void setHauteur(double h) {hauteur=h;}
}
```

Spécialisation :
rajout de méthodes
coinDS(), surface()
et perimetre().

```

public class Cercle extends Forme {
    protected int rayon;
    public Cercle(int x, int y, int r){
        super(x, y);
        setRayon(r);
    }

    public void setRayon(int r)
    {
        rayon = r;
    }
    public double getRayon() {
        return rayon;
    }

    double perimetre(){
        return 2*Math.PI*rayon;
    }

    double surface(){
        return Math.PI*rayon*rayon;
    }
}

```

```

public class Cylindre extends Cercle {
    private double longueur;

    public Cylindre(int x, int y, int rayon, double l)
    {
        super(x,y,rayon); /* appel au constructeur
                           de la super-classe */
        setLongueur(l);
    }

    public double getLongueur(){
        return longueur;
    }

    public void setLongueur(double longueur){
        this.longueur=longueur;
    }

    public double volume()
    {
        return super.surface() * longueur;
    }
}

```

```

public class TestAppelMethode {
public static void main(String arg[]) {
Rectangle rect1 = new Rectangle(5, 3, 2, 5);
Cercle cercle = new Cercle(4, 6, 12);
Cylindre monCylindre = new Cylindre(3, 5, 2, 10);
rect1.deplaceDe(1, 1);
cercle.deplaceDe(1, 1);
monCylindre.deplaceDe(1, 1);
System.out.println("**** Les positions x et y après le déplacement **** ");
System.out.println("rectangle: x=" + rect1.getPosX() + "    y=" + rect1.getPosY());
System.out.println("cercle: x=" + cercle.getPosX() + "    y=" + cercle.getPosY());
System.out.println("cylindre: x=" + monCylindre.getPosX() + "    y=" +
monCylindre.getPosY());
rect1.coinDS();
System.out.println("Le périmètre du cercle est=" + cercle.perimetre());
System.out.println("Le volume du cylindre est=" + monCylindre.vOLUME());
}

```

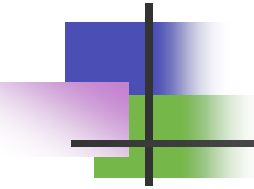
Appel aux méthodes héritées:
deplaceDe(int, int),
getPosX (),
getPosY()

Appel aux méthodes ajoutées:
perimetre(),
volume(),
coinDS()

```

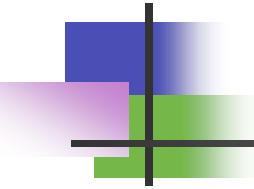
**** les positions x et y après le déplacement ****
rectangle: x=3.0    y=6.0
cercle: x=5.0    y=7.0
cylindre: x=4.0    y=6.0
Le coin sup droit est : 3.0,6.03.0
le perimetre du cercle est=75.39822368615503
le volume du cylindre est=125.66370614359172

```



Redéfinition et surcharge de méthodes

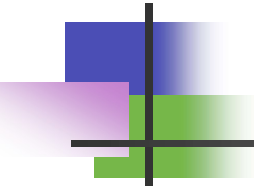
- **La classe enfant peut :**
 - **Redéfinir** des méthodes de la classe parent (même signature).
 - **Surcharger** des méthodes de la même classe (même nom mais pas la même signature).



Méthodes redéfinies

- Une classe enfant peut **redéfinir** une méthode héritée en faveur de sa propre implémentation.
- La nouvelle méthode **doit** avoir **la même signature** que celle du parent mais elle peut avoir un **code différent.**
- Si la méthode à redéfinir de la superclasse est **public**, la méthode de redéfinition de la sous classe **doit** être aussi **public**.
- **Le type d'objet** qui exécute la méthode détermine la **version** de la méthode qui est invoquée, celle du parent ou de l'enfant.

Redéfinir = Override



Redéfinition, le mot clé super

- Une méthode de la classe parent peut être invoquée **explicitement** en utilisant la référence **super**.
- Pour appeler une méthode de la super-classe :

La syntaxe est la suivante :

super.nomMéthode(paramètres);

Exemple 4:

Dans la classe **Cylindre**, on **redéfinit la méthode `surface()`**, car la surface d'un cylindre se calcule différemment de la surface d'un cercle.

```
class Cercle extends Forme {
    protected double rayon;
    Cercle(double x, double y, double r) {
        super(x, y);
        setRayon (r);
    }
    public void setRayon(double r)
    {
        rayon = r;
    }
    public double getRayon() {
        return rayon;
    }

    public double perimetre(){
        return 2*Math.PI*rayon;
    }
    public double surface(){
        return Math.PI*rayon*rayon;
    }
}
```

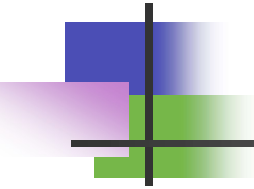
Redéfinition

```
public class Cylindre extends Cercle {
    private double longueur; //rajout

    public Cylindre(int x, int y, int rayon, double l)
    {
        super(x,y, rayon);
        setLongueur(l);
    }

    public double getLongueur(){
        return longueur;
    }
    public double surface() { //redéfinition
        return 2 * super.surface() + super.perimetre()*
        longueur;
    }
    public double volume() //rajout
    {
        return super.surface() * longueur;
    }
}
```

Appel de la méthode surface de la superclasse Cercle en utilisant le mot clé super



Remarque

- Si, dans une super-classe et une sous-classe, il existe une méthode, par exemple la méthode **surface()**, ayant la même signature (même nom, même liste de paramètres) et qui retourne le même type de valeur, et si **objet1** est une instance de la super-classe et **objet2**, une instance de la sous-classe, alors l'instruction suivante **objet2. surface()** va appeler la méthode déclarée dans la sous-classe.
- On dit que la méthode **objet2. surface()** redéfinit la méthode **objet1. surface()**.

```

public class TestCylindre {
public static void main(String args[])
{
    Cylindre monCylindre = new Cylindre(3,5,2,10);

    System.out.println("La longueur est " +
monCylindre.getLongueur());

    System.out.println("Le rayon est " +
monCylindre.getRayon());

    System.out.println("Le volume du cylindre est " +
monCylindre.calculerVolume());

    System.out.println("La surface du cercle est " +
monCylindre.surface());

    System.out.println("Le nombre de cylindres est " +
Cylindre.getNbObjets());
}
}

```

La longueur est 10.0

Le rayon est 2.0

Le volume du cylindre est 125.66370614359172

La surface du cercle est 150.79644737231007

Le nombre de cylindres est 1

C'est la méthode *surface()* de la classe
Cylindre qui est appelée

C'est la méthode *getNbObjets()* de la
classe **Forme** qui est appelée



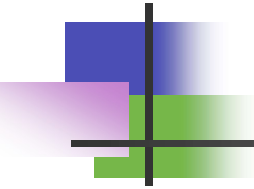
L'annotation @override

- L'annotation `@Override` indique au compilateur qu'une méthode donnée est la redéfinition d'une méthode de la superclasse.
- En plus de rappeler au programmeur qu'il s'agit de la redéfinition d'une méthode de la superclasse, la présence de l'annotation `@Override` va forcer le compilateur à vérifier que la méthode a été redéfinie correctement.

- Exemple dans la classe Cylindre :

`@Override`

```
public double surface() {  
    return 2 * super.surface() + (2 * Math.PI * getRayon()) * longueur;  
}
```

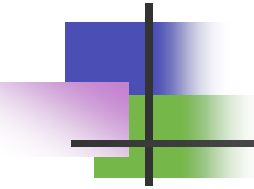


Surcharge vs. Redéfinition

- La **surcharge** concerne plusieurs méthodes avec le même nom dans la **même** classe mais avec **différentes signatures**.
 - La méthode *System.out.println()* est un bon exemple de la surcharge. On peut l'appeler avec différents types et nombres de paramètres.
 - La surcharge de méthodes permet de définir plusieurs constructeurs pour une même classe.
- La **redéfinition** concerne deux méthodes, l'une dans la classe parent et l'autre dans la classe enfant, ayant la même signature.

La surcharge permet de définir une opération de **manières différentes** pour différents paramètres.

La redéfinition permet de définir une **opération similaire de différentes manières** pour différents objets



Exemple de surcharge

```
public class Math {  
    //...  
  
    public static double carre(double x) {  
        return x*x;  
    }  
    public static int carre(int x) {  
        return x*x;  
    }  
  
    //...  
}
```

Le modificateur final

- Le concepteur de classe peut imposer que la classe parent ne soit pas utilisable dans un héritage. Dans ce cas, on utilisera le mot clé final dans la déclaration de la classe.

```
public final class Cylindre extends Cercle {
```

```
...
```

```
}
```

```
public class Cone extends Cylindre {
```

```
...
```

```
}
```

The type Cone can't subclass the final class Cylindre

- On peut aussi définir une méthode comme étant **final**; une méthode finale ne peut pas être redéfinie par ses sous-classes.

```
public class Forme {
```

```
    protected int posX=0;
```

```
    protected int posY=0;
```

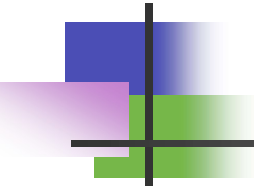
```
    public final void deplaceDe( int dx, int dy) {
```

```
        setPosX(posX+dx); setPosY(posY+dy);
```

```
    }
```

```
}
```

Les classes Rectangle, Cercle et Cylindre qui héritent de la classe Forme ne peuvent pas redéfinir la méthode deplaceDe(...)



Sources

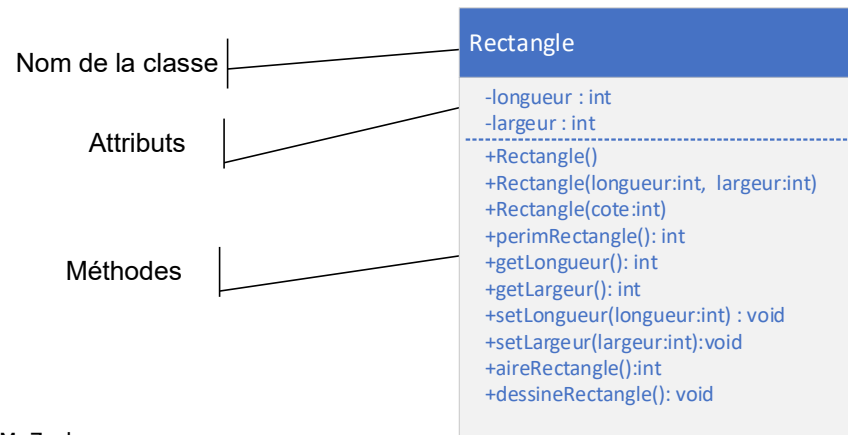
- Kypriano, S. L'héritage
- Tasso, A. (s.d.). Le livre de JAVA premier langage avec 109 exercices corrigés. Éditions EYROLLES.



La relation de composition

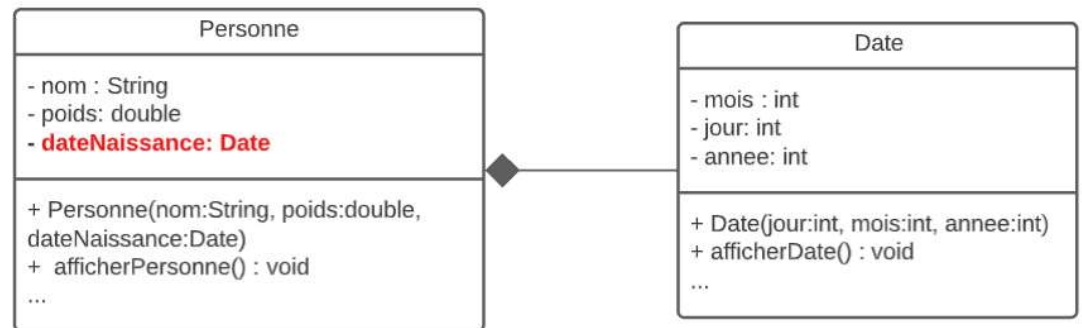
Rappel: modélisation d'une classe

- Voici comment une classe peut être représentée en suivant les conventions de la modélisation objet **UML** (Unified Modeling Language).
- **On spécifie le nom de la classe, les attributs et les méthodes** dans un rectangle comme ici-bas.
- La visibilité des membres (attributs et méthodes) est représentée par les différents symboles suivants :
 - **private ou -** : le membre est accessible uniquement dans la classe en question.
 - **public ou +** : le membre est accessible dans toutes les autres classes du programme.
 - **protected ou #** : le membre est accessible uniquement dans les classes dérivées de la classe en question ainsi qu'aux classes se trouvant dans le même package.
 - **package ou ~ ou rien** : le membre est accessible uniquement dans les classes du même paquetage.



La relation de composition

- La composition représente une forme de réutilisation de logiciel qui permet à une classe de posséder comme attributs des références à des objets d'autres classes.
 - C'est une relation «a un» ou «est composé de» qui signifie qu'un objet peut contenir un ou plusieurs objets d'autres classes.
 - Exemples de relations de composition:
 - Dans le tp1, la classe `Compte` a un champ d'instance de la classe `Client`
 - Dans l'exemple de la semaine 3, la classe `Personne` a un champ d'instance de type `Date`
 - La composition est indiquée par un lien débutant par un losange.
- Exemple: Une Personne a une Date



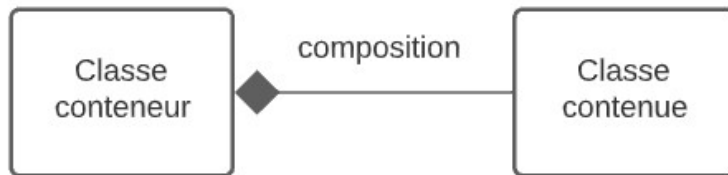
```
public class Personne {
    private String nom;
    private int poids;
    private Date dateNaissance;

    public Personne(String nom, int poids, dateNaissance Date) { ... }
    ...
}

public class Date {
    private int jour;
    private int mois;
    private int annee;
    public Date(int jour, int mois, int annee) {...}
    ...
}
```

composition et agrégation

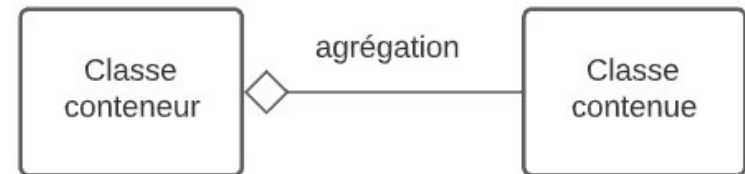
- Le **losange plein** représente la **composition** d'une classe. La classe contenue ne peut pas exister sans la classe conteneur,



- Exemple: La classe `Train` est composée de la classe `Siege`. La classe `Siege` ne peut exister sans la classe `Train`.



- Le **losange vide** représente l'**agrégation** d'une classe. La classe contenue peut exister sans la classe conteneur.



- Exemple: La classe `Bibliotheque` est composée de la classe `Livre`. La classe `Livre` peut exister sans la classe `Bibliotheque`.





La relation d'héritage



L'héritage

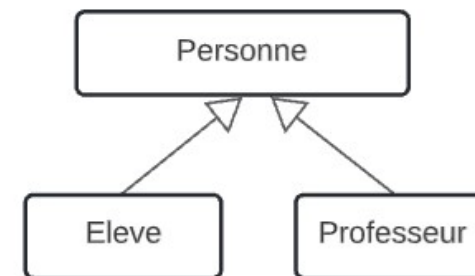
- L'héritage est une autre forme de réutilisation de logiciel qui consiste à créer de nouvelles classes à partir d'une autre classe déjà existante et ainsi récupérer les attributs et les méthodes, sans avoir à la réécrire complètement.
- Lorsqu'une classe hérite d'une autre classe, elle hérite (utilise) ses attributs et ses méthodes et on peut lui ajouter de nouveaux attributs et de nouvelles méthodes pour adapter la classe à la nouvelle situation (la spécialiser).
- Terminologie :
 - La classe définie à partir d'une autre classe est appelée la **classe fille** ou **sous-classe**. Celle qui sert à définir la classe fille est appelée la **classe mère** ou **super-classe**.
 - On dit que la classe fille **dérive** (ou **hérite** de ou **étend** la classe mère).

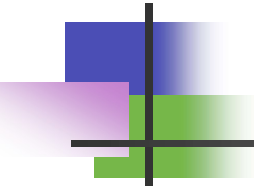
La relation d'héritage

■ Exemples de relations d'héritage

SuperClasse	Sous-classe	Sous-classe
Personne	Eleve Professeur	
Forme	Quadrilatere Cercle Triangle	Rectangle Carre Losange
Prêt	PretAutomobile PretHypothcaire PretPresonnel	
Vehicule	Moto Velo Voiture Bateau	

- La relation d'héritage est une relation «**est un**» : chaque objet d'une sous classe est un objet de la super classe. Par exemple, un `Eleve` est une `Personne`.
- L'héritage est aussi une relation de **spécialisation**. un `Eleve` est une `Personne` particulière.
- L'ensemble de tous les objets de sous-classe (comme `Eleve`) est un sous ensemble de tous les objets de la super-classe (comme `Personne`).
- L'héritage est indiqué par une flèche au bout vide dirigée de la sous-classe vers la superclasse. La superclasse est au-dessus de la sous-classe.





L'héritage en Java

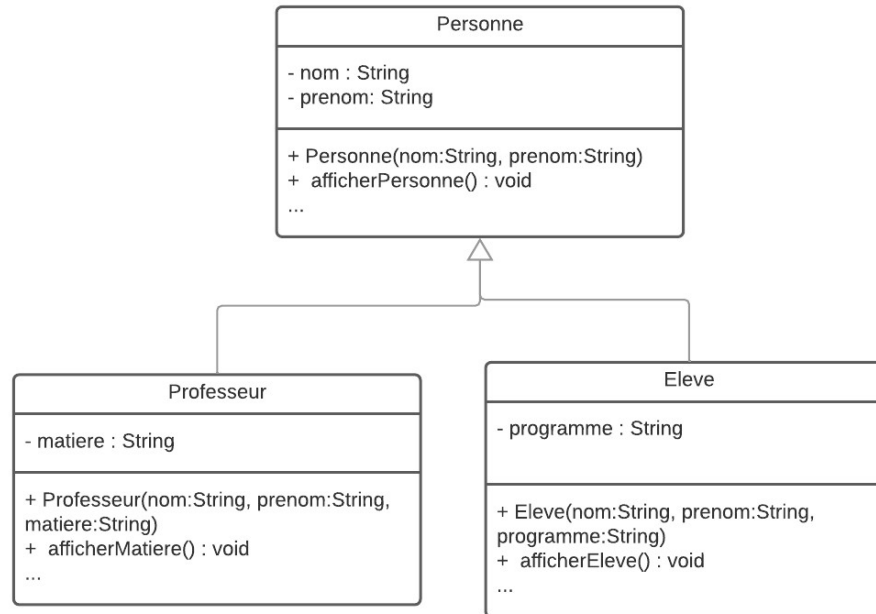
- On utilise le mot clé **extends** pour faire hériter une classe d'une superclasse.

```
Public class NomSousClasse extends NomSuperClasse {  
    /* Déclaration des attributs et méthodes spécifiques à la  
       sous-classe */  
}
```

Exemple :

```
public class Professeur extends Personne {  
    ...  
}
```

La relation d'héritage: exemple (1/3)



La relation d'héritage: exemple (2/3)

```
12 public Personne(String prenom, String nom) {
13     this.prenom = prenom;
14     this.nom = nom;
15 }
16
17 public String getPrenom() {}
20
21 public void setPrenom(String prenom) {}
24
25 public String getNom() {}
28
29 public void setNom(String nom) {}
32
33 public void afficherPersonne() {
34     System.out.println("Prenom:" + getPrenom());
35     System.out.println("Nom:" + getNom());
36 }
37
38 public void afficher() {
39     System.out.println("Renseignements de la personne.");
40     afficherPersonne();
41 }
42 }
```

```
11 public class Professeur extends Personne {
12     private String matiere;
13
14 public Professeur(String prenom, String nom, String matiere) {
15     // Appel du constructeur de la super classe Personne
16     super(prenom, nom);
17     this.matiere = matiere;
18 }
19
20 public String getMatiere() {}
23
24 public void setMatiere(String matieres) {}
27
28 public void afficherProfesseur() {
29     System.out.println("Renseignement de la personne.");
30     // Appel de la méthode afficherPersonne() de la superclasse Personne
31     super.afficherPersonne();
32     System.out.println("Matière:" + getMatiere());
33 }
34 }
```

La relation d'héritage: exemple (3/3)

```
10 public class Eleve extends Personne {
11     private String programme;
12
13     public Eleve(String prenom, String nom, String programme) {
14         super(prenom, nom);
15         this.programme = programme;
16     }
17
18     public String getProgramme() {
19
20     }
21
22     public void setProgramme(String programme) {
23
24     }
25
26     public void afficherEleve() {
27         System.out.println("Renseignements de l'élève:");
28         // Appel de la méthode afficherPersonne() de la superclasse Personne
29         super.afficherPersonne();
30         System.out.println("Programme: " + getProgramme());
31     }
32
33 }
```

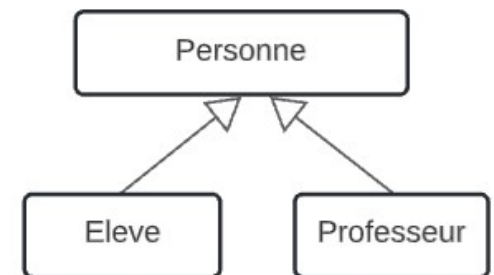


L'organisation des classes et l'héritage

- Quand on développe en orienté objet, on commence par concevoir les **classes les plus générales** que l'on **spécialise** au fur et à mesure en dérivant des **classes plus spécialisées**.
 - Exemple : on commence par concevoir la classe Personne (générale) ensuite les classes Professeur et Eleve (spécialisées)
- Il faut organiser les classes de façon à ce que les propriétés et méthodes communes à plusieurs classes soient placées dans une superclasse.
- Une classe parent peut avoir plusieurs classes enfants.
- Cependant, une classe ne peut pas hériter de plusieurs classes en Java. L'héritage multiple en java n'est pas possible.

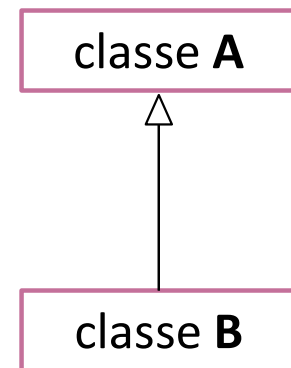
Particularisation-généralisation

- Particularisation-généralisation :
 - Un Professeur **est une** Personne, mais c'est une personne particulière.
 - Un Eleve est une Personne, mais c'est une personne particulière.
- On peut voir une Personne comme étant la **généralisation** de Eleve et de Professeur
- **Une classe enfant** (comme Eleve) **offre de nouveaux services** (méthodes). Elle peut redéfinir **les services rendus par la superclasse** (Personne).



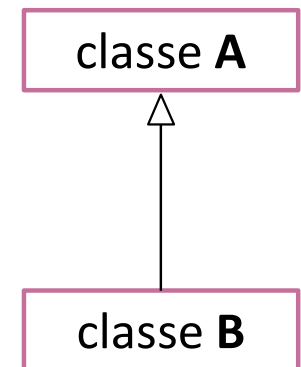
Vocabulaire

- On dit de la classe A:
 - La classe A est la superclasse de la classe B.
 - La classe A est la classe mère de la classe B.
 - La classe A est la classe parent de la classe B.
- On dit pour la classe B:
 - La classe B hérite de la classe A.
 - La classe B dérive de la classe A.
 - La classe B étend (extends) la classe A.
 - La classe B est une classe enfant de la classe A.
 - La classe B est une sous-classe de la classe A.



Réutilisation dans l'héritage

- Le type est hérité : un objet de la classe B est aussi un objet de classe A. La classe B hérite des attributs et des méthodes de la classe A, sauf les constructeurs.
- Qu'est ce qui change entre A et B ?
 - Le code source de B ne comporte que ce qui a changé par rapport au code de A.
 - On n'a pas besoin de recopier tout le code qui est dans A pour le mettre dans B.
- On peut, par exemple, faire les modifications suivantes:
 - Ajouter de nouvelles méthodes dans B .
 - Ajouter de nouveaux attributs dans B.
 - Modifier certaines méthodes de A qui se retrouvent ainsi dans B : d'autres fonctionnalités



Spécialisation

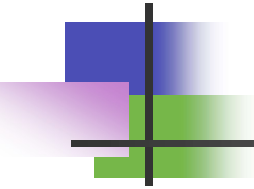
Enrichissement : redéfinition

Le mot-clé super

```
public class Personne {  
    private String prenom;  
    private String nom;  
  
    public Personne(String prenom, String nom) {  
        this.prenom = prenom;  
        this.nom = nom;  
    }  
    ..  
}
```

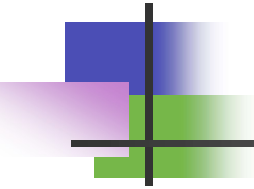
```
public class Professeur extends Personne {  
    private String matiere;  
  
    public Professeur(String prenom, String nom, String matiere) {  
        // Appel du constructeur de la super classe Personne  
        super(prenom, nom);  
        this.matiere = matiere;  
    }  
    ...  
}
```

Attention: si on omet d'appeler le constructeur de la superclasse (dans la première ligne), le constructeur par défaut est appelé implicitement. Donc, il devrait être défini dans la superclasse sinon on aura un message d'erreur.



Le mot-clé **super**

- Le mot-clé ***super*** est utilisé pour désigner la **superclasse**.
- ***super*** permet d'accéder aux membres de la superclasse: ***super.nomMethode()*** de la même manière que l'on accède aux membres de la classe elle-même (comme le mot-clé ***this***).
- Le mot clé ***super*** indique que la recherche de la méthode doit commencer dans la superclasse plutôt que dans la classe courante.
- ***super()*** permet de faire appel au constructeur de la superclasse. Il permet de profiter du constructeur de la superclasse et évite la duplication de code.



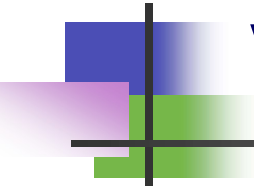
Héritage et constructeur

- **Les constructeurs de la superclasse ne sont jamais hérités dans la sous-classe**, c'est pourquoi il faut les appeler à l'aide du mot-clé ***super***.
- La syntaxe pour appeler ce constructeur est ***super()*** ou ***super(parametres)***.
- Cette instruction doit être **la première ligne** dans la définition du constructeur de la sous-classe.
- **Si *super()* n'est pas explicitement utilisé , alors le constructeur par défaut (constructeur sans paramètre) de la superclasse est appelé.**

Exemple 1

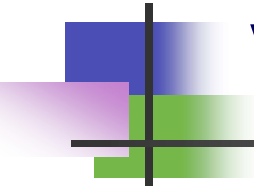
```
public class Forme {
    private int posX;
    private int posY;
    //constructeur par défaut
    public Forme() {
        posX = 0; posY = 0;}
    public Forme(int x, int y) {
        setPosX(x);
        setPosY(y);
    }
    public void setPosY(int y) { posY=y; }
    public void setPosX(int x) { posX=x; }
    public int getPosX() { return posX; }
    public int getPosY() { return posY; }
}
```

```
public class Rectangle extends Forme {
    private double largeur = 3.0;
    private double hauteur = 4.0;
    /*le constructeur par défaut fait appel au
    constructeur par défaut de la classe Forme. On
    a pu omettre le mot clé super car il le fera
    implicitement(pas conseillé)*/
    public Rectangle() {
        super();
        setHauteur(0);
        setLargeur(0);
    }
    /*le constructeur avec paramètres de la classe
    Rectangle fait appel au constructeur avec
    paramètres de la superclasse Forme en utilisant
    super(x,y)*/
    public Rectangle(double h, double l, int x, int
    y) {
        super(x,y);
        setHauteur (h);
        setLargeur(l);
    }
    public double getLargeur() {return largeur;}
    public double getHauteur() {return hauteur;}
    public void setLargeur(double l) {largeur=l;}
    public void setHauteur(double h) {hauteur=h;}
}
```



Visibilité des membres

- Rappel : l'accès aux membres (attributs et méthodes) d'une classe pouvait être :
 - **public** : visibilité totale à l'intérieur et à l'extérieur de la classe (mot-clé **public**)
 - **privé** : visibilité uniquement à l'intérieur de la classe (mot-clé **private**)
 - **par défaut** (aucun modificateur) : visibilité depuis toutes les classes du même paquetage (est aussi valable pour le paquetage par défaut)



Visibilité dans le cas de l'héritage

- Il n'est pas possible d'utiliser un attribut ou méthode privés d'une superclasse dans la classe héritée.
- Et on ne peut pas les rendre public pour pouvoir les utiliser car c'est contre le principe de l'encapsulation.
- **Solution : Modificateur protected**
- Un attribut **protected** ou une méthode **protected** dans une classe **public**, peut être accessible dans *n'importe quelle classe du même package ou dans ses sous-classes*, même si les sous-classes sont dans des packages différents.
- Comparé au modificateur **public**, **protected** permet une meilleure encapsulation mais n'est pas aussi strict que **private**.

Exemple 2 : Modificateur protected

```
public class Forme {  
    protected int posX;  
    protected int posY;  
  
    public Forme(int x, int y) {  
        setPosX(x);  
        setPosY(y);  
    }  
    //setter et getter de posX et posY  
}
```

```
public class Rectangle extends Forme {  
    private double largeur = 3.0;  
    private double hauteur = 4.0;  
    //....  
    public Rectangle(double h, double l, int x, int y) {  
        super(x,y);  
        setHauteur(h);  
        setLargeur(l);  
    }  
    //on ajoute la méthode coindDS à Rectangle  
    public void coindDS() {  
        System.out.println("Le coin sup droit est :  
        " + (posX + largeur) + ", " + posY);  
    }  
    //setter et getter de largeur et hauteur  
}
```

Exemple 3

On a enrichi la classe Forme et la classe Rectangle
On a rajouté les classes Cercle et Cylindre

```
public class Forme {
    protected int posX=0;
    protected int posY=0;
    protected static int nbObjets = 0;

    public Forme(int x, int y){
        setPosX(x);
        setPosY(y);
        nbObjets++;
    }
    public void deplaceDe( int dx, int dy){
        setPosX(posX+dx); setPosY(posY+dy);
    }
    public static int getNbObjets() {
        return nbObjets;
    }
    //les getters et les setters ...
}
```

```
public class Rectangle extends Forme {
    private double largeur = 3.0;
    private double hauteur = 4.0;
    public Rectangle(double h, double l, int x, int y) {
        super(x,y);
        setHauteur(h);
        setLargeur(l);
    }
    public void coinDS() {
        System.out.println("Le coin sup droit est : " +
            (posX + largeur) + ", " + posY);
    }
    public double surface() {
        return largeur*hauteur;
    }
    public double perimetre() {
        return (largeur+hauteur)*2;
    }
    public double getLargeur() {return largeur;}
    public double getHauteur() {return hauteur;}
    public void setLargeur(double l) {largeur=l;}
    public void setHauteur(double h) {hauteur=h;}
}
```

Spécialisation :
rajout de méthodes
coinDS(), surface()
et perimetre().


```

public class Cercle extends Forme {
    protected int rayon;
    public Cercle(int x, int y, int r){
        super(x, y);
        setRayon(r);
    }

    public void setRayon(int r)
    {
        rayon = r;
    }
    public double getRayon() {
        return rayon;
    }

    double perimetre(){
        return 2*Math.PI*rayon;
    }

    double surface(){
        return Math.PI*rayon*rayon;
    }
}

```

```

public class Cylindre extends Cercle {
    private double longueur;

    public Cylindre(int x, int y, int rayon, double l)
    {
        super(x,y,rayon); /* appel au constructeur
                           de la super-classe */
        setLongueur(l);
    }

    public double getLongueur(){
        return longueur;
    }

    public void setLongueur(double longueur){
        this.longueur=longueur;
    }

    public double volume()
    {
        return super.surface() * longueur;
    }
}

```

```

public class TestAppelMethode {
public static void main(String arg[]) {
Rectangle rect1 = new Rectangle(5, 3, 2, 5);
Cercle cercle = new Cercle(4, 6, 12);
Cylindre monCylindre = new Cylindre(3, 5, 2, 10);
rect1.deplaceDe(1, 1);
cercle.deplaceDe(1, 1);
monCylindre.deplaceDe(1, 1);
System.out.println("**** Les positions x et y après le déplacement **** ");
System.out.println("rectangle: x=" + rect1.getPosX() + "    y=" + rect1.getPosY());
System.out.println("cercle: x=" + cercle.getPosX() + "    y=" + cercle.getPosY());
System.out.println("cylindre: x=" + monCylindre.getPosX() + "    y=" +
monCylindre.getPosY());
rect1.coinDS();
System.out.println("Le périmètre du cercle est=" + cercle.perimetre());
System.out.println("Le volume du cylindre est=" + monCylindre.vOLUME());
}

```

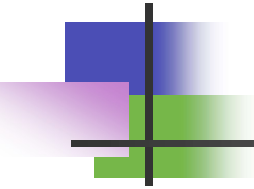
Appel aux méthodes héritées:
deplaceDe(int, int),
getPosX (),
getPosY()

Appel aux méthodes ajoutées:
perimetre(),
volume(),
coinDS()

```

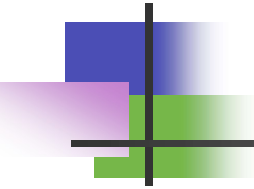
**** les positions x et y après le déplacement ****
rectangle: x=3.0    y=6.0
cercle: x=5.0    y=7.0
cylindre: x=4.0    y=6.0
Le coin sup droit est : 3.0,6.03.0
le perimetre du cercle est=75.39822368615503
le volume du cylindre est=125.66370614359172

```



Redéfinition et surcharge de méthodes

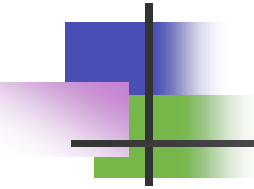
- **La classe enfant peut :**
 - **Redéfinir** des méthodes de la classe parent (même signature).
 - **Surcharger** des méthodes de la même classe (même nom mais pas la même signature).



Méthodes redéfinies

- Une classe enfant peut **redéfinir** une méthode héritée en faveur de sa propre implémentation.
- La nouvelle méthode **doit** avoir **la même signature** que celle du parent mais elle peut avoir un **code différent.**
- Si la méthode à redéfinir de la superclasse est **public**, la méthode de redéfinition de la sous classe **doit** être aussi **public**.
- **Le type d'objet** qui exécute la méthode détermine la **version** de la méthode qui est invoquée, celle du parent ou de l'enfant.

Redéfinir = Override



Redéfinition, le mot clé super

- Une méthode de la classe parent peut être invoquée **explicitement** en utilisant la référence **super**.
- Pour appeler une méthode de la super-classe :

La syntaxe est la suivante :

super.nomMéthode(paramètres);

Exemple 4:

Dans la classe **Cylindre**, on **redéfinit la méthode *surface()***, car la surface d'un cylindre se calcule différemment de la surface d'un cercle.

```
class Cercle extends Forme {
    protected double rayon;
    Cercle(double x, double y, double r) {
        super(x, y);
        setRayon (r);
    }
    public void setRayon(double r)
    {
        rayon = r;
    }
    public double getRayon() {
        return rayon;
    }

    public double perimetre(){
        return 2*Math.PI*rayon;
    }
    public double surface(){
        return Math.PI*rayon*rayon;
    }
}
```

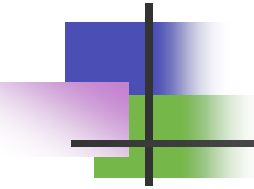
Redéfinition

```
public class Cylindre extends Cercle {
    private double longueur; //rajout

    public Cylindre(int x, int y, int rayon, double l)
    {
        super(x,y, rayon);
        setLongueur(l);
    }

    public double getLongueur(){
        return longueur;
    }
    public double surface() { //redéfinition
        return 2 * super.surface() + super.perimetre()*
        longueur;
    }
    public double volume() //rajout
    {
        return super.surface() * longueur;
    }
}
```

Appel de la méthode surface de la superclasse Cercle en utilisant le mot clé super



Remarque

- Si, dans une super-classe et une sous-classe, il existe une méthode, par exemple la méthode **surface()**, ayant la même signature (même nom, même liste de paramètres) et qui retourne le même type de valeur, et si **objet1** est une instance de la super-classe et **objet2**, une instance de la sous-classe, alors l'instruction suivante **objet2. surface()** va appeler la méthode déclarée dans la sous-classe.
- On dit que la méthode **objet2. surface()** redéfinit la méthode **objet1. surface()**.

```

public class TestCylindre {
public static void main(String args[])
{
    Cylindre monCylindre = new Cylindre(3,5,2,10);

    System.out.println("La longueur est " +
monCylindre.getLongueur());

    System.out.println("Le rayon est " +
monCylindre.getRayon());

    System.out.println("Le volume du cylindre est " +
monCylindre.calculerVolume());

    System.out.println("La surface du cercle est " +
monCylindre.surface());

    System.out.println("Le nombre de cylindres est " +
Cylindre.getNbObjets());
}
}

```

La longueur est 10.0

Le rayon est 2.0

Le volume du cylindre est 125.66370614359172

La surface du cercle est 150.79644737231007

Le nombre de cylindres est 1

C'est la méthode *surface()* de la classe
Cylindre qui est appelée

C'est la méthode *getNbObjets()* de la
classe **Forme** qui est appelée



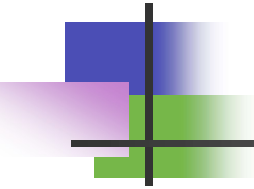
L'annotation @override

- L'annotation `@Override` indique au compilateur qu'une méthode donnée est la redéfinition d'une méthode de la superclasse.
- En plus de rappeler au programmeur qu'il s'agit de la redéfinition d'une méthode de la superclasse, la présence de l'annotation `@Override` va forcer le compilateur à vérifier que la méthode a été redéfinie correctement.

- Exemple dans la classe Cylindre :

`@Override`

```
public double surface() {  
    return 2 * super.surface() + (2 * Math.PI * getRayon()) * longueur;  
}
```

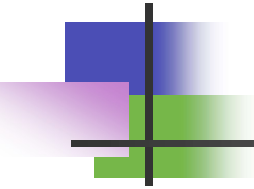


Surcharge vs. Redéfinition

- La **surcharge** concerne plusieurs méthodes avec le même nom dans la **même** classe mais avec **différentes signatures**.
 - La méthode *System.out.println()* est un bon exemple de la surcharge. On peut l'appeler avec différents types et nombres de paramètres.
 - La surcharge de méthodes permet de définir plusieurs constructeurs pour une même classe.
- La **redéfinition** concerne deux méthodes, l'une dans la classe parent et l'autre dans la classe enfant, ayant la même signature.

La surcharge permet de définir une opération de **manières différentes** pour différents paramètres.

La redéfinition permet de définir une **opération similaire de différentes manières** pour différents objets



Exemple de surcharge

```
public class Math {  
    //...  
  
    public static double carre(double x) {  
        return x*x;  
    }  
    public static int carre(int x) {  
        return x*x;  
    }  
  
    //...  
}
```

Le modificateur final

- Le concepteur de classe peut imposer que la classe parent ne soit pas utilisable dans un héritage. Dans ce cas, on utilisera le mot clé final dans la déclaration de la classe.

```
public final class Cylindre extends Cercle {
```

```
...
```

```
}
```

```
public class Cone extends Cylindre {
```

```
...
```

```
}
```

The type Cone can't subclass the final class Cylindre

- On peut aussi définir une méthode comme étant **final**; une méthode finale ne peut pas être redéfinie par ses sous-classes.

```
public class Forme {
```

```
    protected int posX=0;
```

```
    protected int posY=0;
```

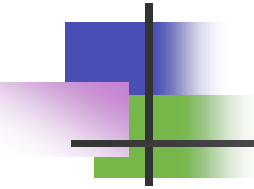
```
    public final void deplaceDe( int dx, int dy) {
```

```
        setPosX(posX+dx); setPosY(posY+dy);
```

```
    }
```

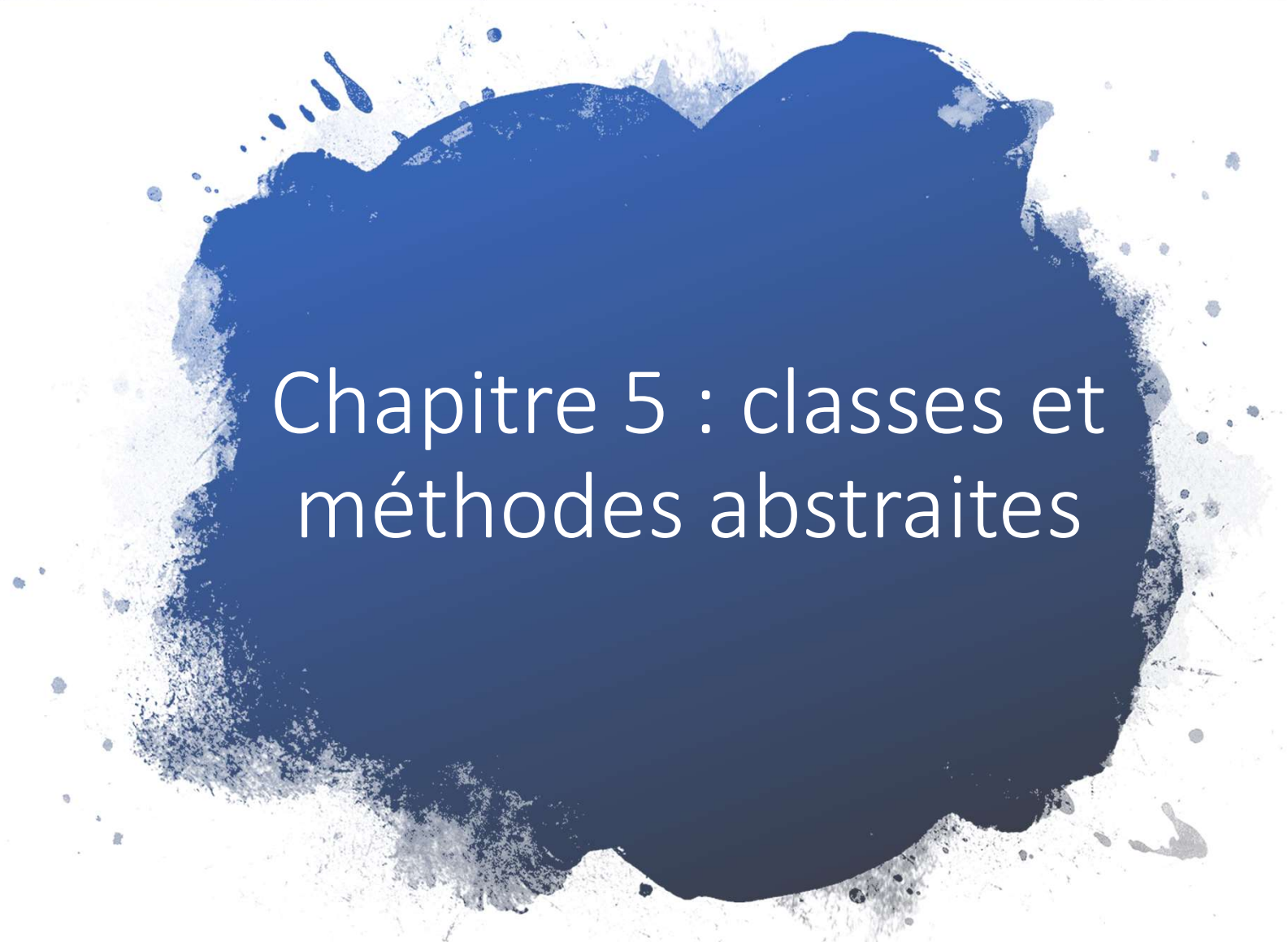
```
}
```

Les classes Rectangle, Cercle et Cylindre qui héritent de la classe Forme ne peuvent pas redéfinir la méthode deplaceDe(...)



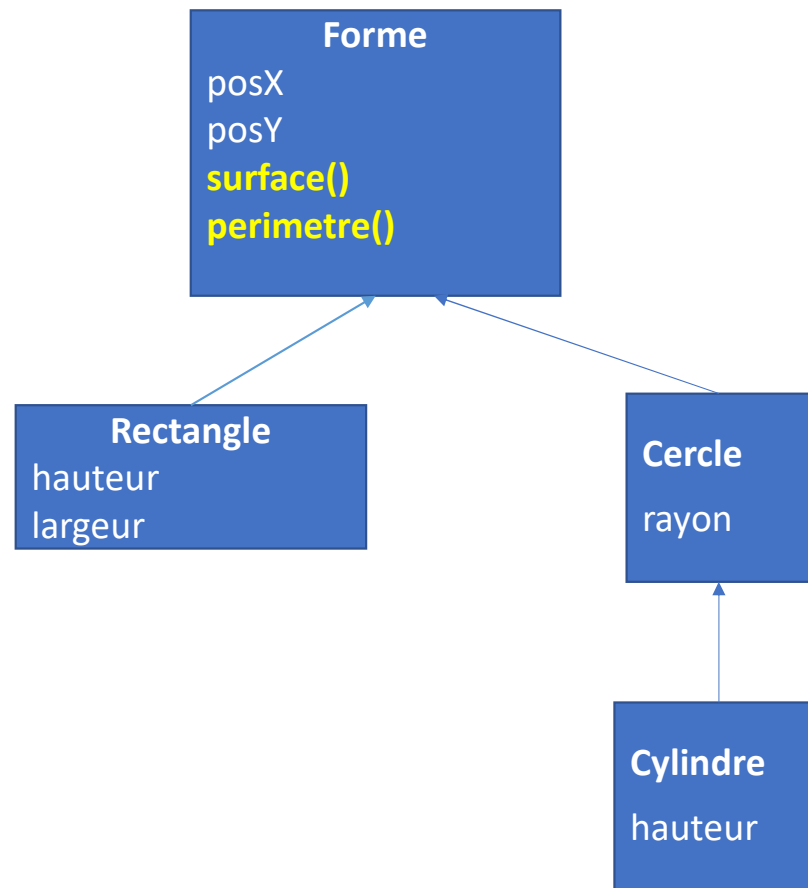
Sources

- Kypriano, S. L'héritage
- Tasso, A. (s.d.). Le livre de JAVA premier langage avec 109 exercices corrigés. Éditions EYROLLES.



Chapitre 5 : classes et méthodes abstraites

Exemple

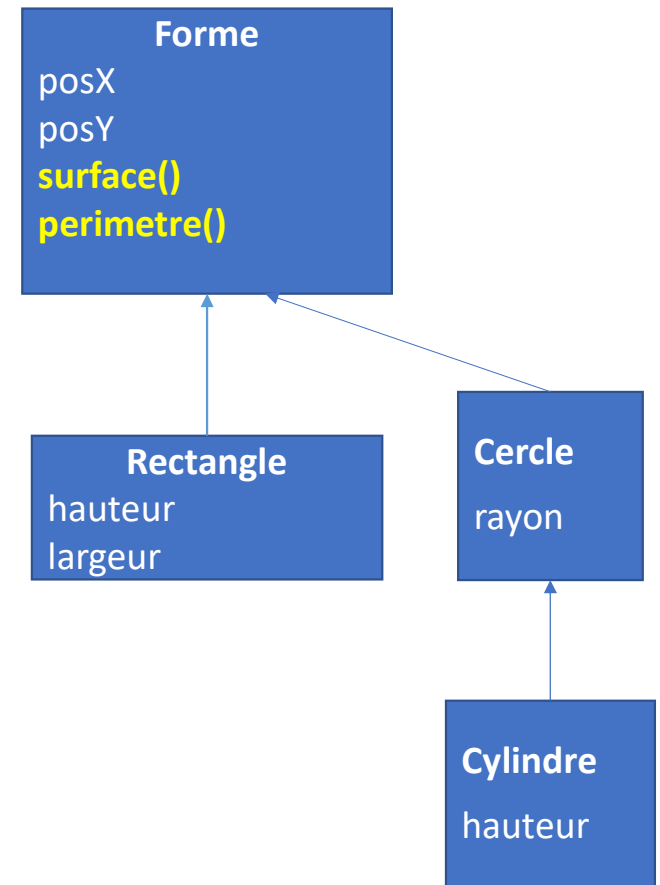


Problématique

Avec l'**héritage**, il est possible de **généraliser** des concepts qui sont présents dans plusieurs classes du programme en les incorporant à une super-classe.

On construit, alors, une structure arborescente reliant différents éléments à un niveau plus abstrait.

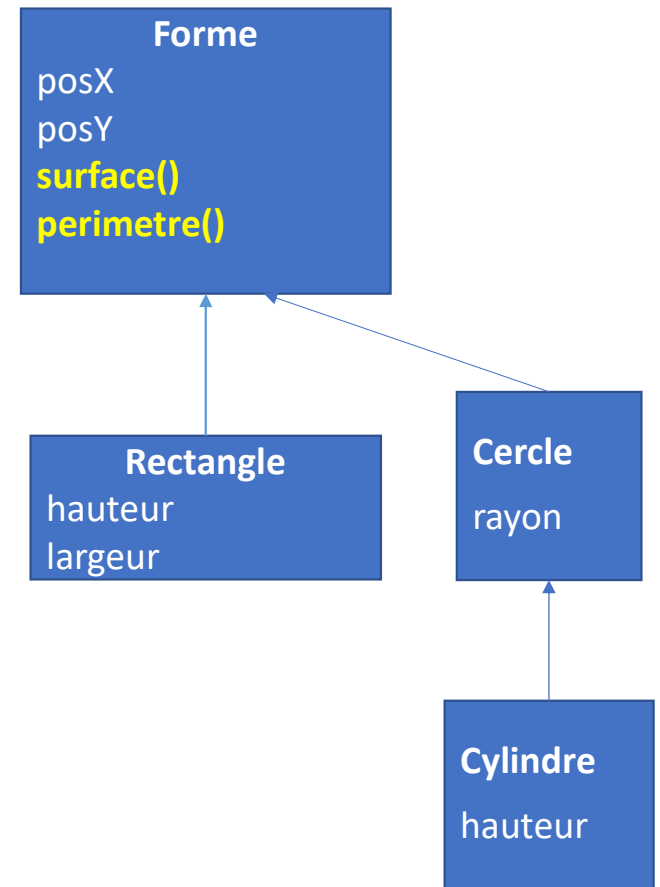
➡ fait partie des mécanismes d'**abstraction** de l'orienté objet.



Problématique

Mais, au niveau le plus élevé d'une hiérarchie de classes, il est parfois impossible de définir une méthode générale qui devra pourtant exister dans toutes les sous-classes.

Exemple : *calculer la surface* d'une Forme quelconque se révèle difficile, tandis qu'à un niveau plus bas on peut le mettre en œuvre.



Exemple :

```
public class Forme {
    protected double posX=0;
    protected double posY=0;
    protected static int nbObjets = 0;

    public Forme(){
        setPosX(0); setPosY(0);
        nbObjets++;
    }
    public Forme(double x, double y){
        setPosX(x); setPosY(y);
        nbObjets++;
    }
    public void deplaceDe(double dx, double dy){
        setPosX(posX+dx); setPosY(posY+dy);
    }
    public double surface() {return 0.0;}
    public double perimetre() {return 0.0;}
    public void afficher() {}

    // et les getters et les setters ...
}
```

```
public class Rectangle extends Forme {
    private double largeur = 3.0;
    private double hauteur = 4.0;
    public Rectangle() {
        super();
        setHauteur(0);
        setLargeur(0);
    }
    public Rectangle(double h, double l, double x, double y) {
        super(x,y);
        setHauteur(h);
        setLargeur(l);
    }
    public void coinDS() {
        System.out.println("Le coin sup droit est : " + (posX
        + largeur) + ", " + posY);
    }
    public double surface() {
        return largeur*hauteur;
    }
    public double perimetre() {
        return (largeur+hauteur)*2;
    }
    public void afficher() {
        System.out.println("[Rectangle] largeur = " + largeur
        + " et hauteur = " + hauteur);
    }
    // et les getters et setters
}
```

```

public class Cercle extends Forme {
    protected double rayon;
    public Cercle(double x, double y, double r){
        super(x, y);
        setRayon(r);
    }

    public void setRayon(double r)
    {
        rayon = r;
    }
    public double getRayon() {
        return rayon;
    }

    double perimetre(){
        return 2*Math.PI*rayon;
    }

    double surface(){
        return Math.PI*rayon*rayon;
    }
    public void afficher() {
        System.out.println("[Cercle] rayon = "
+ rayon);
    }
    // et les getters et setters
}

```

```

public class Cylindre extends Cercle {
    private double longueur;

    public Cylindre(double x, double y, double rayon, double l)
    {
        super(x,y,rayon);
        setLongueur(l);
    }

    public double surface() {
        return 2 * super.surface() + (2 * Math.PI * getRayon())
* Longueur;
    }

    public double volume()
    {
        return super.surface() * longueur;
    }
    public void afficher() {
        System.out.println("[Cylindre] rayon = " + rayon + " et
Longueur = " + longueur);
    }

    // et les getters et setters
}

```

Problématique

- Une telle méthode **surface()** au niveau de la super-classe **Forme** pourrait être nécessaire pour comparer deux surfaces (qui est générale) ou simplement par souci d'abstraction et d'unification des données.
- On veut comparer la surface de deux formes , mais on ne pourra savoir si c'est un Cercle ou Rectangle qu'en cours d'exécution.
- Le programme ne compilera pas si la méthode **surface** n'est pas déclarée au niveau de la classe **Forme**.

```
public class TestPolymorphisme {
    public static void main(String[] args)
    {
        // déclaration et initialisation de deux Formes
        Forme forme1 = new Cercle(5, 3, 6);
        Forme forme2 = new Rectangle(5, 3, 2, 5);

        System.out.println("Les 2 Formes ont La même surface ? " +
            surfaceCompare(forme1, forme2));

        // afficher le cercle
        afficherForme(forme1);

        // afficher le rectangle
        afficherForme(forme2);
    }

    // méthode pour comparer les surfaces de deux Formes
    static boolean surfaceCompare(Forme objet1, Forme objet2)
    {
        return objet1.surface() == objet2.surface();
    }

    // méthode pour afficher des infos sur la forme géométrique
    static void afficherForme(Forme objet)
    {
        System.out.println();
        objet.afficher();
        System.out.println("La surface est " + objet.surface());
        System.out.println("Le périmètre est " +
            objet.perimetre());
    }
}
```

Solution: Méthode abstraite

- C'est une méthode qui dispose de la **signature** (type de retour et paramètres) mais **sans implémentation**.
- Elle est déclarée avec le modificateur **abstract**

Exemple:

```
public abstract double surface();
```

- La méthode sera implémentée par les classes enfants.



Définition : classes Abstraites

Une classe abstraite est une classe **incomplète**. Elle regroupe un ensemble de variables et de méthodes mais **certaines de ses méthodes ne contiennent pas d'instructions (méthodes abstraites)**

- Elle est déclarée avec le modificateur **abstract**

Exemple:

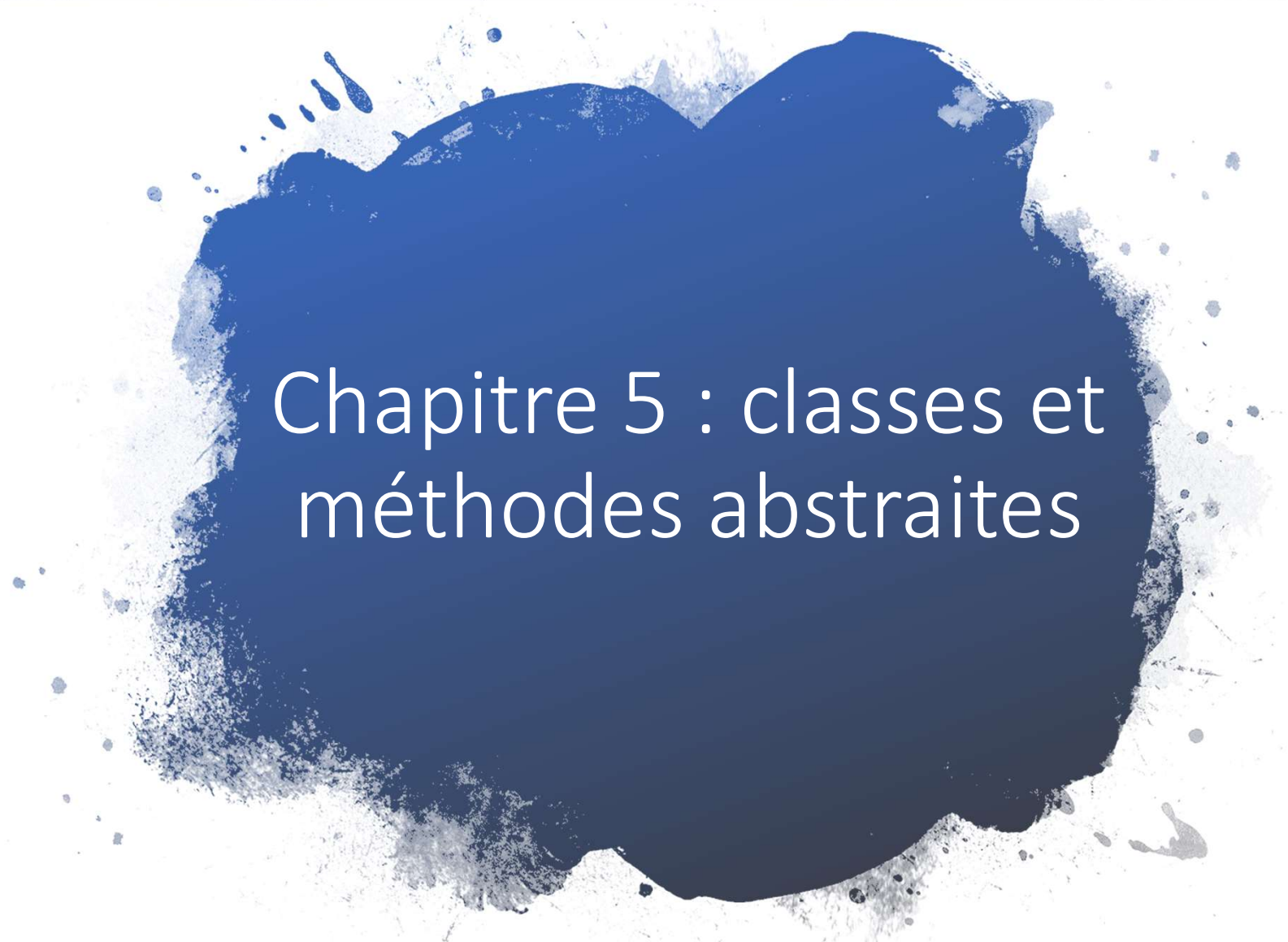
```
public abstract class Forme
```

Classes Abstraites

- **Règle 1** : Une classe est automatiquement abstraite **si une de ses méthodes est abstraite**.
- **Règle 2** : Une classe abstraite **peut contenir des méthodes non abstraites et des déclarations de variables ordinaires**.
- **Règle 3** : On déclare qu'une classe est abstraite avec le mot clé **abstract**.
- **Règle 4** : Une classe abstraite **n'est pas instanciable** (on ne peut pas utiliser les constructeurs d'une classe abstraite et donc on ne peut pas créer d'objet de cette classe.)
- **Règle 5** : Une classe qui hérite d'une classe abstraite ne devient concrète que si elle **implémente toutes les méthodes abstraites** de la classe dont elle hérite.

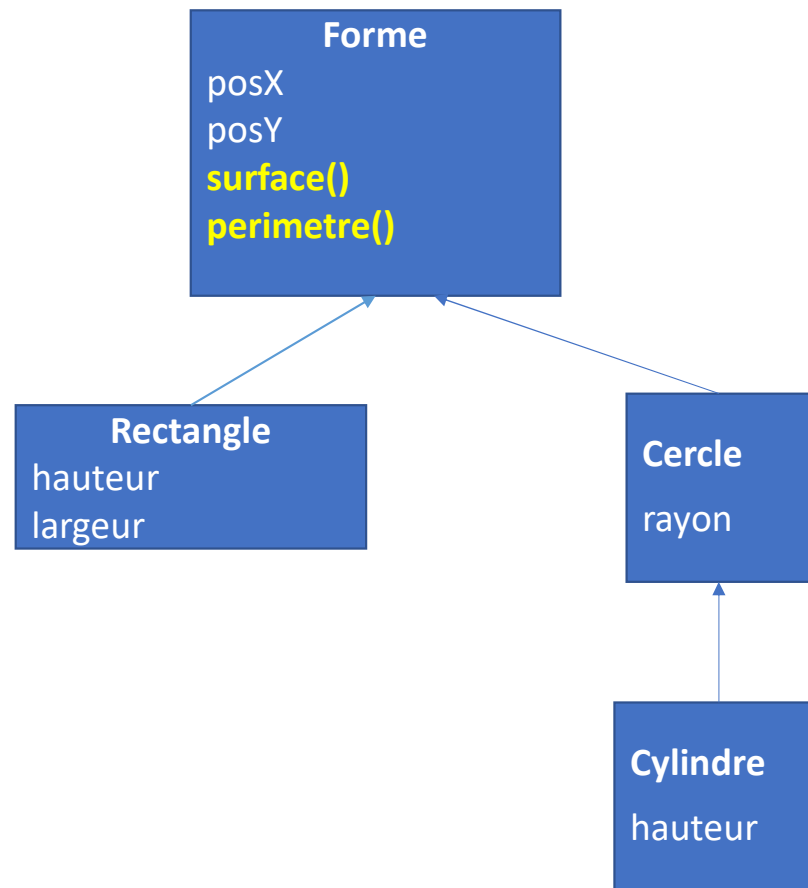
Exemple

```
public abstract class Forme {  
    protected double posX=0;  
    protected double posY=0;  
    protected static int nbObjets = 0;  
  
    public Forme(){  
        setPosX(0); setPosY(0); nbObjets++;  
    }  
    public Forme(double x, double y){  
        setPosX(x); setPosY(y); nbObjets++;  
    }  
    public void deplaceDe(double dx, double dy){  
        setPosX(posX+dx); setPosY(posY+dy);  
    }  
    public abstract double surface();  
    public abstract double perimetre();  
    public abstract void afficher();  
  
    // et les getters et les setters ...  
}
```



Chapitre 5 : classes et méthodes abstraites

Exemple

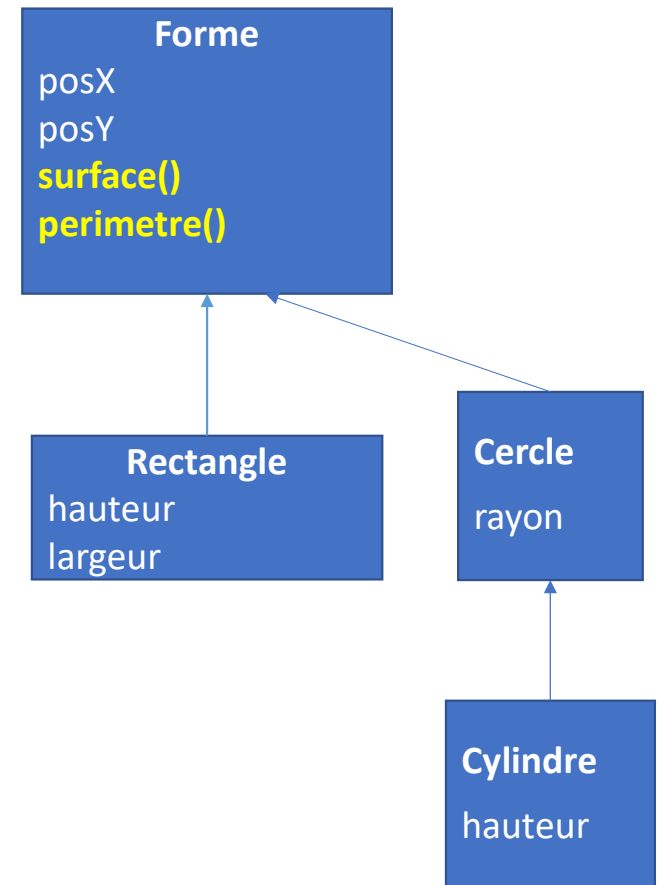


Problématique

Avec l'**héritage**, il est possible de **généraliser** des concepts qui sont présents dans plusieurs classes du programme en les incorporant à une super-classe.

On construit, alors, une structure arborescente reliant différents éléments à un niveau plus abstrait.

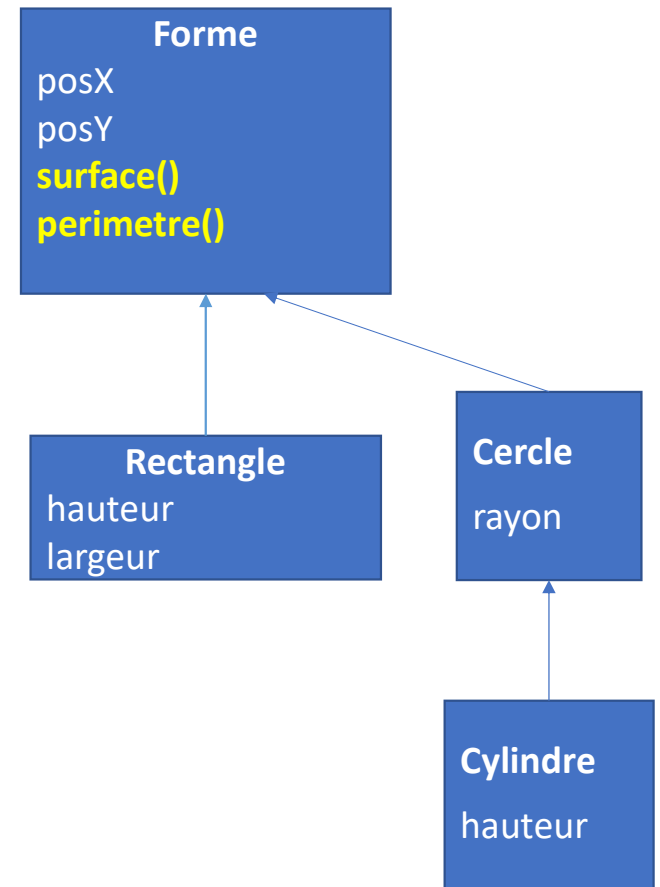
→ fait partie des mécanismes d'**abstraction** de l'orienté objet.



Problématique

Mais, au niveau le plus élevé d'une hiérarchie de classes, il est parfois impossible de définir une méthode générale qui devra pourtant exister dans toutes les sous-classes.

Exemple : *calculer la surface* d'une Forme quelconque se révèle difficile, tandis qu'à un niveau plus bas on peut le mettre en œuvre.



Exemple :

```
public class Forme {
    protected double posX=0;
    protected double posY=0;
    protected static int nbObjets = 0;

    public Forme(){
        setPosX(0); setPosY(0);
        nbObjets++;
    }
    public Forme(double x, double y){
        setPosX(x); setPosY(y);
        nbObjets++;
    }
    public void deplaceDe(double dx, double dy){
        setPosX(posX+dx); setPosY(posY+dy);
    }
    public double surface() {return 0.0;}
    public double perimetre() {return 0.0;}
    public void afficher() {}

    // et les getters et les setters ...
}
```

```
public class Rectangle extends Forme {
    private double largeur = 3.0;
    private double hauteur = 4.0;
    public Rectangle() {
        super();
        setHauteur(0);
        setLargeur(0);
    }
    public Rectangle(double h, double l, double x, double y) {
        super(x,y);
        setHauteur(h);
        setLargeur(l);
    }
    public void coinDS() {
        System.out.println("Le coin sup droit est : " + (posX
        + largeur) + ", " + posY);
    }
    public double surface() {
        return largeur*hauteur;
    }
    public double perimetre() {
        return (largeur+hauteur)*2;
    }
    public void afficher() {
        System.out.println("[Rectangle] largeur = " + largeur
        + " et hauteur = " + hauteur);
    }
}
// et les getters et setters
```

```

public class Cercle extends Forme {
    protected double rayon;
    public Cercle(double x, double y, double r){
        super(x, y);
        setRayon(r);
    }

    public void setRayon(double r)
    {
        rayon = r;
    }
    public double getRayon() {
        return rayon;
    }

    double perimetre(){
        return 2*Math.PI*rayon;
    }

    double surface(){
        return Math.PI*rayon*rayon;
    }
    public void afficher() {
        System.out.println("[Cercle] rayon = "
+ rayon);
    }
    // et les getters et setters
}

```

```

public class Cylindre extends Cercle {
    private double longueur;

    public Cylindre(double x, double y, double rayon, double l)
    {
        super(x,y,rayon);
        setLongueur(l);
    }

    public double surface() {
        return 2 * super.surface() + (2 * Math.PI * getRayon())
* Longueur;
    }

    public double volume()
    {
        return super.surface() * longueur;
    }
    public void afficher() {
        System.out.println("[Cylindre] rayon = " + rayon + " et
Longueur = " + longueur);
    }

    // et les getters et setters
}

```

Problématique

- Une telle méthode **surface()** au niveau de la super-classe **Forme** pourrait être nécessaire pour comparer deux surfaces (qui est générale) ou simplement par souci d'abstraction et d'unification des données.
- On veut comparer la surface de deux formes , mais on ne pourra savoir si c'est un Cercle ou Rectangle qu'en cours d'exécution.
- Le programme ne compilera pas si la méthode **surface** n'est pas déclarée au niveau de la classe **Forme**.

```
public class TestPolymorphisme {
    public static void main(String[] args)
    {
        // déclaration et initialisation de deux Formes
        Forme forme1 = new Cercle(5, 3, 6);
        Forme forme2 = new Rectangle(5, 3, 2, 5);

        System.out.println("Les 2 Formes ont La même surface ? " +
            surfaceCompare(forme1, forme2));

        // afficher le cercle
        afficherForme(forme1);

        // afficher le rectangle
        afficherForme(forme2);
    }

    // méthode pour comparer les surfaces de deux Formes
    static boolean surfaceCompare(Forme objet1, Forme objet2)
    {
        return objet1.surface() == objet2.surface();
    }

    // méthode pour afficher des infos sur la forme géométrique
    static void afficherForme(Forme objet)
    {
        System.out.println();
        objet.afficher();
        System.out.println("La surface est " + objet.surface());
        System.out.println("Le périmètre est " +
            objet.perimetre());
    }
}
```

Solution: Méthode abstraite

- C'est une méthode qui dispose de la **signature** (type de retour et paramètres) mais **sans implémentation**.
- Elle est déclarée avec le modificateur **abstract**

Exemple:

```
public abstract double surface();
```

- La méthode sera implémentée par les classes enfants.



Définition : classes Abstraites

Une classe abstraite est une classe **incomplète**. Elle regroupe un ensemble de variables et de méthodes mais **certaines de ses méthodes ne contiennent pas d'instructions (méthodes abstraites)**

- Elle est déclarée avec le modificateur **abstract**

Exemple:

```
public abstract class Forme
```

Classes Abstraites

- **Règle 1** : Une classe est automatiquement abstraite si une de ses méthodes est abstraite.
- **Règle 2** : Une classe abstraite peut contenir des méthodes non abstraites et des déclarations de variables ordinaires.
- **Règle 3** : On déclare qu'une classe est abstraite avec le mot clé **abstract**.
- **Règle 4** : Une classe abstraite n'est pas instanciable (on ne peut pas utiliser les constructeurs d'une classe abstraite et donc on ne peut pas créer d'objet de cette classe.)
- **Règle 5** : Une classe qui hérite d'une classe abstraite ne devient concrète que si elle implémente toutes les méthodes abstraites de la classe dont elle hérite.

Exemple

```
public abstract class Forme {
    protected double posX=0;
    protected double posY=0;
    protected static int nbObjets = 0;

    public Forme(){
        setPosX(0); setPosY(0); nbObjets++;
    }
    public Forme(double x, double y){
        setPosX(x); setPosY(y); nbObjets++;
    }
    public void deplaceDe(double dx, double dy){
        setPosX(posX+dx); setPosY(posY+dy);
    }
    public abstract double surface();
    public abstract double perimetre();
    public abstract void afficher();

    // et les getters et les setters ...
}
```